5 ระบบผู้แนะนำแบบการกรองร่วม
Collaborative Filtering Recommender System



การกรองร่วม (Collaborative Filtering) แนะนำจากสิ่งที่ผู้ใช้คนอื่นทำ แทนสิ่งที่รายการเป็น บทนี้รวมสองตระกูลหลักไว้ด้วยกัน คือ วิธีเพื่อนบ้านซึ่งเข้าใจง่ายและตรวจสอบได้ทุกขั้น และวิธีปัจจัยแฝงซึ่งเรียนเวกเตอร์ของผู้ใช้และรายการจากข้อมูล

เนื้อหาเดินจากง่ายไปยาก ตั้งแต่ความคล้ายและวิธีเพื่อนบ้าน ค่าเอนเอียง ไปจนถึง MF, ALS, WRMF, BPR และ EASE แล้วปิดด้วยการประเมินผล ความท้าทาย และการใช้ไลบรารี ทุกตัวอย่างใช้ MovieMini-12 ชุดเดียวกับบทที่ 3

เนื้อหา

วัตถุประสงค์ 2
5.1 การกรองร่วมคืออะไร 3
5.2 เมทริกซ์คะแนน 7
5.3 ความคล้าย 14
5.4 User-kNN 20
5.5 Item-kNN 28
5.6 ค่าเอนเอียง 36
5.7 ปัจจัยแฝงและ MF 44
5.8 ALS และ SVD++ 54
5.9 WRMF 59
5.10 BPR 63
5.11 EASE 68
5.12 การสุ่มเชิงลบ 72
5.13 การประเมินผล 76
5.14 ความท้าทาย 83
5.15 แนวปฏิบัติ 88

วัตถุประสงค์สำหรับผู้อ่านประจำบท

เมื่อศึกษาบทนี้จบแล้ว ผู้อ่านควรทำสิ่งต่อไปนี้ได้

- **อธิบาย** แนวคิดของการกรองร่วม (Collaborative Filtering) สองตระกูล คือ วิธีเพื่อนบ้าน (Neighborhood-based) และวิธีปัจจัยแฝง (Latent Factor) และบอกได้ว่าต่างจากระบบอิงเนื้อหาในบทที่ 3–4 อย่างไร
- **อ่านเมทริกซ์คะแนน** และสรุปสถิติพื้นฐาน ได้แก่ ความหนาแน่น ค่าเฉลี่ย จำนวนรายการร่วม และ **โปรไฟล์พฤติกรรมของผู้ใช้**
- **คำนวณความคล้าย** ด้วย Jaccard, Cosine, Pearson และ Adjusted Cosine ด้วยมือ พร้อมอธิบายว่าเหตุใดจึงต้องลบค่าเฉลี่ยและถ่วงนัยสำคัญ
- **ทำนายและจัดอันดับ** ด้วย User-kNN, Item-kNN (แบบคะแนนและแบบโดยปริยาย) แบบจำลองค่าเอนเอียง และ kNN-Baseline โดยแสดงการคำนวณทีละขั้น
- **ฝึกแบบจำลองปัจจัยแฝง** ได้แก่ MF ด้วย SGD, ALS, SVD++, WRMF, BPR และ EASE โดยเข้าใจที่มาของสูตรการปรับค่าในระดับที่คำนวณตามได้
- **อธิบายที่มาของไฮเปอร์พารามิเตอร์** เช่น k, β, λ, η, α ว่ามาจากงานใด ทำหน้าที่อะไร และควรเลือกอย่างไร
- **ประเมินผล** ทั้งการทำนายคะแนนและการจัดอันดับด้วยโปรโตคอลเดียวกับบทที่ 3–4 วิเคราะห์ความท้าทาย และใช้ไลบรารี Surprise และ implicit กับข้อมูลจริงได้

บทนี้ใช้ชุดข้อมูล **MovieMini-12** ชุดเดียวกับบทที่ 1–4 ผู้ใช้หลักยังเป็น u1 ซึ่งชอบ Toy Story 5 (i1), Inside Out 2 (i2) และ The Wild Robot (i3) ภาพยนตร์ที่ u1 ยังไม่ได้ดูยังเป็นเจ็ดเรื่องเดิม (i5, i7, i8, i9, i10, i11, i12) ผลของการกรองร่วมจึงเทียบกับระบบอิงข้อความในบทที่ 3 และระบบหลายรูปแบบในบทที่ 4 ได้ตรงตัว

ตารางที่ 5.1 ความรู้พื้นฐานที่ควรมีก่อนอ่านบทนี้

ความรู้	ใช้ในหัวข้อ	ทบทวนได้ที่
ค่าเฉลี่ย ผลคูณจุด และความคล้ายโคไซน์	5.2–5.5	บทที่ 3 หัวข้อ 3.3
โปรไฟล์ผู้ใช้แบบถ่วงคะแนน ($r - \bar{r}$)	5.2.4, 5.3	บทที่ 4 หัวข้อ 4.8.2
อนุพันธ์อย่างง่าย และการลดตามเกรเดียนต์	5.6–5.10	บทที่ 4 หัวข้อ 4.3.3
MAE, RMSE, HR@K, MRR, nDCG@K และ Paired Bootstrap	5.13	บทที่ 2 หัวข้อ 2.3–2.5 และ 2.10
การเตรียมและทำความสะอาดข้อมูลปฏิสัมพันธ์	5.2, 5.15	บทที่ 1 หัวข้อ 1.13
เมทริกซ์ผกผัน 2×2 และระบบสมการเชิงเส้น	5.8, 5.11	ภาคผนวก ข ของหนังสือ

ลำดับของบทเรียงจาก **ง่ายไปยาก** ช่วงแรก (หัวข้อ 5.1–5.6) เป็นวิธีเพื่อนบ้าน ใช้เพียงการบวก ลบ คูณ หาร และรากที่สอง ช่วงที่สอง (หัวข้อ 5.7–5.11) เป็นวิธีปัจจัยแฝง ใช้อนุพันธ์อย่างง่ายและระบบสมการ 2×2 ช่วงท้าย (หัวข้อ 5.12–5.17) เป็นการประเมินผลและการนำไปใช้จริง ผู้อ่านที่เพิ่งเริ่มควรอ่านกล่อง "อธิบายแบบเข้าใจง่าย" ของแต่ละหัวข้อก่อน แล้วจึงอ่านตัวอย่างการคำนวณ

5.1 การกรองร่วมคืออะไร

5.1.1 จากสิ่งที่รายการเป็น สู่สิ่งที่ผู้คนทำ ดังสรุปในภาพที่ 5.27 ภาพนี้เชื่อมโยงเมทริกซ์คะแนน ความคล้าย และรายการแนะนำให้เห็นเป็นภาพรวม



ภาพที่ 5.27 การกรองร่วมคืออะไร

บทที่ 3 และ 4 สร้างระบบผู้แนะนำจาก **สิ่งที่รายการเป็น** ได้แก่ เรื่องย่อ โปสเตอร์ เพลง และตัวอย่างภาพยนตร์ ระบบรู้ว่า Toy Story 5 เป็นแอนิเมชันครอบครัวที่เกี่ยวกับของเล่น จึงแนะนำเรื่องที่มีเนื้อหาใกล้เคียงกัน

บทนี้เปลี่ยนมุมมองไปใช้ **สิ่งที่ผู้คนทำ** คือคะแนนและการเลือกดูของผู้ใช้จำนวนมาก แนวคิดนี้เรียกว่า **การกรองร่วม (Collaborative Filtering: CF)** คำว่า "ร่วม" หมายถึงผู้ใช้ทุกคนช่วยกันกรองข้อมูลให้กันและกัน โดยไม่จำเป็นต้องรู้ว่ารายการมีเนื้อหาอย่างไรเลย คำนี้มาจากระบบ Tapestry ของ Xerox PARC (Goldberg et al., 1992) และระบบ GroupLens (Resnick et al., 1994)

> **นิยาม 5.1 การกรองร่วม (Collaborative Filtering)**
>
> **การกรองร่วม** คือวิธีทำนายความชอบของผู้ใช้ต่อรายการ โดยใช้ **รูปแบบพฤติกรรมของผู้ใช้จำนวนมาก** ที่บันทึกไว้ในเมทริกซ์ผู้ใช้–รายการ เช่น คะแนน การคลิก หรือการซื้อ ข้อมูลเข้ามีเพียงอย่างเดียวคือเมทริกซ์นี้ ไม่ใช้คุณลักษณะของรายการหรือของผู้ใช้

ตารางที่ 5.2 สิ่งที่สองแนวทางรู้เกี่ยวกับ Toy Story 5 (i1)

คำถาม	ระบบอิงเนื้อหา (บทที่ 3–4)	การกรองร่วม (บทนี้)
i1 คืออะไร	แอนิเมชันครอบครัว เรื่องของเล่น ภาพสดใส	ไม่รู้ รู้เพียงรหัส i1

คำถาม	ระบบอิงเนื้อหา (บทที่ 3–4)	การกรองร่วม (บทนี้)
ใครชอบ i1	ไม่ใช้ข้อมูลนี้ในการหาเรื่องคล้าย	u1 (5) และ u4 (4) ชอบ ส่วน u2 (2) ไม่ชอบ
เรื่องใดคล้าย i1	i2, i3 (เนื้อหาแอนิเมชันคล้ายกัน)	i7, i8 (u4 ชอบทั้งสามเรื่อง)
ภาพยนตร์ใหม่ไม่มีคะแนน	แนะนำได้ทันทีจากเนื้อหา	แนะนำไม่ได้จนกว่าจะมีคนดู
เรื่องที่เนื้อหาต่างแต่คนกลุ่มเดียวกันชอบ	จับไม่ได้	จับได้ (เช่น ความชอบข้ามแนว)

ที่มา: ตารางที่ 5.5 และบทที่ 3 ตารางที่ 3.3

ตารางที่ 5.2 แสดงว่าสองแนวทาง **มองรายการเดียวกันคนละด้าน** ระบบอิงเนื้อหาบอกว่า Toy Story 5 คล้าย Inside Out 2 เพราะเนื้อหาคล้ายกัน ขณะที่การกรองร่วมบอกว่า Toy Story 5 คล้าย Ballerina และ Furiosa เพราะ u4 ชอบทั้งสามเรื่อง ไม่มีด้านใดผิด แต่ละด้านจับความจริงคนละส่วน

อธิบายแบบเข้าใจง่าย ถามเพื่อนที่รสนิยมเหมือนกัน

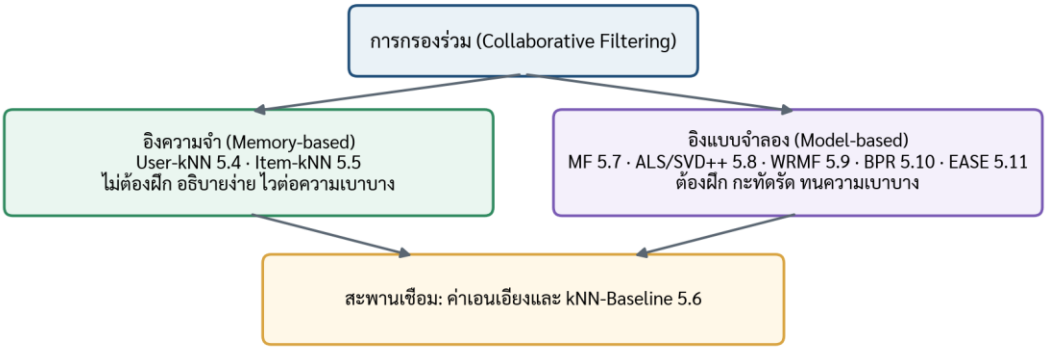
เวลาเลือกหนังเรื่องใหม่ หลายคนจะถามเพื่อนที่ "ชอบหนังแบบเดียวกับเรา" แทนการอ่านเรื่องย่อเอง ถ้าเพื่อนคนนั้นชอบเรื่องไหน เราก็มีแนวโน้มจะชอบด้วย

การกรองร่วมทำสิ่งเดียวกันโดยอัตโนมัติกับผู้ใช้หลายล้านคน ระบบหา "เพื่อนรสนิยมเหมือน" จากคะแนนที่เคยให้ แล้วนำความเห็นของเพื่อนเหล่านั้นมาแนะนำ ระบบไม่ต้องดูหนังเองเลยสักเรื่อง

5.1.2 สมมติฐานและสองตระกูลของการกรองร่วม

การกรองร่วมตั้งอยู่บนสมมติฐานสองข้อ ซึ่งนำไปสู่สองตระกูลของเทคนิค

- **ความชอบมีความคงเส้นคงวา** ผู้ใช้สองคนที่เคยชอบและไม่ชอบภาพยนตร์ชุดเดียวกัน มีแนวโน้มจะเห็นตรงกันในอนาคต สมมติฐานนี้นำไปสู่ **วิธีอิงความจำ (Memory-based)** หรือวิธีเพื่อนบ้าน ซึ่งเก็บคะแนนทั้งหมดไว้ แล้วหาเพื่อนบ้านทุกครั้งที่ทำนาย (หัวข้อ 5.3–5.5)
- **ความชอบมีโครงสร้างแฝง** ความชอบทั้งหมดอธิบายได้ด้วยปัจจัยจำนวนไม่มาก เช่น "ชอบแอนิเมชันครอบครัว" หรือ "ชอบแอ็กชันเร็ว" สมมติฐานนี้นำไปสู่ **วิธีอิงแบบจำลอง (Model-based)** ซึ่งเรียนพารามิเตอร์จำนวนน้อยไว้ก่อน แล้วใช้พารามิเตอร์นั้นทำนาย (หัวข้อ 5.7–5.11)



ภาพที่ 5.1 สองแนวทางของการกรองร่วม

ภาพที่ 5.1 แสดงว่าแบบจำลองค่าเอนเอียงในหัวข้อ 5.6 เป็น **สะพานเชื่อม** ระหว่างสองตระกูล เพราะมีพารามิเตอร์ที่ต้องประมาณเหมือนตระกูลแบบจำลอง แต่คำนวณได้ด้วยสูตรตรง ๆ เหมือนตระกูลความจำ ตารางที่ 5.3 เปรียบเทียบสองตระกูลในเชิงวิศวกรรม

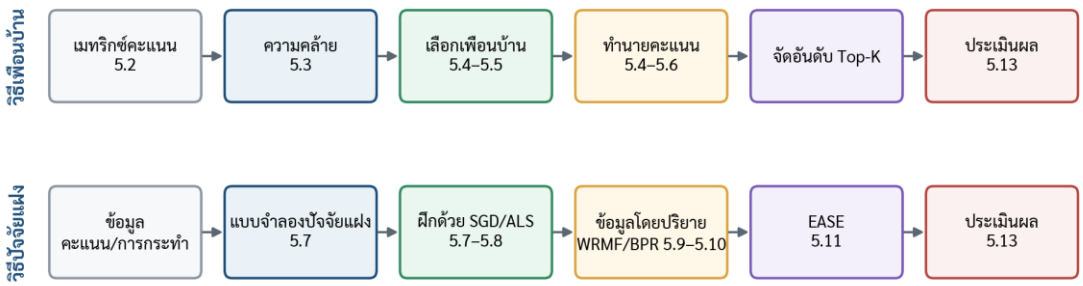
ตารางที่ 5.3 เปรียบเทียบสองตระกูลของการกรองร่วมในเชิงวิศวกรรม

เกณฑ์	อิงความจำ (kNN)	อิงแบบจำลอง (MF และอื่น ๆ)
สิ่งที่ต้องเก็บ	เมทริกซ์คะแนน และตารางความคล้าย	เวกเตอร์ปัจจัยแฝงและค่าเอนเอียง
การฝึก	ไม่มี หรือเพียงคำนวณความคล้ายล่วงหน้า	ต้องฝึกด้วย SGD หรือ ALS
ข้อมูลใหม่	ใช้ได้ทันที	ต้องฝึกใหม่หรือปรับทีละส่วน
ความเบาบาง	อ่อนไหวมาก ต้องมีรายการร่วม	ทนทานกว่า ส่งข้อมูลผ่านปัจจัยแฝง
การอธิบายผล	ง่าย "เพราะคุณชอบ X"	ยากกว่า ปัจจัยแฝงไม่มีชื่อ
การขยายขนาด	คู่ความคล้ายโตเป็นกำลังสอง	พารามิเตอร์โตเชิงเส้น (m + n)k

5.1.3 วิธีคิดของการกรองร่วมในสี่ขั้น

ไม่ว่าจะเป็นเทคนิคใดในบทนี้ การกรองร่วมมีโครงคิดเดียวกันสี่ขั้น ผู้อ่านควรจำโครงนี้ไว้ เพราะทุกหัวข้อต่อจากนี้เป็นเพียงการเปลี่ยนวิธีทำในแต่ละขั้น

1. **ขั้นที่ 1 แทนผู้ใช้ด้วยพฤติกรรม** เช่น แถวคะแนนของผู้ใช้ หรือเวกเตอร์ปัจจัยแฝงที่เรียนจากคะแนน
2. **ขั้นที่ 2 วัดความเชื่อมโยง** ระหว่างผู้ใช้กับผู้ใช้ รายการกับรายการ หรือผู้ใช้กับรายการ
3. **ขั้นที่ 3 ทำนายคะแนน** ของรายการที่ผู้ใช้ยังไม่เคยดู
4. **ขั้นที่ 4 จัดอันดับ** เรียงคะแนนทำนายจากมากไปน้อย แล้วเลือก Top-K ไปแสดง และประเมินผลด้วยการซ่อนคำตอบ



ภาพที่ 5.2 สายงานของวิธีเพื่อนบ้านและปัจจัยแฝง

ภาพที่ 5.2 แสดงสายงานทั้งสองแบบ แถวบนคือวิธีเพื่อนบ้าน ซึ่งเริ่มจากเมทริกซ์คะแนน คำนวณความคล้าย เลือกเพื่อนบ้าน ทำนาย จัดอันดับ และประเมินผล แถวล่างคือวิธีปัจจัยแฝง ซึ่งแทนขั้น "ความคล้ายและเพื่อนบ้าน" ด้วยการ **ฝึกแบบจำลอง** แต่จบด้วยการประเมินผลแบบเดียวกัน ผลของทั้งสองแถวจึงเทียบกันได้ในหัวข้อ 5.13

ตัวอย่างการคำนวณ 5.1 คิดแบบการกรองร่วมด้วยสามัญสำนึก

โจทย์ ควรแนะนำ Conclave (i9) ให้ u1 หรือไม่ โดยยังไม่ใช้สูตรใดเลย

ข้อมูลที่ใช้ ตารางที่ 5.5 ในหัวข้อ 5.2

ขั้นที่ 1 หาคนที่เคยดู i9

- มีเพียง u5 ซึ่งให้ i9 = 5 คะแนน สูงกว่าคะแนนปกติของ u5 เอง

ขั้นที่ 2 ดูว่า u5 รสนิยมเหมือน u1 หรือไม่

- ทั้งสองคนดู The Wild Robot (i3) และ Bridget Jones (i6) ร่วมกัน
- ทั้งสองคนให้ i3 สูงกว่า i6 เหมือนกัน (u1: 4 กับ 1, u5: 4 กับ 3)

ขั้นที่ 3 สรุป

u5 รสนิยมไปทางเดียวกับ u1 และ u5 ชอบ Conclave มาก จึงน่าแนะนำ i9 ให้ u1 แต่หลักฐานมาจากผู้ใช้คนเดียวและรายการร่วมเพียงสองเรื่อง หัวข้อ 5.3–5.4 จะแปลงความคิดนี้เป็นตัวเลขและบอกว่าควรเชื่อหลักฐานนี้มากเพียงใด

5.1.4 การกรองร่วมในระบบจริง

แม้ปัจจุบันจะมีแบบจำลองเชิงลึกจำนวนมาก เทคนิคในบทนี้ยังเป็นส่วนสำคัญของระบบจริง ทั้งในฐานะ **ระบบฐาน (Baseline)** ที่แบบจำลองใหม่ต้องเอาชนะ และในฐานะขั้น **สร้างรายการรายการที่เป็นตัวเลข (Candidate Generation)** ที่เร็วและอธิบายได้

ประเด็นสำคัญ การกรองร่วมในแพลตฟอร์มที่ผู้อ่านคุ้นเคย

- **อีคอมเมิร์ซ** Amazon ใช้ Item-to-Item Collaborative Filtering สร้างแถว "ลูกค้าที่ซื้อสินค้านี้ยังซื้อ..." โดยคำนวณความคล้ายระหว่างสินค้าล่วงหน้าแบบออฟไลน์ (Linden et al., 2003)
- **วิดีโอและภาพยนตร์** การแข่งขัน Netflix Prize (ค.ศ. 2006–2009) ทำให้แบบจำลองปัจจัยแฝงเป็นที่รู้จัก ทีมที่ชนะใช้ MF แบบมีค่าเอนเอียงและ SVD++ เป็นแกนหลัก (Bennett & Lanning, 2007; Koren et al., 2009)
- **เพลง** การแนะนำ "ศิลปินที่คล้ายกัน" มักเริ่มจากการนับผู้ฟังร่วมระหว่างศิลปิน ซึ่งคือ Item-kNN บนข้อมูลโดยปริยายในหัวข้อ 5.5.4 และ MF บนจำนวนครั้งที่ฟังคือแนวคิดของ WRMF ในหัวข้อ 5.9
- **การศึกษา** ระบบแนะนำรายวิชาใช้ข้อมูลว่า "นักศึกษาที่ลงวิชา A มักลงวิชา B ด้วย" ซึ่งเป็น Item-kNN ที่ข้อมูลเป็นการลงทะเบียน ไม่ใช่คะแนน

ทุกหัวข้อของบทจะบอกว่า "ผู้ใช้หนึ่งคน" ถูกแทนด้วยอะไร เพราะโปรไฟล์ผู้ใช้คือหัวใจของการกรองร่วมทุกแบบ ตารางที่ 5.4 สรุปไว้ล่วงหน้า

ตารางที่ 5.4 ผู้ใช้ถูกแทนด้วยอะไรในแต่ละเทคนิคของบทนี้

เทคนิค	โปรไฟล์ของผู้ใช้	หัวข้อ
User-kNN	แถวคะแนนที่ลบค่าเฉลี่ยของผู้ใช้ + รายชื่อเพื่อนบ้าน	5.2.4, 5.4
Item-kNN	รายการที่ผู้ใช้เคยให้คะแนนหรือชอบ (ใช้เป็นจุดอ้างอิง)	5.5
ค่าเอนเอียง / kNN-Baseline	ตัวเลข b_u (ความใจดีหรือความเข้มงวด) + รายการอ้างอิง	5.6

เทคนิค	โปรไฟล์ของผู้ใช้	หัวข้อ
MF / ALS / SVD++	เวกเตอร์ปัจจัยแฝง $\mathbf{p}_u$ + ค่าเอนเอียง	5.7–5.8
WRMF / BPR	เวกเตอร์ปัจจัยแฝงที่เรียนจากการกระทำ	5.9–5.10
EASE	แถวการกระทำ $\mathbf{x}_u$ (ไม่มีพารามิเตอร์ของผู้ใช้)	5.11

5.2 เมทริกซ์คะแนนของ MovieMini-12

5.2.1 เมทริกซ์ผู้ใช้–รายการ ดังสรุปในภาพที่ 5.28 ภาพนี้สรุปความเบาบาง การกระจายคะแนน การลบค่าเฉลี่ย และการแปลงเป็นผลป้อนกลับโดยปริยาย



ภาพที่ 5.28 เมทริกซ์คะแนนและสัญญาณความชอบ

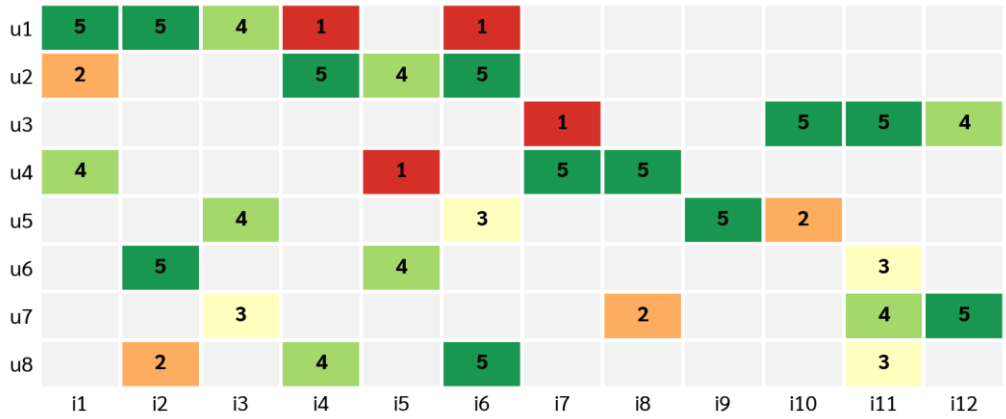
ข้อมูลเข้าเพียงอย่างเดียวของการกรองร่วมคือ **เมทริกซ์ผู้ใช้–รายการ (User–Item Matrix) R** ขนาด $|\mathcal{U}| \times |\mathcal{I}|$ ช่อง r_{ui} คือคะแนนที่ผู้ใช้ u ให้รายการ i

ช่องที่ว่างหมายถึง **ยังไม่มีคะแนน** ไม่ใช่คะแนนศูนย์ ตารางที่ 5.5 และภาพที่ 5.3 แสดงเมทริกซ์ของ MovieMini-12 ซึ่งเป็นข้อมูลชุดเดียวกับบทที่ 3 หัวข้อ 3.2.2 และบทที่ 1 หัวข้อ 1.12

ตารางที่ 5.5 เมทริกซ์คะแนนของ MovieMini-12

ผู้ใช้	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
u1	5	5	4	1		1						
u2	2			5	4	5						
u3							1			5	5	4
u4	4				1		5	5				
u5			4			3			5	2		
u6		5			4						3	
u7			3					2			4	5
u8		2		4		5					3	

ที่มา: บทที่ 3 หัวข้อ 3.2.2 ช่องว่างคือยังไม่มีคะแนน ชื่อเรื่องดูได้จากตารางที่ 3.3



ภาพที่ 5.3 เมทริกซ์คะแนน MovieMini-12

ภาพที่ 5.3 ระบายสีเขียวสำหรับคะแนนสูง สีแดงสำหรับคะแนนต่ำ และสีเทาสำหรับช่องว่าง ผู้อ่านควรสังเกตสองประเด็น

- **ช่องเทามีมากกว่าช่องสี** ผู้ใช้แต่ละคนดูเพียง 3–5 เรื่องจาก 12 เรื่อง
- **มีกลุ่มผู้ใช้ที่ให้คะแนนคล้ายกัน** เช่น u2 กับ u8 ชอบภาพยนตร์ ROMANCE (i4, i6) แต่ไม่ชอบแอนิเมชัน ซึ่งเป็นรสนิยมตรงข้ามกับ u1 ส่วน u3 กับ u7 ชอบ SCI-FI (i11, i12)

5.2.2 สถิติพื้นฐานที่ควรดูก่อนสร้างแบบจำลอง

$$\text{density} = \frac{|\mathcal{K}|}{|\mathcal{U}|\times|\mathcal{I}|}, \qquad \mu = \frac{1}{|\mathcal{K}|}\sum_{(u,i)\in\mathcal{K}} r_{ui} \tag{5.1}$$

โดยที่

- $|\mathcal{K}|$ คือจำนวนคะแนนที่มีอยู่จริง และ $\mathcal{K}$ คือเซตของคู่ (ผู้ใช้, รายการ) ที่มีคะแนน
- $|\mathcal{U}|$ และ $|\mathcal{I}|$ คือจำนวนผู้ใช้และจำนวนรายการ ผลคูณของทั้งสองคือจำนวนช่องทั้งหมดของเมทริกซ์
- r_{ui} คือคะแนนที่ผู้ใช้ u ให้รายการ i และ μ คือค่าเฉลี่ยรวมของคะแนนทั้งหมด

$\mathcal{K}$ คือเซตของคู่ (ผู้ใช้, รายการ) ที่มีคะแนน ความหนาแน่นบอกว่าเมทริกซ์ถูกเติมไปกี่ส่วน ส่วน μ คือค่าเฉลี่ยรวมของทั้งระบบ ซึ่งจะเป็นจุดเริ่มต้นของแบบจำลองค่าเอนเอียงในหัวข้อ 5.6

ตัวอย่างการคำนวณ 5.2 ความหนาแน่น ค่าเฉลี่ยรวม และค่าเฉลี่ยของผู้ใช้

โจทย์ จงหาความหนาแน่น ค่าเฉลี่ยรวม μ ค่าเฉลี่ยของ u1 และค่าเฉลี่ยของ i12

สมการที่ใช้ สมการ (5.1)

ข้อมูลที่ใช้ ตารางที่ 5.5

ขั้นที่ 1 นับจำนวนคะแนน

- $|\mathcal{K}| = 5 + 4 + 4 + 4 + 4 + 3 + 4 + 4 = 32$ (นับตามแถวของ u1 ถึง u8)

ขั้นที่ 2 ความหนาแน่น

- ความหนาแน่น = $32/(8 \times 12) = 32/96 = 0.3333$
- ความเบาบาง = $1 - 0.3333 = 0.6667$

ขั้นที่ 3 ค่าเฉลี่ย

- ผลรวมคะแนนทั้งหมด = 116 จึงได้ $\mu = 116/32 = 3.6250$
- u1: $(5 + 5 + 4 + 1 + 1)/5 = 16/5 = 3.2000$
- i12: $(4 + 5)/2 = 4.5000$ จากผู้ใช้เพียงสองคน (u3 และ u7)

ขั้นที่ 4 ตีความ

ความหนาแน่น 33% สูงกว่าระบบจริงมาก ระบบจริงเช่น Netflix Prize มีความหนาแน่นราว 1% และระบบอีคอมเมิร์ซอาจต่ำกว่า 0.01% (Bennett & Lanning, 2007) แต่แม้ชุดเล็กนี้ภาพยนตร์บางเรื่องก็มีคะแนนเพียงหนึ่งหรือสองค่า ค่าเฉลี่ยของรายการเหล่านั้นจึงไม่น่าเชื่อถือ

ตารางที่ 5.6 ค่าเฉลี่ยและจำนวนคะแนนของผู้ใช้และรายการ

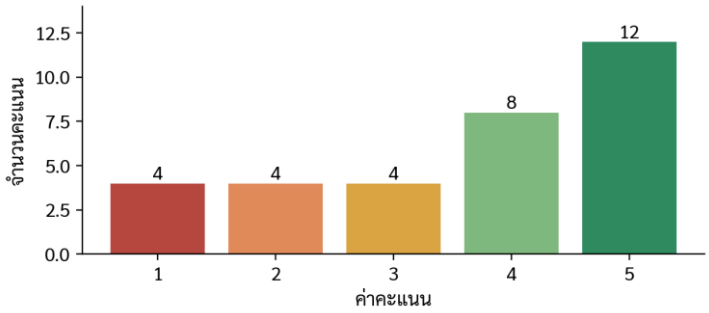
ผู้ใช้	จำนวน	ค่าเฉลี่ย	รายการ	จำนวน	ค่าเฉลี่ย	รายการ	จำนวน	ค่าเฉลี่ย
u1	5	3.2000	i1	3	3.6667	i7	2	3.0000
u2	4	4.0000	i2	3	4.0000	i8	2	3.5000
u3	4	3.7500	i3	3	3.6667	i9	1	5.0000
u4	4	3.7500	i4	3	3.3333	i10	2	3.5000
u5	4	3.5000	i5	3	3.0000	i11	4	3.7500
u6	3	4.0000	i6	4	3.5000	i12	2	4.5000
u7	4	3.5000						
u8	4	3.5000						

ที่มา: คำนวณจากตารางที่ 5.5

ตารางที่ 5.6 ให้ข้อสังเกตที่จะใช้ตลอดบท

- **ผู้ใช้ใช้มาตราคะแนนต่างกัน** u1 มีค่าเฉลี่ย 3.20 ขณะที่ u2 และ u6 มีค่าเฉลี่ย 4.00 คะแนน 4 ของ u1 จึงหมายถึง "ชอบมากกว่าปกติ" แต่คะแนน 4 ของ u2 หมายถึง "ปกติ"

- **รายการมีหลักฐานไม่เท่ากัน** i6 และ i11 มีผู้ให้คะแนนสี่คน แต่ i9 มีเพียงหนึ่งคน ค่าเฉลี่ย 5.00 ของ i9 จึงไม่ได้แปลว่าเป็นภาพยนตร์ที่ดีที่สุด



ภาพที่ 5.4 การกระจายคะแนนของ MovieMini-12

ภาพที่ 5.4 แสดงว่าคะแนน 5 มี 12 ค่า คะแนน 4 มี 8 ค่า และคะแนน 1, 2 และ 3 มีอย่างละ 4 ค่า ผู้ใช้ในชุดนี้จึงให้คะแนน **ค่อนไปทางสุดขั้ว** คือชอบมากหรือไม่ชอบเลย ลักษณะนี้ทำให้ความคลาดเคลื่อนของการทำนายสูงมากเมื่อระบบทายผิดฝั่ง ดังที่จะเห็นในหัวข้อ 5.13.1

5.2.3 จำนวนรายการร่วมของผู้ใช้แต่ละคู่

วิธีอิงความจำเทียบผู้ใช้สองคนจาก **รายการที่ทั้งคู่ให้คะแนนร่วมกัน** เท่านั้น จำนวนรายการร่วมจึงบอกว่า "มีหลักฐานให้เทียบมากเพียงใด" ตารางที่ 5.7 นับจำนวนนี้ของผู้ใช้ทุกคู่ ค่าบนแนวทแยงคือจำนวนเรื่องที่ผู้ใช้คนนั้นให้คะแนน

ตารางที่ 5.7 จำนวนรายการที่ผู้ใช้แต่ละคู่ให้คะแนนร่วมกัน

ผู้ใช้	u1	u2	u3	u4	u5	u6	u7	u8
u1	5	3	0	1	2	1	1	3
u2	3	4	0	2	1	1	0	2
u3	0	0	4	1	1	1	2	1
u4	1	2	1	4	0	1	1	0
u5	2	1	1	0	4	0	1	1
u6	1	1	1	1	0	3	1	2
u7	1	0	2	1	1	1	4	1
u8	3	2	1	0	1	2	1	4

ที่มา: นับจากตารางที่ 5.5

ตัวอย่างการคำนวณ 5.3 นับรายการร่วมของ u1

โจทย์ จงหาจำนวนรายการร่วมของ u1 กับ u2, u3 และ u5 แล้วสรุปภาพรวมของผู้ใช้ทั้ง 28 คู่

สมการที่ใช้ จำนวนรายการร่วม $|\mathcal{I}_{uv}| = |\mathcal{I}_u \cap \mathcal{I}_v|$

ข้อมูลที่ใช้ ตารางที่ 5.5 และ 5.7

- u1 ดู {i1, i2, i3, i4, i6} และ u2 ดู {i1, i4, i5, i6} → ร่วมกัน {i1, i4, i6} = 3 เรื่อง
- u3 ดู {i7, i10, i11, i12} → ไม่มีเรื่องร่วมกับ u1 เลย (0 เรื่อง)

- u5 ดู {i3, i6, i9, i10} → ร่วมกัน {i3, i6} = 2 เรื่อง
- ผู้ใช้ 8 คนมี $8 \times 7/2 = 28$ คู่ แบ่งเป็น 0 เรื่อง 6 คู่, 1 เรื่อง 15 คู่, 2 เรื่อง 5 คู่ และ 3 เรื่อง 2 คู่

ตีความ ผู้ใช้ 21 คู่จาก 28 คู่ (75%) มีรายการร่วมไม่เกินหนึ่งเรื่อง แม้ความหนาแน่นจะสูงถึง 33% นี่คือเหตุผลที่หัวข้อ 5.3.3 ต้องลดความเชื่อถือของความคล้ายที่คำนวณจากหลักฐานน้อย

5.2.4 โปรไฟล์ผู้ใช้เชิงพฤติกรรม

ในการกรองร่วม โปรไฟล์ของผู้ใช้ไม่ใช่เวกเตอร์ของเนื้อหา แต่เป็น **แถวของเมทริกซ์คะแนน** ปัญหาคือผู้ใช้แต่ละคนใช้มาตราคะแนนต่างกัน บางคนใจดีให้ 4–5 เกือบทุกเรื่อง บางคนเข้มงวดให้ 3 กับเรื่องที่ชอบแล้ว

ทางแก้คือ **ลบค่าเฉลี่ยของผู้ใช้ (Mean-centering)** ก่อน เพื่อให้คะแนนบอก "ชอบมากกว่าหรือน้อยกว่าปกติของตัวเอง" ค่าบวกแปลว่าชอบมากกว่าปกติ ค่าลบแปลว่าชอบน้อยกว่าปกติ

$$\tilde{r}_{ui} = r_{ui} - \bar{r}_u, \qquad \bar{r}_u = \frac{1}{|\mathcal{I}_u|} \sum_{i \in \mathcal{I}_u} r_{ui} \tag{5.2}$$

โดยที่

- r_{ui} คือคะแนนดิบที่ผู้ใช้ u ให้รายการ i
- $\bar{r}_u$ คือค่าเฉลี่ยคะแนนของผู้ใช้ u คิดจากทุกเรื่องที่ u ให้คะแนน
- $\mathcal{I}_u$ คือเซตของรายการที่ผู้ใช้ u ให้คะแนน และ $|\mathcal{I}_u|$ คือจำนวนรายการในเซตนั้น
- $\tilde{r}_{ui}$ คือคะแนนหลังลบค่าเฉลี่ย ค่าบวกแปลว่าชอบมากกว่าปกติของตัวเอง ค่าลบแปลว่าชอบน้อยกว่าปกติ

ตัวอย่างการคำนวณ 5.4 โปรไฟล์เชิงพฤติกรรมของ u1, u2 และ u5

โจทย์ จงหาแถวคะแนนหลังลบค่าเฉลี่ยของ u1, u2 และ u5

สมการที่ใช้ สมการ (5.2)

ข้อมูลที่ใช้ ค่าเฉลี่ยของผู้ใช้จากตารางที่ 5.6

u1 ($\bar{r} = 3.20$)

- i1: $5 - 3.20 = +1.80$, i2: $5 - 3.20 = +1.80$, i3: $4 - 3.20 = +0.80$, i4: $1 - 3.20 = -2.20$, i6: $1 - 3.20 = -2.20$

u2 ($\bar{r} = 4.00$)

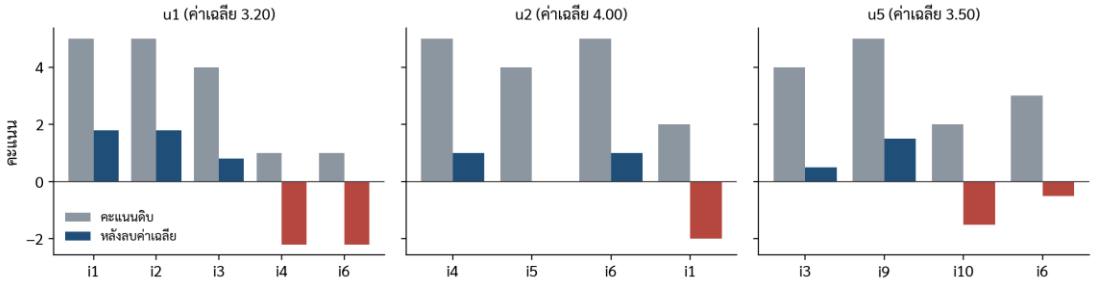
- i4: $5 - 4.00 = +1.00$, i5: $4 - 4.00 = +0.00$, i6: $5 - 4.00 = +1.00$, i1: $2 - 4.00 = -2.00$

u5 ($\bar{r} = 3.50$)

- i3: $4 - 3.50 = +0.50$, i9: $5 - 3.50 = +1.50$, i10: $2 - 3.50 = -1.50$, i6: $3 - 3.50 = -0.50$

ตีความ

- u1 กับ u2 ให้เครื่องหมายตรงข้ามกันทุกเรื่องที่ดูร่วมกัน i1 บวกสำหรับ u1 แต่ลบสำหรับ u2 ส่วน i4 และ i6 กลับกัน จึงเป็น **รสนิยมตรงข้าม**
- u5 ให้ i3 สูงกว่าปกติและ i6 ต่ำกว่าปกติเหมือน u1 จึงเป็น **รสนิยมเหมือนกัน** ในเรื่องที่ดูร่วมกัน
- โปรไฟล์นี้คือโปรไฟล์ถ่วงคะแนนของบทที่ 4 หัวข้อ 4.8.2 ที่ยังไม่นำไปคูณกับเวกเตอร์เนื้อหา



ภาพที่ 5.5 คะแนนก่อนและหลังลบค่าเฉลี่ย

ภาพที่ 5.5 เทียบแท่งสีเทา (คะแนนดิบ) กับแท่งสีน้ำเงินและสีแดง (หลังลบค่าเฉลี่ย) คะแนนดิบของทุกคนเป็นบวกทั้งหมด จึงมองไม่ออกว่าใครชอบหรือไม่ชอบเรื่องใด หลังลบค่าเฉลี่ยแล้ว แท่งสีแดงชี้ลงชัดเจน เช่น i4 และ i6 ของ u1 หรือ i1 ของ u2

อธิบายแบบเข้าใจง่าย ลบค่าเฉลี่ยเหมือนปรับไม้บรรทัดให้ตรงกัน

ถ้าครูสองคนตรวจข้อสอบ ครูใจดีให้คะแนนเฉลี่ย 80 ครูเข้มงวดให้คะแนนเฉลี่ย 60 นักเรียนที่ได้ 75 จากครูใจดีอาจทำได้แย่กว่านักเรียนที่ได้ 70 จากครูเข้มงวด

การลบค่าเฉลี่ยคือการถามว่า "ได้มากกว่าค่าปกติของครูคนนั้นเท่าใด" คะแนนหนังก็เช่นกัน คะแนน 4 ของ u1 (+0.8) แสดงความชอบชัดกว่าคะแนน 4 ของ u2 (0.0)

5.2.5 ผลป้อนกลับชัดแจ้งและโดยปริยาย

คะแนน 1–5 เป็น **ผลป้อนกลับชัดแจ้ง (Explicit Feedback)** เพราะผู้ใช้บอกความชอบตรง ๆ ระบบจริงส่วนใหญ่มีแต่ **ผลป้อนกลับโดยปริยาย (Implicit Feedback)** เช่น การคลิก การดูจนจบ หรือการซื้อ ซึ่งบอกว่าผู้ใช้ "สนใจ" แต่ไม่บอกว่า "ไม่ชอบ" อะไร (Hu et al., 2008)

หัวข้อ 5.3–5.8 ใช้คะแนนชัดแจ้ง ส่วนหัวข้อ 5.9–5.12 แปลงคะแนนเป็นข้อมูลโดยปริยายด้วยกติกาเดียวกับบทที่ 3–4 ดังสมการ (5.3)

$$x_{ui} = \begin{cases} 1 & \text{if } r_{ui} \geq 4 \\ 0 & \text{otherwise} \end{cases} \tag{5.3}$$

โดยที่

- x_{ui} คือค่าความชอบแบบโดยปริยายของผู้ใช้ u ต่อรายการ i มีค่า 1 หรือ 0 เท่านั้น
- $r_{ui} \geq 4$ คือเกณฑ์ "ชอบ" ที่ใช้ร่วมกับบทที่ 3–4 ช่องที่ไม่มีคะแนนหรือมีคะแนนต่ำกว่า 4 ได้ 0

ตารางที่ 5.8 เมทริกซ์การชอบ X ที่ได้จากสมการ (5.3)

ผู้ใช้	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
u1	1	1	1	.	.	.	.	.	.	.	.	.
u2	.	.	.	1	1	1	.	.	.	.	.	.
u3	.	.	.	.	.	.	.	.	.	1	1	1
u4	1	.	.	.	.	.	1	1	.	.	.	.
u5	.	.	1	.	.	.	.	.	1	.	.	.

ผู้ใช้	i1	i2	i3	i4	i5	i6	i7	i8	i9	i10	i11	i12
u6	·	1	·	·	1	·	·	·	·	·	·	·
u7	·	·	·	·	·	·	·	·	·	·	1	1
u8	·	·	·	1	·	1	·	·	·	·	·	·

ที่มา: แปลงจากตารางที่ 5.5 จุด (·) คือ 0

ตารางที่ 5.8 มีค่า 1 ทั้งหมด 20 ช่อง ค่า 1 เปรียบเหมือนการดูจนจบหรือกดถูกใจ ส่วนค่า 0 **ปนกันสองความหมาย** คือ "ไม่ชอบ" (เช่น u1 กับ i4) และ "ยังไม่เคยเห็น" (เช่น u1 กับ i9) ระบบแยกสองความหมายนี้ไม่ได้ ซึ่งเป็นปัญหาหลักของข้อมูลโดยปริยาย

ข้อควรระวัง ช่องว่างไม่ใช่คะแนนศูนย์

ถ้าเติมช่องว่างของ u1 ด้วย 0 แล้วหาค่าเฉลี่ย จะได้ $16/12 = 1.3333$ ซึ่งบอกผิดว่า u1 ไม่ชอบเกือบทุกเรื่อง ค่าที่ถูกคือ $16/5 = 3.2$

ทุกสูตรของคะแนนชัดแจ้งในบทนี้จึงรวมเฉพาะช่องที่มีคะแนนจริง สัญลักษณ์ $\mathcal{I}_u$ (รายการที่ u ให้คะแนน) และ $\mathcal{U}_i$ (ผู้ใช้ที่ให้คะแนน i) มีไว้เพื่อการนี้

โปรแกรม 5.1 สถิติพื้นฐาน จำนวนรายการร่วม และโปรไฟล์หลังลบค่าเฉลี่ย

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
U = list(R)
I = ['i%d' % k for k in range(1, 13)]

# 1) จำนวนคะแนน ความหนาแน่น และค่าเฉลี่ยรวม
ratings = [r for d in R.values() for r in d.values()]
print('จำนวน', len(ratings))
print('ความหนาแน่น', round(len(ratings) / (len(U) * len(I)), 4))
print('mu', np.mean(ratings))

# 2) จำนวนรายการร่วมของผู้ใช้ทุกคู่
co = []
for a, u in enumerate(U):
    for v in U[a + 1:]:
        co.append(len(set(R[u]) & set(R[v])))
print('คู่ที่มีรายการร่วม 0-3 เรื่อง', [co.count(n) for n in range(4)])

# 3) โปรไฟล์หลังลบค่าเฉลี่ยของผู้ใช้
mean = {u: np.mean(list(d.values())) for u, d in R.items()}
for u in ('u1', 'u2', 'u5'):
    prof = {i: round(float(r - mean[u]), 2) for i, r in R[u].items()}
```

```
    print(u, prof)


# 4) เมทริกซ์การชอบ X (คะแนน >= 4 เป็น 1)
X = np.array([[1 if R[u].get(i, 0) >= 4 else 0 for i in I]
              for u in U])
print('จำนวนช่องที่เป็น 1 ใน X =', X.sum())
```

5.3 ความคล้ายระหว่างผู้ใช้

วิธีอิงความจำต้องตอบคำถามว่า "ผู้ใช้สองคนมีรสนิยมคล้ายกันเพียงใด" คำตอบมาจากการเทียบพฤติกรรมบน **รายการที่ทั้งสองคนให้คะแนนร่วมกัน** $\mathcal{I}_{uv} = \mathcal{I}_u \cap \mathcal{I}_v$ หัวข้อนี้ไล่จากมาตรวัดที่ง่ายที่สุด (Jaccard) ไปยังมาตรวัดที่นิยมที่สุด (Pearson) แล้วจึงแก้ปัญหาหลักฐานน้อยด้วยการถ่วงนัยสำคัญ ดังสรุปในภาพที่ 5.29 ภาพนี้สรุปการใช้รายการร่วม มุมระหว่างเวกเตอร์ สหสัมพันธ์ และการถ่วงตามปริมาณหลักฐาน



ภาพที่ 5.29 การวัดความคล้ายระหว่างผู้ใช้

5.3.1 Jaccard และ Cosine: สองมาตรวัดเริ่มต้น

$$\text{Jaccard}(u,v) = \frac{|\mathcal{I}_u \cap \mathcal{I}_v|}{|\mathcal{I}_u \cup \mathcal{I}_v|}, \qquad \cos(u,v) = \frac{\sum_{i \in \mathcal{I}_{uv}} r_{ui}\, r_{vi}}{\sqrt{\sum_{i \in \mathcal{I}_{uv}} r_{ui}^2}\sqrt{\sum_{i \in \mathcal{I}_{uv}} r_{vi}^2}} \tag{5.4}$$

โดยที่

- $\mathcal{I}_u \cap \mathcal{I}_v$ คือรายการที่ผู้ใช้ u และ v ให้คะแนนทั้งคู่ (ส่วนร่วม) เขียนย่อว่า $\mathcal{I}_{uv}$
- $\mathcal{I}_u \cup \mathcal{I}_v$ คือรายการที่อย่างน้อยหนึ่งคนให้คะแนน (ส่วนรวม)
- r_{ui} และ r_{vi} คือคะแนนดิบของสองคนต่อรายการ i เดียวกัน ตัวเศษของ Cosine คือผลคูณจุด ตัวส่วนคือผลคูณของขนาดเวกเตอร์

- **Jaccard** ดูเพียงว่า "ดูเรื่องเดียวกันมากเพียงใด" ไม่สนใจคะแนนเลย จึงเหมาะกับข้อมูลโดยปริยาย เช่น การคลิกหรือการซื้อ
- **Cosine บนคะแนนดิบ** ดูทั้งเรื่องที่ดูร่วมและค่าคะแนน โดยวัดมุมระหว่างเวกเตอร์คะแนนสองเวกเตอร์ เช่นเดียวกับความคล้ายของเวกเตอร์ข้อความในบทที่ 3 หัวข้อ 3.3

ตัวอย่างการคำนวณ 5.5 Jaccard และ Cosine ระหว่าง u1 กับ u2

โจทย์ จงหา Jaccard และ Cosine บนคะแนนดิบของคู่ u1–u2

สมการที่ใช้ สมการ (5.4)

ข้อมูลที่ใช้ ตารางที่ 5.5 และตัวอย่างการคำนวณ 5.3

ขั้นที่ 1 Jaccard

- ส่วนร่วม {i1, i4, i6} = 3 เรื่อง
- ส่วนรวม {i1, i2, i3, i4, i5, i6} = 6 เรื่อง
- $\text{Jaccard} = 3/6 = 0.5000$

ขั้นที่ 2 Cosine บนรายการร่วม (เรียง i1, i4, i6)

- u1: (5, 1, 1) และ u2: (2, 5, 5)
- ผลคูณจุด: $5 \times 2 + 1 \times 5 + 1 \times 5 = 20$
- ขนาด: $\sqrt{25 + 1 + 1} = 5.1962$ และ $\sqrt{4 + 25 + 25} = 7.3485$
- $\cos = 20/(5.1962 \times 7.3485) = 20/38.1838 = 0.5238$

ขั้นที่ 3 ตีความ

ทั้งสองมาตรวัดบอกว่า u1 กับ u2 "คล้ายกันพอสมควร" (0.50 และ 0.52) แต่ตัวอย่างการคำนวณ 5.4 แสดงแล้วว่าทั้งสองคนมีรสนิยมตรงข้ามกัน สาเหตุคือคะแนน 1–5 เป็นบวกทุกค่า เวกเตอร์คะแนนดิบจึงชี้ไปทางเดียวกันเสมอ ปัญหานี้เหมือนกับเวกเตอร์ภาพดิบในบทที่ 4 หัวข้อ 4.5.6

5.3.2 สหสัมพันธ์เพียร์สัน

ทางแก้ของปัญหาในหัวข้อ 5.3.1 คือ **ลบค่าเฉลี่ยของผู้ใช้ก่อน** แล้วจึงหาโคไซน์ ผลที่ได้คือ **สหสัมพันธ์เพียร์สัน (Pearson Correlation)** ซึ่งเป็นมาตรวัดหลักของ User-kNN (Resnick et al., 1994)

$$\text{sim}(u, v) = \frac{\sum_{i \in \mathcal{I}_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in \mathcal{I}_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in \mathcal{I}_{uv}} (r_{vi} - \bar{r}_v)^2}} \tag{5.5}$$

โดยที่

- $\mathcal{I}_{uv}$ คือรายการที่ผู้ใช้ u และ v ให้คะแนนร่วมกัน
- $r_{ui} - \bar{r}_u$ และ $r_{vi} - \bar{r}_v$ คือคะแนนหลังลบค่าเฉลี่ยของแต่ละคน (สมการ (5.2))
- $\text{sim}(u, v)$ คือความคล้ายมีค่าตั้งแต่ -1 (รสนิยมตรงข้าม) ถึง $+1$ (รสนิยมเหมือนกัน)

อ่านสมการ (5.5) ได้เป็นสามส่วน

- **ตัวเศษ** คูณ "ความเบี่ยงเบนจากปกติ" ของสองคนในแต่ละเรื่องแล้วบวกกัน ถ้าทั้งคู่ชอบมากกว่าปกติ (บวก × บวก) หรือไม่ชอบเหมือนกัน (ลบ × ลบ) ผลคูณเป็นบวก
- **ตัวส่วน** คือขนาดของเวกเตอร์ความเบี่ยงเบนของแต่ละคน ทำให้ผลลัพธ์อยู่ในช่วง -1 ถึง $+1$

- **ค่าเฉลี่ย** $\bar{r}_u$ คิดจากทุกเรื่องที่ผู้ใช้ให้คะแนน ไม่ใช่เฉพาะเรื่องร่วม ตามแนวทางของ Aggarwal (2016)

ตัวอย่างการคำนวณ 5.6 Pearson ระหว่าง u1 กับ u5 และระหว่าง u1 กับ u2

โจทย์ จงหาความคล้าย Pearson ของสองคู่

สมการที่ใช้ สมการ (5.5)

ข้อมูลที่ใช้ คะแนนหลังลบค่าเฉลี่ยจากตัวอย่างการคำนวณ 5.4

ขั้นที่ 1 u1 กับ u5 บนรายการร่วม {i3, i6}

- u1 หลังลบค่าเฉลี่ย: (0.8, −2.2) และ u5: (0.5, −0.5)
- ตัวเศษ: $(0.8)(0.5) + (-2.2)(-0.5) = 0.4 + 1.1 = 1.5$
- ตัวส่วน: $\sqrt{0.64 + 4.84} \times \sqrt{0.25 + 0.25} = 2.3409 \times 0.7071 = 1.6553$
- $\text{sim}(u1, u5) = 1.5/1.6553 = 0.9062$

ขั้นที่ 2 u1 กับ u2 บนรายการร่วม {i1, i4, i6}

- u1: (1.8, −2.2, −2.2) และ u2: (−2.0, 1.0, 1.0)
- ตัวเศษ: $(1.8)(-2) + (-2.2)(1) + (-2.2)(1) = -3.6 - 2.2 - 2.2 = -8.0$
- ตัวส่วน: $\sqrt{3.24 + 4.84 + 4.84} \times \sqrt{4 + 1 + 1} = 3.5944 \times 2.4495 = 8.8045$
- $\text{sim}(u1, u2) = -8.0/8.8045 = -0.9086$

ขั้นที่ 3 ตีความ

- u5 มีรสนิยมคล้าย u1 มาก (+0.9062) ส่วน u2 มีรสนิยมตรงข้ามเกือบพอดี (−0.9086) ต่างจาก Cosine บนคะแนนดิบที่ให้ +0.5238
- ข้อมูลมีประโยชน์ทั้งสองทาง เพื่อนที่เหมือนกันช่วยบอกว่าควรแนะนำอะไร เพื่อนที่ตรงข้ามช่วยบอกว่าควรหลีกเลี่ยงอะไร
- แต่ผลของ u1–u5 มาจากรายการร่วมเพียงสองเรื่อง จึงยังไม่น่าเชื่อถือมาก

อธิบายแบบเข้าใจง่าย Pearson ถามว่า "ขึ้นลงพร้อมกันหรือไม่"

ลองนึกถึงหุ้นสองตัว ราคาของหุ้นตัวหนึ่งอยู่ราว 100 บาท อีกตัวอยู่ราว 10 บาท ราคาต่างกันมาก แต่ถ้าวันไหนตัวหนึ่งขึ้น อีกตัวก็ขึ้น วันไหนตัวหนึ่งลง อีกตัวก็ลง เราถือว่าหุ้นสองตัวนี้ "เคลื่อนไหวไปด้วยกัน"

Pearson ไม่สนใจระดับคะแนน (ราคา) สนใจเพียงว่าเมื่อเทียบกับปกติของตัวเอง สองคนชอบมากขึ้นหรือน้อยลง "พร้อมกัน" หรือไม่

ข้อควรระวัง Pearson ใช้ไม่ได้ดีเมื่อรายการร่วมน้อยหรือคะแนนไม่แปรผัน

- ถ้ามีรายการร่วมเพียงเรื่องเดียว ค่าจะเป็น +1 หรือ −1 เสมอ เช่น u1–u4 (ร่วมกันเฉพาะ i1) ได้ +1.0000 ทั้งที่หลักฐานมีชิ้นเดียว
- ถ้าผู้ใช้คนหนึ่งให้คะแนนเท่ากับค่าเฉลี่ยของตัวเองในทุกเรื่องร่วม ตัวส่วนเป็นศูนย์ ระบบต้องกำหนดให้ความคล้ายเป็น 0
- ถ้าไม่มีรายการร่วมเลย เช่น u1–u3 ความคล้ายไม่นิยาม บทนี้กำหนดให้เป็น 0

5.3.3 การถ่วงนัยสำคัญเมื่อรายการร่วมมีน้อย

ข้อควรระวังข้างต้นนำไปสู่ปัญหาสำคัญ ผู้ใช้ที่บังเอิญเห็นตรงกันในเรื่องเดียวจะได้ความคล้ายสูงสุด และอาจกลายเป็นเพื่อนบ้านที่มีน้ำหนักมากที่สุด วิธีแก้คือ **ถ่วงนัยสำคัญ (Significance Weighting)** คือลดค่าความคล้ายตามจำนวนรายการร่วม (Herlocker et al., 1999)

$$\mathrm{sim}'(u,v) = \frac{\min(|\mathcal{I}_{uv}|,\beta)}{\beta}\,\mathrm{sim}(u,v) \tag{5.6}$$

โดยที่

- $\mathrm{sim}(u,v)$ คือความคล้าย Pearson จากสมการ (5.5) และ $\mathrm{sim}'(u,v)$ คือความคล้ายหลังถ่วง
- $|\mathcal{I}_{uv}|$ คือจำนวนรายการร่วมของสองคน
- β คือจำนวนรายการร่วมที่ถือว่าเชื่อถือได้เต็มที่ บทนี้ใช้ $\beta = 3$ และ $\min(\cdot)$ คือการเลือกค่าที่น้อยกว่า

สมการ (5.6) คูณความคล้ายด้วยตัวคูณระหว่าง 0 ถึง 1 ถ้ารายการร่วมมีตั้งแต่ β เรื่องขึ้นไป ตัวคูณเป็น 1 (เชื่อเต็มที่) ถ้ามีน้อยกว่า ตัวคูณจะลดลงตามสัดส่วน

ตัวอย่างการคำนวณ 5.7 ถ่วงนัยสำคัญด้วย $\beta = 3$

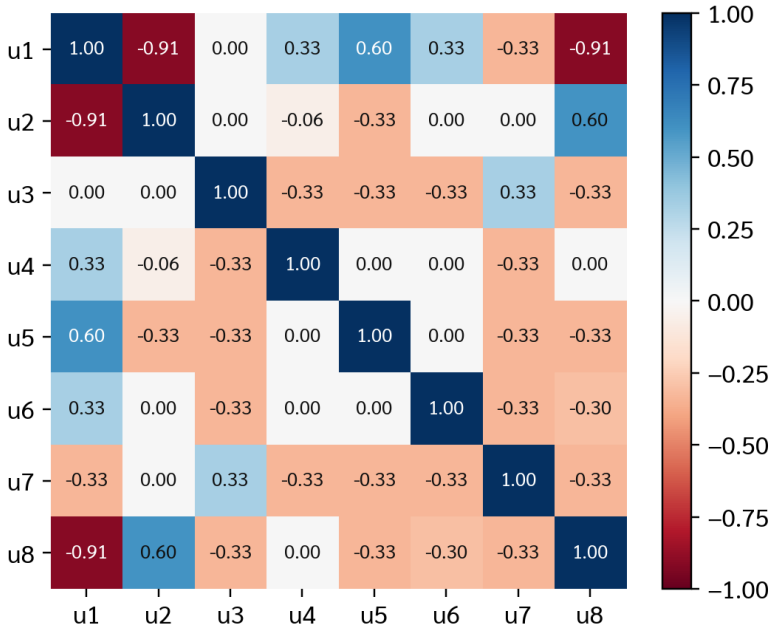
โจทย์ จงหาความคล้ายหลังถ่วงของ u1 กับ u4, u5 และ u2

สมการที่ใช้ สมการ (5.6) ที่ $\beta = 3$

ข้อมูลที่ใช้ Pearson จากตัวอย่างการคำนวณ 5.6 และจำนวนรายการร่วมจากตารางที่ 5.7

- u1–u4 (รายการร่วม 1 เรื่อง): $\frac{\min(1,3)}{3}(1.0000) = \frac{1}{3} = 0.3333$
- u1–u5 (2 เรื่อง): $\frac{2}{3}(0.9062) = 0.6041$
- u1–u2 (3 เรื่อง): $\frac{3}{3}(-0.9086) = -0.9086$ ไม่ถูกลด

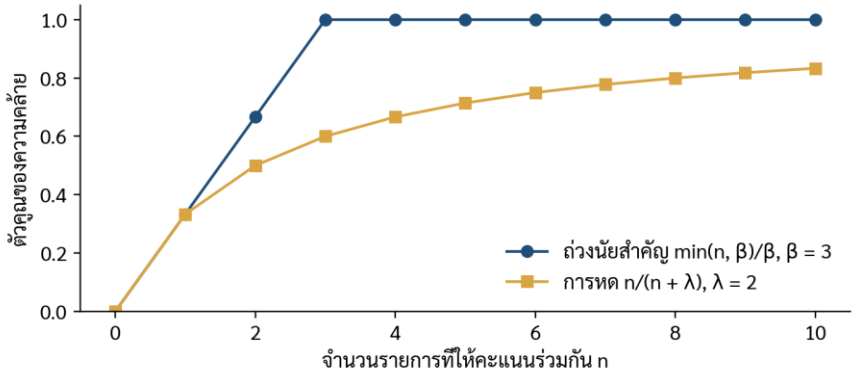
ตีความ หลังถ่วงแล้ว u5 ยังเป็นเพื่อนบ้านที่ดีที่สุดของ u1 แต่ u4 กับ u6 ซึ่งมีรายการร่วมเพียงเรื่องเดียวถูกลดจาก 1.0000 เหลือ 0.3333 ความคล้ายที่มาจากหลักฐานน้อยจึงไม่ครอบงำคำทำนายอีกต่อไป



ภาพที่ 5.6 ความคล้าย Pearson หลังถ่วงนัยสำคัญ

ภาพที่ 5.6 แสดงความคล้ายหลังถ่วงของทุกคู่ สีน้ำเงินคือรสนิยมเหมือน สีแดงคือรสนิยมตรงข้าม

- u1 มีเพื่อนบ้านบวกสามคน (u4, u5, u6) และเพื่อนบ้านลบแรงสองคน (u2, u8)
- u3 ไม่มีรายการร่วมกับ u1 เลย จึงได้ 0
- คู่ u2–u8 (รสนิยม ROMANCE เหมือนกัน) และ u3–u7 (SCI-FI) มีความคล้ายบวกสูง ตรงกับกลุ่มที่สังเกตได้จากภาพที่ 5.3



ภาพที่ 5.7 การถ่วงตามจำนวนรายการร่วม

ภาพที่ 5.7 เทียบตัวคูณสองแบบ เส้นสีน้ำเงินคือการถ่วงนัยสำคัญที่ $\beta = 3$ ซึ่งเพิ่มเป็นเส้นตรงแล้วคงที่ที่ 1 ตั้งแต่ n = 3 เส้นสีเหลืองคือ **การหด (Shrinkage)** $n/(n+\lambda)$ ที่ $\lambda = 2$ ซึ่งเพิ่มอย่างนุ่มนวลและเข้าใกล้ 1 โดยไม่ถึง 1 SHRINKAGE รูปแบบเดียวกันนี้จะกลับมาอีกครั้งในแบบจำลองค่าเอนเอียง (หัวข้อ 5.6)

ที่มาของไฮเปอร์พารามิเตอร์ β ของการถ่วงนัยสำคัญ และ λ ของ SHRINKAGE

- **β มาจากไหน** Herlocker et al. (1999) ทดลองบนข้อมูล MovieLens ซึ่งผู้ใช้มีคะแนนหลายสิบถึงหลายร้อยเรื่อง แล้วพบว่า β = 50 ให้ความแม่นยำดีที่สุด β จึงมีความหมายว่า "จำนวนรายการร่วมที่ถือว่าเชื่อถือได้เต็มที่"
- **ทำไมบทนี้ใช้ β = 3** ผู้ใช้แต่ละคนมีคะแนนเพียง 3–5 เรื่อง และคู่ที่มีรายการร่วมมากที่สุดมีเพียง 3 เรื่อง (ตารางที่ 5.7) ถ้าใช้ 50 ความคล้ายทุกคู่จะถูกลดเหลือไม่เกิน 3/50 = 0.06 จนแยกเพื่อนบ้านไม่ออก
- **λ ของ SHRINKAGE** ทำงานแบบเดียวกันแต่นุ่มนวลกว่า ค่า λ = 2 หมายถึง "เมื่อมีรายการร่วม 2 เรื่อง เชื่อความคล้ายครึ่งหนึ่ง" ในระบบจริงควรเลือกด้วยชุดตรวจสอบ
- **หลักการเลือก** ตั้งค่าให้คู่ที่มีรายการร่วม "ปกติ" ของข้อมูลได้ตัวคูณใกล้ 1 และคู่ที่มีรายการร่วมน้อยผิดปกติถูกลดลงชัดเจน

5.3.4 เปรียบเทียบมาตรวัดทั้งหมดของ u1

ตารางที่ 5.9 ความคล้ายของ u1 กับผู้ใช้อื่นทุกคนด้วยสี่มาตรวัด

ผู้ใช้	รายการร่วม	Jaccard	Cosine ดิบ	Pearson	Pearson ถ่วงแล้ว
u2	3	0.5000	0.5238	−0.9086	−0.9086
u3	0	0.0000	0.0000	0.0000	0.0000
u4	1	0.1250	1.0000	1.0000	0.3333
u5	2	0.2857	0.9216	0.9062	0.6041
u6	1	0.1429	1.0000	1.0000	0.3333
u7	1	0.1250	1.0000	−1.0000	−0.3333
u8	3	0.5000	0.5451	−0.9063	−0.9063

ที่มา: สมการ (5.4), (5.5) และ (5.6) ที่ β = 3

ตารางที่ 5.9 ยืนยันบทเรียนของหัวข้อนี้สามข้อ

- **Cosine ดิบบอกว่าทุกคนคล้าย u1** (0.52–1.00) แม้กระทั่ง u2 และ u8 ที่รสนิยมตรงข้าม จึงไม่เหมาะกับคะแนนชัดแจ้ง
- **Jaccard ให้ u2 และ u8 สูงที่สุด** (0.50) เพราะดูเรื่องเดียวกันมาก แต่ไม่รู้ว่าเห็นตรงข้ามกัน
- **Pearson ถ่วงแล้ว** เป็นมาตรวัดเดียวที่จัดลำดับได้สมเหตุสมผล คือ u5 เหมือนที่สุด u4 และ u6 เหมือนเล็กน้อย u2 และ u8 ตรงข้ามชัดเจน หัวข้อ 5.4 จึงใช้มาตรวัดนี้

โปรแกรม 5.2 Jaccard, Cosine, Pearson และการถ่วงนัยสำคัญของ u1

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
```

```
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
mean = {u: np.mean(list(d.values())) for u, d in R.items()}



def measures(u, v, beta=3):
    """คืนค่า (รายการร่วม, Jaccard, Cosine, Pearson, Pearson ถ่วงแล้ว)"""
    co = [i for i in R[u] if i in R[v]]
    jac = len(co) / len(set(R[u]) | set(R[v]))
    if not co:
        return 0, jac, 0.0, 0.0, 0.0
    a = np.array([R[u][i] for i in co], float)
    b = np.array([R[v][i] for i in co], float)
    cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
    ac = a - mean[u]
    bc = b - mean[v]
    d = np.linalg.norm(ac) * np.linalg.norm(bc)
    pear = ac @ bc / d if d else 0.0
    weighted = pear * min(len(co), beta) / beta
    return len(co), jac, cos, pear, weighted



for v in ['u2', 'u3', 'u4', 'u5', 'u6', 'u7', 'u8']:
    print(v, [round(float(x), 4) for x in measures('u1', v)])
```

5.4 การกรองร่วมอิงผู้ใช้ (User-kNN)

User-kNN ทำนายคะแนนของผู้ใช้ u ต่อรายการ i จากคะแนนที่ **เพื่อนบ้าน k คนที่คล้าย u ที่สุดและเคยให้คะแนน i** ให้ไว้ หัวข้อนี้สร้างสูตรขึ้นทีละขั้นจากแบบง่ายที่สุด เพื่อให้เห็นว่าแต่ละส่วนของสูตรแก้ปัญหาอะไร ดังสรุปในภาพที่ 5.30 ภาพนี้สรุปการเลือกผู้ใช้ที่คล้ายกัน การรวมคะแนนแบบถ่วงน้ำหนัก และการจัดอันดับรายการ



ภาพที่ 5.30 การแนะนำด้วยเพื่อนบ้านของผู้ใช้

5.4.1 จากค่าเฉลี่ยธรรมดาสู่สูตรที่ลบค่าเฉลี่ย

สมมติว่าเลือกเพื่อนบ้านได้แล้ว คือเซต $N_k(u;i)$ ของผู้ใช้ k คนที่ความคล้ายหลังถ่วง (สมการ (5.6)) เป็นบวกสูงสุดและเคยให้คะแนน i สูตรทำนายพัฒนาได้สามขั้น

1. **แบบที่ 1 ค่าเฉลี่ยธรรมดา** เฉลี่ยคะแนนของเพื่อนบ้านทุกคนเท่ากัน ง่ายแต่ไม่สนใจว่าใครคล้ายเรามากกว่า
2. **แบบที่ 2 ค่าเฉลี่ยถ่วงน้ำหนัก** ให้เพื่อนบ้านที่คล้ายมากมีน้ำหนักมาก (สมการ (5.7))
3. **แบบที่ 3 ถ่วงน้ำหนักบนคะแนนที่ลบค่าเฉลี่ย** ใช้ "ความเบี่ยงเบนจากปกติ" ของเพื่อนบ้าน แล้วบวกกลับด้วยค่าเฉลี่ยของผู้ใช้เอง (สมการ (5.8)) ซึ่งเป็นสูตรมาตรฐานของ Resnick et al. (1994)

$$\hat{r}_{ui}^{\,\mathrm{w}} = \frac{\sum_{v \in N_k(u;i)} \mathrm{sim}'(u,v)\, r_{vi}}{\sum_{v \in N_k(u;i)} |\mathrm{sim}'(u,v)|} \tag{5.7}$$

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_k(u;i)} \mathrm{sim}'(u,v)\, (r_{vi} - \bar{r}_v)}{\sum_{v \in N_k(u;i)} |\mathrm{sim}'(u,v)|} \tag{5.8}$$

โดยที่

- $\hat{r}_{ui}^{\,\mathrm{w}}$ คือคำทำนายแบบค่าเฉลี่ยถ่วงน้ำหนักบนคะแนนดิบ และ $\hat{r}_{ui}$ คือคำทำนายแบบลบค่าเฉลี่ย
- $N_k(u;i)$ คือเซตของเพื่อนบ้าน k คนที่คล้าย u มากที่สุดและเคยให้คะแนน i
- $\mathrm{sim}'(u,v)$ คือความคล้ายหลังถ่วงจากสมการ (5.6) ใช้เป็นน้ำหนักของเพื่อนบ้าน v
- $r_{vi} - \bar{r}_v$ คือความเบี่ยงเบนจากปกติของเพื่อนบ้าน v ต่อรายการ i และ $\bar{r}_u$ คือค่าเฉลี่ยของผู้ใช้เป้าหมาย

ตัวส่วนของทั้งสองสมการใช้ค่าสัมบูรณ์ของความคล้าย เพื่อให้น้ำหนักรวมเป็น 1 พอดี ถ้าไม่มีเพื่อนบ้านเลย ระบบใช้ค่าเฉลี่ยของผู้ใช้ $\bar{r}_u$ เป็นคำทำนาย

ตัวอย่างการคำนวณ 5.8 สามแบบของสูตรทำนายสำหรับ u1 กับ i5 Wicked

โจทย์ ใช้เพื่อนบ้านสองคนที่เคยดู i5 และมีความคล้ายบวกกับ u1 ทำนาย $\hat{r}_{u1,i5}$ ด้วยสามแบบ

สมการที่ใช้ สมการ (5.7) และ (5.8)

ข้อมูลที่ใช้ ความคล้ายหลังถ่วงจากตารางที่ 5.9 ค่าเฉลี่ยของผู้ใช้จากตารางที่ 5.6

ขั้นที่ 1 หาเพื่อนบ้าน

- ผู้ที่ให้คะแนน i5 คือ u2 (4), u4 (1) และ u6 (4)
- u2 มีความคล้าย -0.9086 เป็นลบ จึงไม่นับ
- เหลือ u4 ($r = 1$, $\bar{r} = 3.75$, sim′ = 0.3333) และ u6 ($r = 4$, $\bar{r} = 4.00$, sim′ = 0.3333)

ขั้นที่ 2 แบบที่ 1 และ 2

- ค่าเฉลี่ยธรรมดา: $(1 + 4)/2 = 2.5$
- ถ่วงน้ำหนัก: $\frac{0.3333(1)+0.3333(4)}{0.3333+0.3333} = 2.5000$ (เท่ากับแบบที่ 1 เพราะน้ำหนักเท่ากัน)

ขั้นที่ 3 แบบที่ 3 (ลบค่าเฉลี่ย)

- ความเบี่ยงเบนของ u4: $1 - 3.75 = -2.75$ (ไม่ชอบมาก)
- ความเบี่ยงเบนของ u6: $4 - 4.00 = 0$ (เฉย ๆ เท่ากับปกติ)
- ค่าเฉลี่ยถ่วงของความเบี่ยงเบน: $\frac{0.3333(-2.75)+0.3333(0)}{0.6667} = -1.375$
- $\hat{r}_{u1,i5} = 3.2 + (-1.375) = 1.8250$

ขั้นที่ 4 ตีความ

แบบที่ 1–2 ตีความคะแนน 4 ของ u6 ว่า "ชอบ" แต่ u6 ให้คะแนน 4 เป็นปกติ (ค่าเฉลี่ย 4.00) แบบที่ 3 จึงอ่านว่า u6 "เฉย ๆ" และ u4 "ไม่ชอบมาก" ผลคือ u1 น่าจะให้ Wicked ต่ำกว่าปกติของตัวเองมาก การลบค่าเฉลี่ยจึงเปลี่ยนคำทำนายจาก 2.5 เป็น 1.8250

5.4.2 ขั้นตอนวิธีของ User-kNN

ขั้นตอนวิธี User-kNN สำหรับผู้ใช้ u และรายการ i

1. คำนวณค่าเฉลี่ย $\bar{r}_v$ ของผู้ใช้ทุกคน (ทำครั้งเดียวล่วงหน้าได้)
2. หาผู้ใช้ทุกคน v ที่เคยให้คะแนน i
3. คำนวณ $\text{sim}'(u, v)$ ด้วยสมการ (5.5) และ (5.6) แล้วเก็บเฉพาะค่าที่เป็นบวก
4. เรียงจากมากไปน้อย แล้วเลือก k คนแรกเป็น $N_k(u; i)$ (ถ้าเท่ากันเรียงตามรหัสผู้ใช้)
5. ถ้า $N_k(u; i)$ ว่าง ให้ $\hat{r}_{ui} = \bar{r}_u$ ถ้าไม่ว่าง ใช้สมการ (5.8)
6. ทำซ้ำกับทุกรายการที่ u ยังไม่ดู แล้วเรียงคำทำนายเพื่อเลือก Top-K

ขั้นตอนที่ 3 คือส่วนที่ใช้เวลามากที่สุด เพราะต้องเทียบ u กับผู้ใช้ทุกคนที่ดู i ถ้าคำนวณความคล้ายของผู้ใช้ทุกคู่ไว้ล่วงหน้า ต้องใช้หน่วยความจำเป็นสัดส่วนกับ $|\mathcal{U}|^2$ ซึ่งรับได้เมื่อมีผู้ใช้ 8 คน แต่เป็นไปไม่ได้เมื่อมีผู้ใช้หลายร้อยล้านคน นี่คือเหตุผลหลักที่ระบบขนาดใหญ่หันไปใช้ Item-kNN (หัวข้อ 5.5) และแบบจำลองปัจจัยแฝง (หัวข้อ 5.7) หัวข้อ 5.14.4 จะคำนวณต้นทุนนี้ด้วยตัวเลข

5.4.3 ทำนายคะแนนของ u1 ทั้งเจ็ดเรื่อง

ตัวอย่างการคำนวณ 5.9 ทำนายคะแนนของ u1 ต่อ i9 Conclave

โจทย์ ใช้ $k = 2$ จงทำนาย $\hat{r}_{u1,i9}$

สมการที่ใช้ สมการ (5.8)

ข้อมูลที่ใช้ ความคล้ายหลังถ่วงจากตัวอย่างการคำนวณ 5.7 ค่าเฉลี่ยจากตารางที่ 5.6

ขั้นที่ 1 ใครเคยให้คะแนน i9

- มีเพียง u5 ($r = 5, \bar{r}_{u5} = 3.5$) ความคล้ายกับ u1 = 0.6041 เป็นบวก จึงเป็นเพื่อนบ้านคนเดียว

ขั้นที่ 2 แทนค่าในสมการ (5.8)

$$\hat{r}_{u1,i9} = 3.2 + \frac{0.6041(5 - 3.5)}{0.6041} = 3.2 + 1.5 = 4.7000$$

ขั้นที่ 3 ตีความ

u5 ชอบ Conclave สูงกว่าค่าเฉลี่ยตัวเอง 1.5 คะแนน ระบบจึงทำนายว่า u1 จะให้สูงกว่าค่าเฉลี่ยของ u1 เท่ากัน คือ 4.7000 สังเกตว่าเมื่อมีเพื่อนบ้านคนเดียว ค่าความคล้ายตัดกันระหว่างตัวเศษกับตัวส่วน ความคล้ายจึงมีผลแค่ตอน **เลือก** เพื่อนบ้าน ไม่มีผลต่อขนาดของคำทำนาย

ตารางที่ 5.10 คำทำนาย User-kNN ของ u1 ต่อเจ็ดเรื่อง (k = 2, β = 3)

รหัส	ชื่อเรื่อง	คำทำนาย	เพื่อนบ้าน (ความคล้าย)	อันดับ
i5	Wicked	1.8250	u4 (0.3333), u6 (0.3333)	6
i7	Ballerina	4.4500	u4 (0.3333)	2
i8	Furiosa: A Mad Max Saga	4.4500	u4 (0.3333)	3
i9	Conclave	4.7000	u5 (0.6041)	1
i10	Dune: Part Two	1.7000	u5 (0.6041)	7
i11	Mickey 17	2.2000	u6 (0.3333)	5
i12	Project Hail Mary	3.2000	— (ใช้ $\bar{r}_u$)	4

ที่มา: สมการ (5.8) และตารางที่ 5.5

ตารางที่ 5.10 ให้ Top-3 ของ u1 เป็น i9, i7 และ i8 (i7 กับ i8 เสมอกันที่ 4.4500 จึงเรียงตามเลขรหัส) ผู้อ่านควรสังเกตสามประเด็น

- คำแนะนำ i7 และ i8 มาจาก u4 ซึ่งชอบ Toy Story 5 เหมือน u1 และชอบภาพยนตร์แอ็กชัน แต่หลักฐานความคล้ายมาจากรายการร่วมเพียงเรื่องเดียว
- i12 ไม่มีเพื่อนบ้านบวกเลย (u3 ไม่มีรายการร่วมกับ u1 และ u7 มีความคล้ายลบ) จึงได้ค่าเฉลี่ยของ u1 คือ 3.2
- Wicked ได้อันดับเกือบสุดท้าย ตรงข้ามกับบทที่ 3–4 ซึ่งให้ Wicked เป็นอันดับหนึ่ง เพราะการกรองร่วมไม่รู้เลยว่า Wicked มีเนื้อหาคล้ายแอนิเมชันที่ u1 ชอบ

อธิบายแบบเข้าใจง่าย เพื่อนบ้านคือคณะกรรมการชั่วคราว

ทุกครั้งที่ระบบต้องทำนายคะแนนหนึ่งช่อง ระบบจะตั้ง "คณะกรรมการ" จากผู้ใช้ที่เคยดูเรื่องนั้นและมีรสนิยมคล้ายเรา กรรมการแต่ละคนบอกว่าตัวเองชอบเรื่องนั้นมากหรือน้อยกว่าปกติเท่าใด

ความเห็นของกรรมการที่คล้ายเรามากมีน้ำหนักมาก แล้วระบบจึงแปลงความเห็นนั้นกลับเป็นมาตราคะแนนของเรา คณะกรรมการชุดนี้ตั้งใหม่ทุกครั้งที่เปลี่ยนเรื่อง เพราะคนที่เคยดูแต่ละเรื่องไม่เหมือนกัน

5.4.4 ตัวอย่างครบวงจร: ซ่อนคำตอบ ทำนาย จัดอันดับ และวัดผล

คำทำนายในตารางที่ 5.10 ยังบอกไม่ได้ว่าระบบดีหรือไม่ เพราะเราไม่รู้ว่า u1 ชอบ Conclave จริงหรือไม่ การประเมินแบบออฟไลน์จึงต้อง **ซ่อนคำตอบที่รู้อยู่แล้ว** แล้วตรวจว่าระบบนำคำตอบนั้นกลับมาได้ในอันดับใด ตัวอย่างต่อไปนี้เดินครบทุกขั้นของโปรโตคอลในหัวข้อ 5.13 สำหรับกรณีเดียว เพื่อให้เห็นว่าตัวเลขในตารางสรุปผลเกิดขึ้นอย่างไร

ตัวอย่างการคำนวณ 5.10 ซ่อน i3 The Wild Robot ของ u1 แล้วประเมิน User-kNN ครบวงจร

โจทย์ ซ่อนคะแนน $r_{u1,i3} = 4$ แล้วดูว่า User-kNN จัด i3 ไว้อันดับใด

สมการที่ใช้ สมการ (5.5), (5.6) และ (5.8)

ข้อมูลที่ใช้ ตารางที่ 5.5 ที่ตัดคะแนน r(u1, i3) ออก

ขั้นที่ 1 แบ่งข้อมูลและคำนวณค่าเฉลี่ยใหม่

- ข้อมูลฝึกของ u1 เหลือ {i1 = 5, i2 = 5, i4 = 1, i6 = 1}
- ค่าเฉลี่ยต้องคำนวณใหม่จากข้อมูลฝึก: $\bar{r}_{u1} = 12/4 = 3.0000$ ไม่ใช่ 3.2

ขั้นที่ 2 คำนวณความคล้ายใหม่ทั้งหมด

- u5 เหลือรายการร่วมเรื่องเดียว (i6) เพราะ i3 ถูกซ่อน ความคล้ายหลังถ่วงจึงลดจาก 0.6041 เหลือ 0.3333
- u4 และ u6 ได้ 0.3333 เท่าเดิม ส่วน u2 = −0.9428 และ u8 = −0.9272 ยังเป็นลบ

ขั้นที่ 3 ทำนายและจัดอันดับรายการที่เป็นตัวเลขแปดเรื่อง

อันดับ	รายการที่เป็นตัวเลข	เพื่อนบ้าน	คำทำนาย
1	i9	u5	4.5000
2	i7	u4	4.2500
3	i8	u4	4.2500
4	**i3 (ที่ซ่อน)**	u5	3.5000
5	i12	—	3.0000
6	i11	u6	2.0000
7	i5	u4, u6	1.6250
8	i10	u5	1.5000

ขั้นที่ 4 คำนวณตัวชี้วัดของกรณีนี้

- เรื่องที่ซ่อนอยู่อันดับ 4 จึงได้ HR@3 = 0, RR = $1/4 = 0.2500$ และ nDCG@3 = 0

ขั้นที่ 5 ตีความ

User-kNN พลาดกรณีนี้ด้วยเหตุผลที่ชัดเจน เพื่อนบ้านคนเดียวที่ดู The Wild Robot คือ u5 ซึ่งให้เพียง 4 คะแนน สูงกว่าค่าเฉลี่ยของตัวเองเพียง 0.5 ขณะที่ u5 ให้ Conclave 5 คะแนน และ u4 ให้ภาพยนตร์แอ็กชันสองเรื่อง 5 คะแนน ระบบจึงจัดสามเรื่องนั้นไว้ก่อน ส่วนระบบอิงเนื้อหาของบทที่ 4 จัด The Wild Robot ไว้อันดับต้น เพราะภาพและวิดีโอของแอนิเมชันสามเรื่องคล้ายกันมาก

ข้อควรระวัง ข้อผิดพลาดที่พบบ่อยในการประเมินแบบซ่อนคำตอบ

- **คำนวณค่าเฉลี่ยจากข้อมูลทั้งหมดก่อนซ่อน** ถ้าใช้ $\bar{r}_{u1} = 3.2$ แทน 3.0 ข้อมูลของรายการที่ซ่อนจะรั่วเข้าสู่การทำนาย เรียกว่า **การรั่วไหลของข้อมูล (Data Leakage)** (บทที่ 2 หัวข้อ 2.8)
- **ใช้ความคล้ายที่คำนวณไว้ล่วงหน้าจากข้อมูลทั้งหมด** ความคล้ายของ u1–u5 จะเป็น 0.6041 แทน 0.3333 เพราะยังนับ i3 เป็นรายการร่วม เท่ากับให้ระบบเห็นคำตอบก่อนสอบ
- **ลืมตัดรายการที่ผู้ใช้เคยให้คะแนนแล้ว** ระบบจะแนะนำ i1 หรือ i2 ที่ u1 ดูไปแล้ว และตัวชี้วัดจะคำนวณบนรายการที่ผู้ใช้ไม่ต้องการ

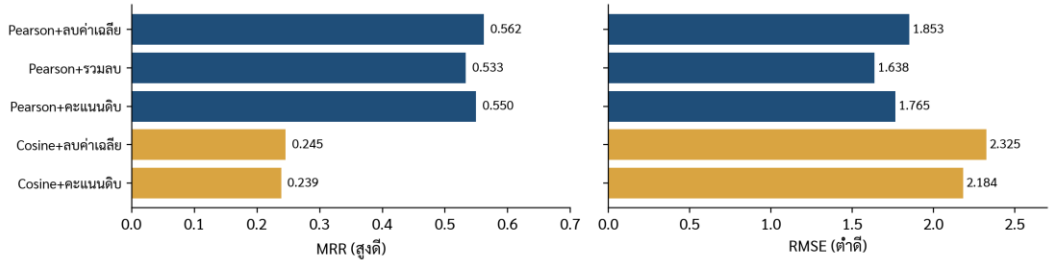
5.4.5 ทางเลือกในการออกแบบ User-kNN

User-kNN มีทางเลือกในการออกแบบหลายจุด ได้แก่ มาตรวัดความคล้าย การลบค่าเฉลี่ย การยอมรับเพื่อนบ้านที่ความคล้ายเป็นลบ และจำนวนเพื่อนบ้าน k หัวข้อนี้ทดลองทีละทางเลือกด้วยโปรโตคอลของหัวข้อ 5.13 ทั้งแบบจัดอันดับ (MRR) และแบบทำนายคะแนน (RMSE)

ตารางที่ 5.11 ผลของทางเลือกในการออกแบบ User-kNN

รูปแบบ	HR@3	MRR	nDCG@3	RMSE
Pearson + ลบค่าเฉลี่ย + เพื่อนบ้านบวก (ค่าหลัก)	0.7500	0.5621	0.5762	1.8534
Pearson + ลบค่าเฉลี่ย + รวมเพื่อนบ้านลบ	0.6000	0.5333	0.4881	1.6379
Pearson + ค่าเฉลี่ยถ่วงน้ำหนักของคะแนนดิบ	0.7000	0.5496	0.5446	1.7652
Cosine ดิบ + ลบค่าเฉลี่ย	0.2000	0.2455	0.1381	2.3255
Cosine ดิบ + ค่าเฉลี่ยถ่วงน้ำหนักของคะแนนดิบ	0.2000	0.2385	0.1315	2.1836

ที่มา: LOO 20 กรณีและกันออกทีละคะแนน 32 ครั้ง (หัวข้อ 5.13) ทุกแบบใช้ $\beta = 3$



ภาพที่ 5.8 ผลของ User-kNN ห้ารูปแบบ

ตารางที่ 5.11 และภาพที่ 5.8 ให้ข้อสังเกตสามข้อ

- **มาตรวัดความคล้ายสำคัญที่สุด** การเปลี่ยนจาก Pearson เป็น Cosine ดิบทำให้ MRR ลดลงเกินครึ่ง (0.5621 เป็น 0.2455) เพราะ Cosine ดิบให้ความคล้ายบวกแก่เกือบทุกคู่ ระบบจึงยืมความชอบจากผู้ใช้ที่รสนิยมตรงข้าม (หัวข้อ 5.3.1)
- **เพื่อนบ้านลบช่วยการทำนายคะแนนแต่ไม่ช่วยการจัดอันดับ** RMSE ลดลงจาก 1.8534 เป็น 1.6379 แต่ MRR ลดลงจาก 0.5621 เป็น 0.5333
- **ตัวชี้วัดสองแบบอาจเลือก "รูปแบบที่ดีที่สุด" ต่างกัน** ผู้พัฒนาจึงต้องเลือกตัวชี้วัดหลักให้ตรงกับเป้าหมายของระบบก่อนเปรียบเทียบ (บทที่ 2 หัวข้อ 2.7)

ตัวอย่างต่อไปนี้แสดงกลไกของเพื่อนบ้านลบให้เห็นทีละขั้น

สูตรในสมการ (5.8) ใช้ความคล้ายที่เป็นลบได้ เพราะตัวส่วนเป็นค่าสัมบูรณ์ แนวคิดคือ "ถ้าคนรสนิยมตรงข้ามไม่ชอบเรื่องนี้ เราน่าจะชอบ" ตัวอย่างต่อไปนี้แสดงผลของการเปิดใช้เพื่อนบ้านลบ

ตัวอย่างการคำนวณ 5.11 u1 กับ i11 Mickey 17 เมื่อใช้เพื่อนบ้านลบ

โจทย์ เทียบคำทำนาย $\hat{r}_{u1,i11}$ เมื่อใช้เฉพาะเพื่อนบ้านบวก กับเมื่อใช้ทุกคนที่ความคล้ายไม่เป็นศูนย์

สมการที่ใช้ สมการ (5.8)

ข้อมูลที่ใช้ ตารางที่ 5.9 และ 5.6

ขั้นที่ 1 ผู้ที่ให้คะแนน i11

- u3 ($\text{sim}' = 0$ ไม่นับ), u6 ($r = 3, \bar{r} = 4.0, \text{sim}' = +0.3333$)
- u7 ($r = 4, \bar{r} = 3.5, \text{sim}' = -0.3333$) และ u8 ($r = 3, \bar{r} = 3.5, \text{sim}' = -0.9063$)

ขั้นที่ 2 เฉพาะเพื่อนบ้านบวก

- มีเพียง u6: $\hat{r} = 3.2 + 0.3333(3 - 4)/0.3333 = 3.2 - 1 = 2.2000$

ขั้นที่ 3 รวมเพื่อนบ้านลบ

- ตัวเศษ: $(-0.9063)(3 - 3.5) + (0.3333)(3 - 4) + (-0.3333)(4 - 3.5) = 0.4532 - 0.3333 - 0.1667 = -0.0469$
- ตัวส่วน: $0.9063 + 0.3333 + 0.3333 = 1.5729$
- $\hat{r} = 3.2 + (-0.0469/1.5729) = 3.1702$

ขั้นที่ 4 ตีความ

u8 ซึ่งรสนิยมตรงข้าม u1 ให้ Mickey 17 ต่ำกว่าปกติเล็กน้อย สูตรจึงตีความว่า u1 น่าจะชอบและดึงคำทำนายขึ้นจาก 2.2 เป็น 3.1702 ข้อสรุปนี้เสี่ยง เพราะ "ตรงข้ามใน ROMANCE " ไม่ได้แปลว่า "ตรงข้ามใน SCI-FI" ด้วย

ตารางที่ 5.12 ผลของ k และเพื่อนบ้านลบต่อการจัดอันดับของ User-kNN

การตั้งค่า	HR@3	MRR
k = 1 เฉพาะเพื่อนบ้านบวก	0.7500	0.5621
k = 2 เฉพาะเพื่อนบ้านบวก	0.7500	0.5621
k = 3 เฉพาะเพื่อนบ้านบวก	0.7500	0.5621
k = 8 เฉพาะเพื่อนบ้านบวก	0.7500	0.5621
k = 8 รวมเพื่อนบ้านลบ	0.6000	0.5333

ที่มา: โปรโตคอลกันออกทีละรายการ 20 กรณี (หัวข้อ 5.13.2)

ตารางที่ 5.12 แสดงว่าบน MovieMini-12 ค่า k ไม่มีผลเลย เพราะแต่ละรายการมีผู้ให้คะแนนเพียง 1–4 คน และมีเพื่อนบ้านบวกไม่เกินสองคน ส่วนการเปิดใช้เพื่อนบ้านลบทำให้ MRR ลดลงจาก 0.5621 เป็น 0.5333 ตรงกับข้อสังเกตในตัวอย่างการคำนวณ 5.11

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนเพื่อนบ้าน k และการใช้เพื่อนบ้านลบ

- **k มาจากไหน** Herlocker et al. (1999) ทดลองบน MovieLens แล้วพบว่า k ระหว่าง 20–50 ให้ความแม่นยำดี k น้อยเกินทำให้คำทำนายแกว่งตามความเห็นของคนไม่กี่คน k มากเกินดึงความเห็นของคนที่ไม่ค่อยคล้ายเข้ามาเจือจาง
- **ทำไมบทนี้ใช้ k = 2** ชุดข้อมูลเล็กจนแต่ละรายการมีเพื่อนบ้านบวกไม่เกินสองคน ค่าที่มากกว่านี้จึงให้ผลเหมือนกัน (ตารางที่ 5.12) ค่า 2 ทำให้ตัวอย่างอ่านง่าย
- **เพื่อนบ้านลบ** งานส่วนใหญ่ใช้เฉพาะเพื่อนบ้านบวก เพราะความคล้ายลบมักมาจากหลักฐานน้อยและไม่ถ่ายทอดข้ามแนวภาพยนตร์ ผลบน MovieMini-12 ยืนยันแนวปฏิบัตินี้
- **วิธีเลือกในงานจริง** ลอง k เป็นลำดับ เช่น 10, 20, 50, 100 บนชุดตรวจสอบ แล้วเลือกค่าที่ตัวชี้วัดการจัดอันดับดีที่สุด

โปรแกรม 5.3 User-kNN สามแบบ และผลของเพื่อนบ้านลบ

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
mean = {u: np.mean(list(d.values())) for u, d in R.items()}


def sim(u, v, beta=3):
```

```
    """Pearson ที่ถ่วงนัยสำคัญด้วย beta"""
    co = [i for i in R[u] if i in R[v]]
    if not co:
        return 0.0
    a = np.array([R[u][i] - mean[u] for i in co])
    b = np.array([R[v][i] - mean[v] for i in co])
    d = np.linalg.norm(a) * np.linalg.norm(b)
    s = a @ b / d if d else 0.0
    return s * min(len(co), beta) / beta



def predict(u, i, k=2, center=True, neg=False):
    # 1) ผู้ใช้ที่เคยให้คะแนน i และความคล้ายกับ u
    cand = [(sim(u, v), v) for v in R if v != u and i in R[v]]
    # 2) เก็บเฉพาะความคล้ายบวก (หรือไม่เป็นศูนย์ ถ้า neg=True)
    if neg:
        cand = [t for t in cand if t[0] != 0]
    else:
        cand = [t for t in cand if t[0] > 0]
    nb = sorted(cand, key=lambda t: -abs(t[0]))[:k]
    if not nb:
        return mean[u]
    den = sum(abs(s) for s, v in nb)
    # 3) แบบถ่วงน้ำหนักบนคะแนนดิบ
    if not center:
        return sum(s * R[v][i] for s, v in nb) / den
    # 4) แบบลบค่าเฉลี่ย แล้วบวกค่าเฉลี่ยของ u กลับ
    dev = sum(s * (R[v][i] - mean[v]) for s, v in nb)
    return mean[u] + dev / den



print('i5 แบบถ่วง', round(float(predict('u1', 'i5', center=False)), 4))
print('i5 แบบลบค่าเฉลี่ย', round(float(predict('u1', 'i5')), 4))
print('i11 บวกอย่างเดียว', round(float(predict('u1', 'i11', k=8)), 4))
print('i11 รวมลบ', round(float(predict('u1', 'i11', k=8, neg=True)), 4))
CANDS = ['i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12']
print({i: round(float(predict('u1', i)), 4) for i in CANDS})
```

5.5 การกรองร่วมอิงรายการ (Item-kNN)

Item-kNN กลับมุมมองจาก "ผู้ใช้ที่คล้ายกัน" เป็น "รายการที่คล้ายกัน" ระบบหาว่ารายการ i คล้ายกับรายการใดที่ผู้ใช้เคยให้คะแนน แล้วทำนายจากคะแนนที่ผู้ใช้ให้รายการเหล่านั้น (Sarwar et al., 2001) แนวคิดนี้คือหลักของ "ลูกค้าที่ซื้อสินค้านี้ยังซื้อ..." ที่ Amazon ใช้ (Linden et al., 2003) ดังสรุปในภาพที่ 5.31 ภาพนี้สรุปการเชื่อมรายการที่เคยชอบกับรายการคล้ายกันก่อนจัดอันดับคำแนะนำ



ภาพที่ 5.31 การแนะนำด้วยเพื่อนบ้านของรายการ

5.5.1 ความคล้ายระหว่างรายการ: Adjusted Cosine

ความคล้ายของรายการคิดจาก **คอลัมน์** ของเมทริกซ์คะแนน โดยดูเฉพาะผู้ใช้ $\mathcal{U}_{ij}$ ที่ให้คะแนนทั้ง i และ j มาตรวัดที่นิยมคือ Adjusted Cosine ซึ่งลบค่าเฉลี่ยของ **ผู้ใช้** (ไม่ใช่ของรายการ) เพราะความต่างของมาตราคะแนนมาจากผู้ใช้

$$\text{sim}(i,j) = \frac{\sum_{u\in\mathcal{U}_{ij}}(r_{ui}-\bar{r}_u)(r_{uj}-\bar{r}_u)}{\sqrt{\sum_{u\in\mathcal{U}_{ij}}(r_{ui}-\bar{r}_u)^2}\sqrt{\sum_{u\in\mathcal{U}_{ij}}(r_{uj}-\bar{r}_u)^2}} \tag{5.9}$$

โดยที่

- $\mathcal{U}_{ij}$ คือผู้ใช้ที่ให้คะแนนทั้งรายการ i และรายการ j
- $r_{ui}-\bar{r}_u$ และ $r_{uj}-\bar{r}_u$ คือคะแนนของผู้ใช้ u ต่อสองรายการ หลังลบค่าเฉลี่ยของผู้ใช้คนเดียวกัน
- $\text{sim}(i,j)$ คือความคล้ายระหว่างรายการ มีค่าระหว่าง -1 ถึง $+1$

ตัวอย่างการคำนวณ 5.12 สามวิธีวัดความคล้ายของ i4 It Ends with Us กับ i6 Bridget Jones

โจทย์ คู่ i4–i6 มีผู้ใช้ร่วมสามคน (u1, u2, u8) จงหา Cosine ดิบ, Pearson ของรายการ และ Adjusted Cosine แล้วเทียบกัน

สมการที่ใช้ สมการ (5.9) และรูปแบบของสมการ (5.4)

ข้อมูลที่ใช้ ตารางที่ 5.5 และ 5.6

ขั้นที่ 1 คะแนนดิบของผู้ใช้ร่วม (i4, i6)

- u1: (1, 1), u2: (5, 5), u8: (4, 5)
- Cosine ดิบ: $\frac{1+25+20}{\sqrt{1+25+16}\sqrt{1+25+25}} = \frac{46}{6.4807 \times 7.1414} = 0.9939$

ขั้นที่ 2 ลบค่าเฉลี่ยของรายการ (Pearson ของรายการ)

- $\bar{r}_{i4} = 3.3333$ และ $\bar{r}_{i6} = 3.5$ (ตารางที่ 5.6)
- u1: (−2.3333, −2.5000), u2: (+1.6667, +1.5000), u8: (+0.6667, +1.5000)
- ผลลัพธ์ = 0.9670

ขั้นที่ 3 ลบค่าเฉลี่ยของผู้ใช้ (Adjusted Cosine)

- u1 ($\bar{r} = 3.2$): (−2.2, −2.2), u2 ($\bar{r} = 4.0$): (1.0, 1.0), u8 ($\bar{r} = 3.5$): (0.5, 1.5)
- ตัวเศษ: $(-2.2)(-2.2) + (1)(1) + (0.5)(1.5) = 4.84 + 1 + 0.75 = 6.59$
- ตัวส่วน: $\sqrt{4.84 + 1 + 0.25} \times \sqrt{4.84 + 1 + 2.25} = 2.4678 \times 2.8443 = 7.0191$
- Adjusted Cosine = $6.59/7.0191 = 0.9389$

ขั้นที่ 4 ตีความ

- ทั้งสามวิธีบอกว่า i4 กับ i6 คล้ายกันมาก เพราะทั้งสามคนให้สองเรื่องนี้ไปทางเดียวกัน
- Cosine ดิบสูงเกินจริง (0.9939) ด้วยเหตุผลเดียวกับหัวข้อ 5.3.1
- Adjusted Cosine ต่ำที่สุด (0.9389) เพราะเห็นว่า u8 ให้ i6 สูงกว่าปกติของตัวเองมากกว่า i4 อยู่ 1 คะแนน จึงลดความคล้ายลงเล็กน้อย

บทนี้ถ่วงนัยสำคัญของความคล้ายระหว่างรายการด้วย $\beta = 3$ เช่นเดียวกับหัวข้อ 5.3.3 โดยนับจำนวนผู้ใช้ร่วมแทนจำนวนรายการร่วม

5.5.2 ทำนายจากรายการที่ผู้ใช้เคยให้คะแนน

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{j \in N_k(i;u)} \text{sim}(i,j)\,(r_{uj} - \bar{r}_u)}{\sum_{j \in N_k(i;u)} |\text{sim}(i,j)|} \tag{5.10}$$

โดยที่

- $N_k(i;u)$ คือรายการ k เรื่องที่ผู้ใช้ u เคยให้คะแนนและคล้ายกับรายการ i มากที่สุด
- $\text{sim}(i,j)$ คือความคล้ายระหว่างรายการจากสมการ (5.9) หลังถ่วงนัยสำคัญ
- $r_{uj} - \bar{r}_u$ คือคะแนนของผู้ใช้ u ต่อรายการเพื่อนบ้าน j หลังลบค่าเฉลี่ยของ u

สมการ (5.10) มีโครงเดียวกับสมการ (5.8) ต่างกันที่ "เพื่อนบ้าน" คือ **รายการ k เรื่องที่ u เคยให้คะแนน** และคล้ายกับ i มากที่สุด ส่วนคะแนนที่นำมาเฉลี่ยคือคะแนนของ **ผู้ใช้คนเดียวกัน** หลังลบค่าเฉลี่ยของตัวเอง

ตารางที่ 5.13 Adjusted Cosine หลังถ่วงของเรื่องที่ u1 ยังไม่ดู

ยังไม่ดู	i1 (5)	i2 (5)	i3 (4)	i4 (1)	i6 (1)
i5	−0.08	0	0	0	0
i7	0.33	0	0	0	0
i8	0.33	0	0.33	0	0
i9	0	0	0.33	0	−0.33
i10	0	0	−0.33	0	0.33
i11	0	−0.08	−0.33	−0.33	−0.33

ยังไม่ดู	i1 (5)	i2 (5)	i3 (4)	i4 (1)	i6 (1)
i12	0	0	−0.33	0	0

ที่มา: สมการ (5.9) ถ่วงด้วย $\beta = 3$ คอลัมน์คือเรื่องที่ u1 ให้คะแนน (ตัวเลขในวงเล็บ)

ตารางที่ 5.13 คือ "แผนที่ความเชื่อมโยง" ที่ Item-kNN ใช้ ค่าบวกแปลว่าผู้ใช้ร่วมให้สองเรื่องไปทางเดียวกัน ค่าลบแปลว่าไปทางตรงข้าม ส่วนใหญ่เป็น ±0.33 เพราะมีผู้ใช้ร่วมเพียงคนเดียว ค่าในกรอบนี้คือสิ่งที่ตัวอย่างต่อไปนี้ใช้

ตัวอย่างการคำนวณ 5.13 ทำนายคะแนนของ u1 ต่อ i8 Furiosa ด้วย Item-kNN

โจทย์ จงทำนาย $\hat{r}_{u1,i8}$ ที่ $k = 2$

สมการที่ใช้ สมการ (5.9) และ (5.10)

ข้อมูลที่ใช้ แถว i8 ของตารางที่ 5.13 และ $\bar{r}_{u1} = 3.2$

ขั้นที่ 1 ความคล้ายของ i8 กับรายการของ u1

- i8–i1: ผู้ใช้ร่วมคือ u4 (i8: $5 - 3.75 = 1.25$, i1: $4 - 3.75 = 0.25$) → Adjusted Cosine = 1 → ถ่วงแล้ว $1 \times 1/3 = 0.3333$
- i8–i3: ผู้ใช้ร่วมคือ u7 (i8: $2 - 3.5 = -1.5$, i3: $3 - 3.5 = -0.5$) → 1 → 0.3333
- i8–i2, i8–i4, i8–i6: ไม่มีผู้ใช้ร่วม → 0

ขั้นที่ 2 ทำนาย

$$\hat{r}_{u1,i8} = 3.2 + \frac{0.3333(5 - 3.2) + 0.3333(4 - 3.2)}{0.3333 + 0.3333} = 3.2 + 1.3 = 4.5000$$

ขั้นที่ 3 ตีความ

ระบบสรุปว่า "คนที่ให้คะแนน Furiosa เคยให้คะแนน Toy Story 5 และ The Wild Robot ไปในทิศเดียวกัน" และ u1 ชอบสองเรื่องนั้น จึงทำนาย 4.5000 สังเกตว่าหลักฐานของ i8–i3 มาจาก u7 ซึ่ง **ไม่ชอบทั้งสองเรื่อง** ความคล้ายเชิงบวกของ Adjusted Cosine จึงหมายถึง "ให้คะแนนไปทางเดียวกัน" ไม่จำเป็นต้องหมายถึง "ชอบเหมือนกัน"

ตารางที่ 5.14 คำทำนาย Item-kNN ของ u1 ต่อเจ็ดเรื่อง (k = 2, $\beta = 3$)

รหัส	ชื่อเรื่อง	คำทำนาย	รายการเพื่อนบ้าน (ความคล้าย)	อันดับ
i5	Wicked	3.2000	— (ใช้ $\bar{r}_u$)	4
i7	Ballerina	5.0000	i1 (0.3333)	1
i8	Furiosa: A Mad Max Saga	4.5000	i1 (0.3333), i3 (0.3333)	2
i9	Conclave	4.0000	i3 (0.3333)	3
i10	Dune: Part Two	1.0000	i6 (0.3333)	7
i11	Mickey 17	3.2000	— (ใช้ $\bar{r}_u$)	5
i12	Project Hail Mary	3.2000	— (ใช้ $\bar{r}_u$)	6

ที่มา: สมการ (5.9) และ (5.10)

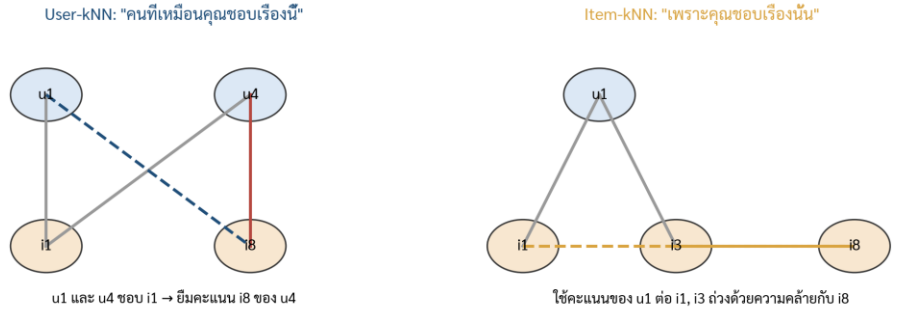
ตารางที่ 5.14 ให้ Top-3 เป็น i7, i8 และ i9 ใกล้เคียงกับ User-kNN รายละเอียดที่ควรสังเกตมีดังนี้

- i7 ได้ 5.0000 เพราะมีเพื่อนบ้านเรื่องเดียวคือ i1 ซึ่ง u1 ให้ 5 คำทำนายจึงเท่ากับคะแนนของ i1 พอดี
- i10 ได้ 1.0000 ต่ำที่สุด เพราะ i10 คล้ายกับ i6 (ผ่าน u5 ที่ให้ทั้งคู่ต่ำกว่าปกติ) และ u1 ให้ i6 เพียง 1
- i5, i11 และ i12 ไม่มีเพื่อนบ้านบวกเลย (แถวในตารางที่ 5.13 เป็นศูนย์หรือลบทั้งหมด) จึงได้ค่าเฉลี่ยของ u1 คือ 3.2

อธิบายแบบเข้าใจง่าย Item-kNN ตอบว่า "เพราะคุณชอบเรื่องนั้น"

User-kNN ถามว่า "คนที่เหมือนคุณชอบอะไร" ส่วน Item-kNN ถามว่า "เรื่องนี้เหมือนเรื่องไหนที่คุณเคยดูแล้วคุณให้เรื่องนั้นเท่าไร"

คำถามแบบหลังมีข้อดีคือ ใช้คะแนนของผู้ใช้คนเดียวกันทั้งหมด จึงไม่ต้องกังวลเรื่องมาตราคะแนนข้ามคน และอธิบายผลกับผู้ใช้ได้ง่าย เช่น "แนะนำ Furiosa เพราะคุณชอบ Toy Story 5"



ภาพที่ 5.9 หลักฐานของ User-kNN และ Item-kNN

ภาพที่ 5.9 เทียบหลักฐานของสองวิธีสำหรับคู่เดียวกัน (u1, i8) ด้านซ้าย User-kNN เห็นว่า u1 กับ u4 ชอบ i1 เหมือนกัน จึงยืมคะแนน i8 ของ u4 ด้านขวา Item-kNN เห็นว่า i8 คล้าย i1 และ i3 จึงใช้คะแนนที่ u1 ให้สองเรื่องนั้นเอง คำทำนายของสองวิธีจึงใกล้กัน (4.45 และ 4.50) แต่มาจากเหตุผลต่างกัน

5.5.3 เหตุใดระบบขนาดใหญ่จึงนิยม Item-kNN

Item-kNN ได้รับความนิยมในอุตสาหกรรมด้วยเหตุผลเชิงวิศวกรรมสามข้อ

- **รายการมักมีน้อยกว่าผู้ใช้มาก** ข้อมูล Netflix Prize มีผู้ใช้ 480,189 คน จึงมีคู่ผู้ใช้ราว 1.15×10^{11} คู่ แต่มีภาพยนตร์ 17,770 เรื่อง คู่ภาพยนตร์จึงมีเพียงราว 1.58×10^{8} คู่ น้อยกว่าเกือบหนึ่งพันเท่า
- **ความคล้ายของรายการเปลี่ยนช้า** รายการหนึ่งมีผู้ให้คะแนนจำนวนมาก คะแนนใหม่ไม่กี่ค่าจึงแทบไม่เปลี่ยนความคล้าย ระบบคำนวณตารางความคล้ายล่วงหน้าเป็นรอบ ๆ ได้
- **อธิบายผลได้ตรงไปตรงมา** เหตุผลอ้างถึงรายการที่ผู้ใช้รู้จักอยู่แล้ว เช่น "เพราะคุณชอบ Toy Story 5"

ตารางที่ 5.15 เปรียบเทียบ User-kNN กับ Item-kNN

ประเด็น	User-kNN	Item-kNN
สิ่งที่เทียบ	ผู้ใช้กับผู้ใช้ (แถว)	รายการกับรายการ (คอลัมน์)
ความเสถียร	รสนิยมผู้ใช้เปลี่ยนเร็ว ต้องคำนวณใหม่บ่อย	ความสัมพันธ์ของรายการเปลี่ยนช้า คำนวณล่วงหน้าได้
ผู้ใช้ใหม่ที่มีประวัติไม่กี่เรื่อง	ต้องเทียบกับผู้ใช้ทุกคนใหม่	ใช้ตารางความคล้ายเดิมได้ทันที

ประเด็น	User-kNN	Item-kNN
การอธิบายผล	"คนที่รสนิยมเหมือนคุณชอบเรื่องนี้"	"เพราะคุณชอบ Toy Story 5"
ความแปลกใหม่ของคำแนะนำ	สูงกว่า	ต่ำกว่า มักคล้ายของเดิม
ผล LOO บน MovieMini-12 (หัวข้อ 5.13)	HR@3 0.7500 MRR 0.5621	HR@3 0.6500 MRR 0.4655

ในระบบจริง Item-kNN แบ่งงานเป็นสองส่วน **ส่วนออฟไลน์** คำนวณความคล้ายของรายการทุกคู่ที่มีผู้ใช้ร่วมกัน แล้วเก็บเฉพาะเพื่อนบ้าน M อันดับแรกของแต่ละรายการ **ส่วนออนไลน์** อ่านรายการล่าสุดที่ผู้ใช้มีปฏิสัมพันธ์ ดึงเพื่อนบ้านจากตาราง รวมคะแนน ตัดรายการที่ผู้ใช้เคยเห็นแล้วส่งคืน Top-K เวลาตอบสนองจึงขึ้นกับจำนวนรายการในประวัติของผู้ใช้ ไม่ขึ้นกับจำนวนผู้ใช้ทั้งระบบ

ตัวอย่างการคำนวณ 5.14 หน่วยความจำของตารางเพื่อนบ้าน

โจทย์ คลังสินค้ามี 1 ล้านรายการ จงเทียบหน่วยความจำเมื่อเก็บความคล้ายทุกคู่ กับเมื่อเก็บเพียง M = 100 เพื่อนบ้านต่อรายการ

สมการที่ใช้ จำนวนช่อง = $|\mathcal{I}|^2$ หรือ $M \times |\mathcal{I}|$

ข้อมูลที่ใช้ สมมติว่าแต่ละช่องใช้ 8 ไบต์ (ค่าความคล้าย float32 และรหัสรายการ int32)

- เก็บทุกคู่: $(10^6)^2 = 10^{12}$ ช่อง หรือราว 8 เทราไบต์ ซึ่งเป็นไปไม่ได้
- เก็บ M = 100: $100 \times 10^6 = 10^8$ ช่อง หรือราว 8×10^8 ไบต์ = 800 เมกะไบต์ ซึ่งอยู่ในหน่วยความจำของเครื่องเดียวได้

ตีความ การตัดเหลือ M อันดับแรกเป็นการประมาณ เพราะความคล้ายที่ถูกตัดทิ้งอาจสำคัญกับผู้ใช้บางคน แต่ในทางปฏิบัติ M ระดับหลักร้อยให้ผลใกล้เคียงการเก็บทั้งหมด (Sarwar et al., 2001)

ที่มาของไฮเปอร์พารามิเตอร์ k ของ Item-kNN และการเก็บเพื่อนบ้านล่วงหน้า

- **k มาจากไหน** Sarwar et al. (2001) ทดลองบน MovieLens แล้วพบว่า k ประมาณ 20–30 เพื่อนบ้านให้ผลดี การเพิ่ม k เกินนั้นแทบไม่ช่วย
- **ขนาดตารางล่วงหน้า** งานเดียวกันพบว่าการเก็บเพียงเพื่อนบ้านสูงสุดไม่กี่สิบเรื่องต่อรายการช่วยลดหน่วยความจำได้มาก โดยความแม่นยำลดลงน้อย ระบบจริงจึงคำนวณตาราง "เพื่อนบ้านของแต่ละรายการ" ไว้ล่วงหน้าเป็นงานออฟไลน์ (บทที่ 7)
- **บทนี้ใช้ k = 2 และ β = 3** เพื่อให้สอดคล้องกับ User-kNN และอ่านตัวอย่างได้ง่าย

โปรแกรม 5.4 Adjusted Cosine และ Item-kNN

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
```

```
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
mean = {u: np.mean(list(d.values())) for u, d in R.items()}



def adj_cos(i, j, beta=3):
    """Adjusted Cosine ที่ถ่วงน้ำหนักด้วย beta"""
    us = [u for u in R if i in R[u] and j in R[u]]
    if not us:
        return 0.0
    a = np.array([R[u][i] - mean[u] for u in us])
    b = np.array([R[u][j] - mean[u] for u in us])
    d = np.linalg.norm(a) * np.linalg.norm(b)
    s = a @ b / d if d else 0.0
    return s * min(len(us), beta) / beta



print('i4-i6 (ไม่ถ่วง)', round(float(adj_cos('i4', 'i6', beta=1)), 4))



def predict(u, i, k=2):
    # เพื่อนบ้าน = เรื่องที่ u เคยให้คะแนนและคล้าย i ในทางบวก
    cand = [(adj_cos(i, j), j) for j in R[u]]
    nb = sorted([t for t in cand if t[0] > 0], key=lambda t: -t[0])[:k]
    if not nb:
        return mean[u]
    num = sum(s * (R[u][j] - mean[u]) for s, j in nb)
    den = sum(abs(s) for s, j in nb)
    return mean[u] + num / den



CANDS = ['i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12']
print({i: round(float(predict('u1', i)), 4) for i in CANDS})
```

5.5.4 Item-kNN สำหรับผลป้อนกลับโดยปริยาย

ระบบจริงส่วนใหญ่ไม่มีคะแนน 1–5 มีเพียงข้อมูลว่าผู้ใช้คลิก ซื้อ หรือดูรายการใด (หัวข้อ 5.2.5) ในกรณีนี้ Adjusted Cosine ใช้ไม่ได้ เพราะไม่มีค่าเฉลี่ยให้ลบ Item-kNN แบบโดยปริยายจึงใช้ **ความคล้ายโคไซน์บนเวกเตอร์ 0/1** ซึ่งเท่ากับการนับผู้ใช้ที่ชอบร่วมกันแล้วปรับขนาดด้วยความนิยม และให้คะแนนรายการที่เป็นตัวเลขด้วยผลรวมของความคล้ายกับรายการที่ผู้ใช้ชอบ นี่คือหลักการของแถว "ลูกค้าที่ซื้อสินค้านี้ยังซื้อ..." (Linden et al., 2003)

$$\mathrm{sim}(i,j) = \frac{\left|\mathcal{U}_i^{\mathrm{like}} \cap \mathcal{U}_j^{\mathrm{like}}\right|}{\sqrt{\left|\mathcal{U}_i^{\mathrm{like}}\right|\left|\mathcal{U}_j^{\mathrm{like}}\right|}}, \qquad \mathrm{score}(u,j) = \sum_{i \in L_u} \mathrm{sim}\,(i,j) \tag{5.11}$$

โดยที่

- $\mathcal{U}_i^{\mathrm{like}}$ คือเซตของผู้ใช้ที่ชอบรายการ i ($x_{ui} = 1$) และ $|\cdot|$ คือจำนวนสมาชิก
- ตัวเศษคือจำนวนผู้ใช้ที่ชอบทั้งสองรายการ ตัวส่วนปรับตามความนิยมของแต่ละรายการ
- L_u คือเซตของรายการที่ผู้ใช้ u ชอบ และ $\mathrm{score}(u,j)$ คือคะแนนจัดอันดับของรายการ j
- $\mathcal{U}_i^{\mathrm{like}}$ คือเซตของผู้ใช้ที่ชอบรายการ i (ในบทนี้คือให้คะแนน 4–5 ตามสมการ (5.3))

- L_u คือเซตของรายการที่ผู้ใช้ u ชอบ
- ตัวส่วนป้องกันไม่ให้รายการยอดนิยมมากคล้ายกับทุกรายการ และ score ไม่ใช่คะแนน 1–5 แต่เป็นคะแนนสำหรับจัดอันดับเท่านั้น

ตัวอย่างการคำนวณ 5.15 แถว "ผู้ที่ชอบเรื่องนี้ยังชอบ" ของ i1 และ Top-3 ของ u1

โจทย์ จงหารายการที่คล้าย Toy Story 5 มากที่สุด และคะแนนของรายการที่เป็นตัวเลขทั้งเจ็ดเรื่องของ u1

สมการที่ใช้ สมการ (5.11)

ข้อมูลที่ใช้ เมทริกซ์การชอบในตารางที่ 5.8

ขั้นที่ 1 เซตผู้ชอบของแต่ละรายการ

- i1: {u1, u4}, i2: {u1, u6}, i3: {u1, u5}, i5: {u2, u6}, i7: {u4}, i8: {u4}, i9: {u5}

ขั้นที่ 2 ความคล้ายของ i1 กับรายการอื่น

- $\text{sim}(i1, i7) = \frac{|\{u4\}|}{\sqrt{2 \times 1}} = 0.7071$ และ $\text{sim}(i1, i8) = 0.7071$
- $\text{sim}(i1, i2) = \frac{|\{u1\}|}{\sqrt{2 \times 2}} = 0.5000$ และ $\text{sim}(i1, i3) = 0.5000$
- แถว "ผู้ที่ชอบ Toy Story 5 ยังชอบ" จึงเรียงเป็น i7 Ballerina, i8 Furiosa, i2 Inside Out 2

ขั้นที่ 3 คะแนนของรายการที่เป็นตัวเลขของ u1 (L_{u1} = {i1, i2, i3})

- i7: $\text{sim}(i1, i7) + \text{sim}(i2, i7) + \text{sim}(i3, i7) = 0.7071 + 0 + 0 = 0.7071$
- i8 ได้ 0.7071 (จาก i1), i9 ได้ 0.7071 (จาก i3 เพราะ u5 ชอบทั้งคู่), i5 ได้ 0.5000 (จาก i2 เพราะ u6 ชอบทั้งคู่)
- i10, i11 และ i12 ได้ 0 เพราะไม่มีใครชอบทั้งเรื่องเหล่านี้และเรื่องของ u1

ขั้นที่ 4 ตีความ

Top-3 คือ i7, i8 และ i9 (เสมอกันที่ 0.7071 จึงเรียงตามรหัส) เป็นสามเรื่องเดียวกับ Item-kNN แบบคะแนน แม้ระบบนี้ใช้เพียงข้อมูลว่า "ชอบหรือไม่" ข้อจำกัดคือคะแนนเสมอกันเกิดง่ายมาก ระบบจริงจึงเพิ่มสัญญาณอื่น เช่น ความใหม่ของปฏิสัมพันธ์หรือจำนวนครั้งที่ดู เพื่อแยกคะแนนที่เสมอกัน บนโปรโตคอล LOO ของหัวข้อ 5.13 วิธีนี้ได้ HR@3 0.6000 และ MRR 0.5327

โปรแกรม 5.5 Item-kNN แบบโดยปริยายด้วยการคูณเมทริกซ์

```
import numpy as np


R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
items = ['i%d' % k for k in range(1, 13)]

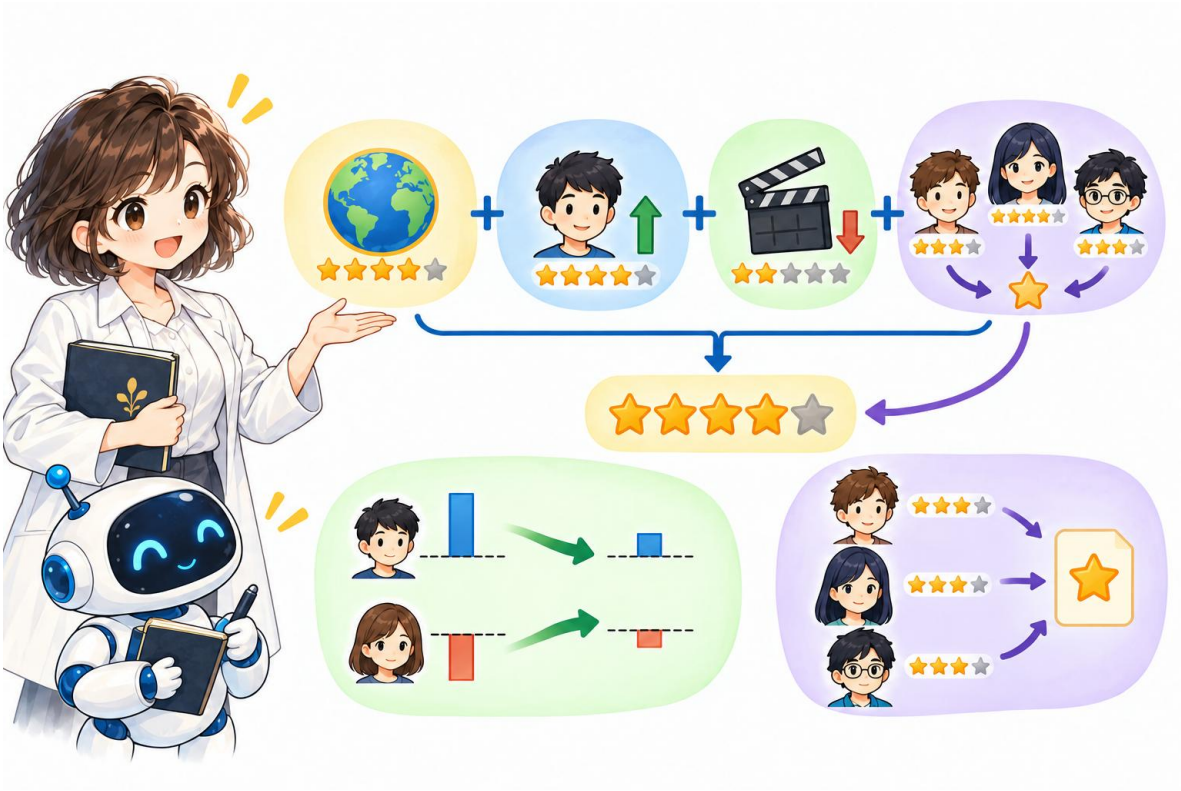

# เมทริกซ์การชอบ X (8 x 12)
X = np.array([[1.0 if R[u].get(i, 0) >= 4 else 0.0 for i in items]
              for u in R])
# G[i, j] = จำนวนผู้ใช้ที่ชอบทั้ง i และ j
```

```
G = X.T @ X
n = np.sqrt(np.diag(G))
n[n == 0] = 1
# ความคล้ายแบบโดยปริยาย (โคไซน์ของคอลัมน์)
S = G / n[:, None] / n[None, :]
np.fill_diagonal(S, 0)
order = sorted(range(12), key=lambda b: (-S[0, b], b))[:3]
print('คล้าย i1:', [(items[b], round(float(S[0, b]), 4)) for b in order])


# คะแนน = ผลรวมความคล้ายกับเรื่องที่ u1 ชอบ
score = X[0] @ S
print({items[b]: round(float(score[b]), 4)
       for b in range(12) if items[b] not in R['u1']})
```

5.6 แบบจำลองค่าเอนเอียงและ kNN-Baseline

หัวข้อ 5.4–5.5 ไม่มี "การเรียนรู้" ระบบคำนวณจากคะแนนโดยตรงทุกครั้ง หัวข้อนี้เป็นแบบจำลองแรกที่ **เรียนพารามิเตอร์** จากข้อมูล และเป็นฐานของการแยกตัวประกอบเมทริกซ์ในหัวข้อ 5.7 ดังสรุปในภาพที่ 5.32 ภาพนี้สรุปองค์ประกอบของคะแนน SHRINKAGE ค่าประมาณ และการเติมส่วนที่เหลือด้วยข้อมูลเพื่อนบ้าน



ภาพที่ 5.32 ค่าเอนเอียงและ kNN Baseline

5.6.1 คะแนนประกอบด้วยสามส่วน

ก่อนจะเรียนรสนิยมที่ซับซ้อน ระบบควรอธิบายส่วนที่ง่ายที่สุดของคะแนนให้ได้ก่อน คะแนนจำนวนมากอธิบายได้ด้วยสามส่วน

- **ค่าเฉลี่ยรวม** μ ของทั้งระบบ เช่น ผู้ใช้แอปนี้ให้คะแนนเฉลี่ย 3.625
- **ค่าเอนเอียงของผู้ใช้** b_u บอกว่าผู้ใช้ใจดี ($b_u > 0$) หรือเข้มงวด ($b_u < 0$) กว่าคนทั่วไป

- **ค่าเอนเอียงของรายการ** b_i บอกว่ารายการได้รับความนิยมสูง ($b_i > 0$) หรือต่ำ ($b_i < 0$) กว่าเรื่องทั่วไป

$$\hat{r}_{ui} = \mu + b_u + b_i \tag{5.12}$$

โดยที่

- μ คือค่าเฉลี่ยรวมของทั้งระบบ
- b_u คือค่าเอนเอียงของผู้ใช้ ค่าบวกแปลว่าใจดีกว่าคนทั่วไป ค่าลบแปลว่าเข้มงวดกว่า
- b_i คือค่าเอนเอียงของรายการ ค่าบวกแปลว่าได้รับความนิยมมากกว่าเรื่องทั่วไป

สมการ (5.12) เรียกว่า **แบบจำลองค่าเอนเอียง (Baseline Predictor)** (Koren, 2010) ตัวอย่างเช่น ถ้า $\mu = 3.6$ ผู้ใช้เข้มงวด $b_u = -0.3$ และภาพยนตร์ยอดนิยม $b_i = +0.5$ คำทำนายคือ $3.6 - 0.3 + 0.5 = 3.8$

5.6.2 ที่มาของสูตรค่าเอนเอียงและ SHRINKAGE

ค่าเอนเอียงหาได้จากการ **ลดค่าสูญเสียกำลังสอง** เริ่มจาก b_i ของรายการ i หนึ่งเรื่อง โดยตั้ง $b_u = 0$ ไว้ก่อน แล้วเพิ่มบทลงโทษ λb_i^2 เพื่อไม่ให้ค่าใหญ่เกินเมื่อหลักฐานน้อย

$$L(b_i) = \sum_{u \in \mathcal{U}_i} (r_{ui} - \mu - b_i)^2 + \lambda\, b_i^2 \tag{5.13}$$

โดยที่

- $L(b_i)$ คือค่าสูญเสียของรายการ i ซึ่งต้องการทำให้น้อยที่สุด
- $\mathcal{U}_i$ คือผู้ใช้ที่ให้คะแนนรายการ i
- λ คือค่า SHRINKAGE ยิ่งมากยิ่งดึง b_i เข้าหาศูนย์

การหาค่าต่ำสุดทำได้สามขั้น

1. **หาอนุพันธ์** ตามกฎลูกโซ่ $\frac{dL}{db_i} = -2\sum_{u \in \mathcal{U}_i} (r_{ui} - \mu - b_i) + 2\lambda b_i$
2. **ตั้งให้เป็นศูนย์** แล้วหารด้วย 2 ได้ $\sum_u (r_{ui} - \mu) - |\mathcal{U}_i|\, b_i - \lambda b_i = 0$
3. **แก้หา** b_i ได้ $b_i(|\mathcal{U}_i| + \lambda) = \sum_u (r_{ui} - \mu)$ ซึ่งคือสมการแรกของ (5.14)

$$b_i = \frac{\sum_{u \in \mathcal{U}_i} (r_{ui} - \mu)}{\lambda + |\mathcal{U}_i|}, \qquad b_u = \frac{\sum_{i \in \mathcal{I}_u} (r_{ui} - \mu - b_i)}{\lambda + |\mathcal{I}_u|} \tag{5.14}$$

โดยที่

- $|\mathcal{U}_i|$ คือจำนวนผู้ใช้ที่ให้คะแนนรายการ i และ $|\mathcal{I}_u|$ คือจำนวนรายการที่ผู้ใช้ u ให้คะแนน
- $r_{ui} - \mu$ คือส่วนที่คะแนนต่างจากค่าเฉลี่ยรวม และ $r_{ui} - \mu - b_i$ คือส่วนที่เหลือหลังหักค่าเอนเอียงของรายการ
- λ ในตัวส่วนคือ SHRINKAGE เปรียบได้กับผู้ชมสมมติ λ คนที่ให้คะแนนเท่าค่าเฉลี่ยพอดี

สมการที่สองของ (5.14) ได้ด้วยวิธีเดียวกัน โดยถือว่า b_i รู้แล้ว และใช้ "ส่วนที่เหลือ" $r_{ui} - \mu - b_i$ เป็นเป้าหมาย ลำดับ "รายการก่อน ผู้ใช้ทีหลัง" มาจาก Koren (2010) เพราะรายการมักมีคะแนนมากกว่าผู้ใช้ จึงประมาณได้แม่นกว่า

ตัวอย่างการคำนวณ 5.16 ตรวจว่าสูตรให้ค่าที่ทำให้ค่าสูญเสียต่ำที่สุดจริง

โจทย์ รายการ i9 มีคะแนนเดียวคือ 5 จงหา $L(b_{i9})$ ที่ $\lambda = 2$ สำหรับ b_{i9} หลายค่า แล้วหาค่าที่ดีที่สุด

สมการที่ใช้ สมการ (5.13) และ (5.14)

ข้อมูลที่ใช้ $\mu = 3.625$ จากตัวอย่างการคำนวณ 5.2

ขั้นที่ 1 ค่าจากสูตร

- $b_{i9} = (5 - 3.625)/(2 + 1) = 1.375/3 = 0.4583$

ขั้นที่ 2 แทนค่าในค่าสูญเสีย

- ที่ $b = 0.0000$: $(1.375 - 0.0000)^2 + 2(0.0000)^2 = 1.8906$
- ที่ $b = 0.2500$: $(1.375 - 0.2500)^2 + 2(0.2500)^2 = 1.3906$
- ที่ $b = 0.4583$: $(1.375 - 0.4583)^2 + 2(0.4583)^2 = 1.2604$
- ที่ $b = 0.7500$: $(1.375 - 0.7500)^2 + 2(0.7500)^2 = 1.5156$
- ที่ $b = 1.3750$: $(1.375 - 1.3750)^2 + 2(1.3750)^2 = 3.7812$

ขั้นที่ 3 ตีความ

ค่าสูญเสียต่ำที่สุด (1.2604) อยู่ที่ $b_{i9} = 0.4583$ ตามสูตรพอดี ค่าที่ไม่หด ($b = 1.375$) อธิบายคะแนน 5 ได้สมบูรณ์ แต่เสียบทลงโทษ $2 \times 1.375^2 = 3.7813$ ผลคือ λ ทำหน้าที่ "ต่อรอง" ระหว่างการอธิบายข้อมูลกับการไม่เชื่อหลักฐานชิ้นเดียวมากเกินไป

อธิบายแบบเข้าใจง่าย λ คือผู้ชมสมมติที่ให้คะแนนเท่าค่าเฉลี่ย

สูตร $b_i = \sum(r - \mu)/(\lambda + n)$ มีความหมายง่าย ๆ ว่า "นอกจากผู้ชมจริง n คนแล้ว ให้นับผู้ชมสมมติอีก λ คนที่ให้คะแนนเท่าค่าเฉลี่ยพอดี"

ภาพยนตร์ที่มีผู้ชมจริงหลายพันคน ผู้ชมสมมติสองคนแทบไม่มีผล แต่ภาพยนตร์ที่มีผู้ชมจริงคนเดียว ผู้ชมสมมติสองคนจะดึงค่าเฉลี่ยกลับเข้าหาค่ากลางอย่างมาก นี่คือเหตุผลที่ Conclave ไม่ได้ค่าเอนเอียงสูงเกินจริง

5.6.3 คำนวณค่าเอนเอียงของ MovieMini-12

ตัวอย่างการคำนวณ 5.17 ค่าเอนเอียงของ i9, i5 และ u1 ด้วย $\lambda = 2$

โจทย์ จงหา b_{i9}, b_{i5}, b_{u1} และคำทำนาย $\hat{r}_{u1,i9}$

สมการที่ใช้ สมการ (5.14) แล้วสมการ (5.12)

ข้อมูลที่ใช้ $\mu = 3.6250$ จากตัวอย่างการคำนวณ 5.2 และตารางที่ 5.5

ขั้นที่ 1 ค่าเอนเอียงของรายการ

- i9 (คะแนน 5): $b_{i9} = (5 - 3.625)/(2 + 1) = 0.4583$ ถ้าไม่หด ($\lambda = 0$) จะได้ 1.3750
- i5 (คะแนน 4, 1, 4): $b_{i5} = (0.375 - 2.625 + 0.375)/(2 + 3) = -1.875/5 = -0.3750$

ขั้นที่ 2 ค่าเอนเอียงของ u1 (ใช้ b_i ของทุกเรื่องที่ u1 ให้คะแนน)

- i1: $5 - 3.625 - (0.0250) = 1.3500$
- i2: $5 - 3.625 - (0.2250) = 1.1500$
- i3: $4 - 3.625 - (0.0250) = 0.3500$
- i4: $1 - 3.625 - (-0.1750) = -2.4500$
- i6: $1 - 3.625 - (-0.0833) = -2.5417$
- $b_{u1} = -2.1417/(2 + 5) = -0.3060$

ขั้นที่ 3 ทำนายด้วยสมการ (5.12)

$$\hat{r}_{u1,i9} = 3.625 + (-0.3060) + 0.4583 = 3.7774$$

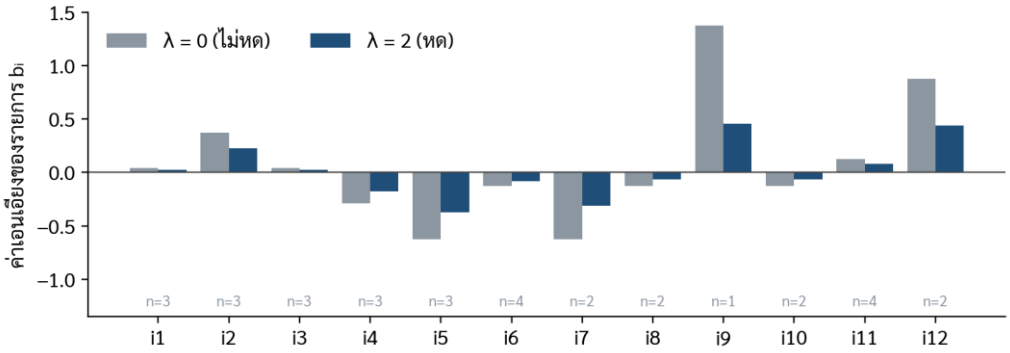
ขั้นที่ 4 ตีความ

u1 ให้คะแนนต่ำกว่าค่าเฉลี่ยของระบบเล็กน้อย ($b_{u1} < 0$) เพราะให้ 1 สองเรื่อง ส่วน Conclave ได้ค่าเอนเอียงบวกสูงสุดในคลัง เพราะมีคนให้ 5 แต่ SHRINKAGE ลดค่านี้จาก 1.3750 เหลือ 0.4583

ตารางที่ 5.16 ค่าเอนเอียงของรายการเมื่อไม่หดและเมื่อหด

รายการ	n	b_i (λ = 0)	b_i (λ = 2)
i1	3	0.0417	0.0250
i2	3	0.3750	0.2250
i3	3	0.0417	0.0250
i4	3	−0.2917	−0.1750
i5	3	−0.6250	−0.3750
i6	4	−0.1250	−0.0833
i7	2	−0.6250	−0.3125
i8	2	−0.1250	−0.0625
i9	1	1.3750	0.4583
i10	2	−0.1250	−0.0625
i11	4	0.1250	0.0833
i12	2	0.8750	0.4375

ที่มา: สมการ (5.14) ด้วย μ = 3.625



ภาพที่ 5.10 ผลของ SHRINKAGE ค่าเอนเอียงรายการ

ตารางที่ 5.16 และภาพที่ 5.10 แสดงว่า SHRINKAGE ลดค่าเอนเอียงของรายการที่มีคะแนนน้อยมากกว่ารายการที่มีคะแนนมาก

- i9 (n = 1) ถูกลดเหลือหนึ่งในสาม และ i12 (n = 2) ถูกลดเหลือครึ่งหนึ่ง เพราะตัวคูณคือ $n/(n+\lambda)$

- i6 และ i11 (n = 4) ถูกลดเหลือสองในสาม ซึ่งน้อยกว่า
- แบบจำลองนี้ไม่มีส่วนใดเป็นรสนิยมเฉพาะตัว **ผู้ใช้ทุกคนได้อันดับรายการเหมือนกัน** ต่างกันแค่ระดับคะแนน จึงเป็นเพียงจุดเริ่มต้นของแบบจำลองในหัวข้อ 5.7

ที่มาของไฮเปอร์พารามิเตอร์ ค่า SHRINKAGE λ ของแบบจำลองค่าเอนเอียง

λ มาจากไหน Koren (2010) ใช้ λ ประมาณ 25 สำหรับรายการและ 10 สำหรับผู้ใช้บนข้อมูล Netflix Prize ซึ่งรายการมีคะแนนหลายพันค่า ความหมายตามกล่องอธิบายแบบเข้าใจง่ายคือ "ผู้ชมสมมติ 25 คน" ซึ่งเล็กมากเมื่อเทียบกับผู้ชมจริง

ผลบน MovieMini-12 การกันออกทีละคะแนน (หัวข้อ 5.13.1) ให้ความคลาดเคลื่อนดังตารางต่อไปนี้

λ	MAE	RMSE
0	2.1248	2.3972
2	1.7139	1.9531
5	1.5229	1.7424
10	1.4171	1.6245

ยิ่งหดมาก ความคลาดเคลื่อนยิ่งลด เพราะข้อมูลน้อยจนค่าเฉลี่ยของรายการเชื่อถือไม่ได้ บทนี้ใช้ λ = 2 ในตัวอย่างเพื่อให้ตัวเลขอ่านง่าย แต่ผลข้างต้นบอกว่าบนข้อมูลชุดนี้ค่าที่ใหญ่กว่าจะดีกว่า

โปรแกรม 5.6 แบบจำลองค่าเอนเอียงและการตรวจค่าสูญเสีย

```
import numpy as np


R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}


items = ['i%d' % k for k in range(1, 13)]
lam = 2.0
mu = np.mean([r for d in R.values() for r in d.values()])


# ค่าเอนเอียงของรายการ (คำนวณก่อน)
bi = {}
for i in items:
    dev = [R[u][i] - mu for u in R if i in R[u]]
    bi[i] = sum(dev) / (lam + len(dev))


# ค่าเอนเอียงของผู้ใช้ (ใช้ส่วนที่เหลือหลังหัก bi)
bu = {}
for u, d in R.items():
    dev = [r - mu - bi[i] for i, r in d.items()]
    bu[u] = sum(dev) / (lam + len(dev))


print('mu', mu, 'bi9', round(bi['i9'], 4), 'bi5', round(bi['i5'], 4))
print('bu1', round(bu['u1'], 4))
```

```
def loss_i9(b):
    """ค่าสูญเสียของ i9 (คะแนนเดียวคือ 5)"""
    return (5 - mu - b) ** 2 + lam * b ** 2


for b in (0, 0.25, bi['i9'], 0.75, 1.375):
    print('b =', round(float(b), 4), 'L =', round(float(loss_i9(b)), 4))
CANDS = ['i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12']
print({i: round(float(mu + bu['u1'] + bi[i]), 4) for i in CANDS})
```

5.6.4 kNN-Baseline: ให้เพื่อนบ้านอธิบายส่วนที่เหลือ

แบบจำลองค่าเอนเอียงอธิบายส่วนที่ง่ายของคะแนน แต่ไม่จับรสนิยมเฉพาะคู่ ส่วน Item-kNN จับรสนิยมเฉพาะคู่ แต่เมื่อไม่มีเพื่อนบ้านก็ถอยไปใช้ค่าเฉลี่ยของผู้ใช้ซึ่งหยาบมาก **kNN-Baseline** รวมสองแนวทางเข้าด้วยกัน ให้ค่าเอนเอียงอธิบายส่วนที่ง่าย แล้วให้เพื่อนบ้านอธิบายเฉพาะ **ส่วนที่เหลือ** $r_{uj} - b_{uj}$ ซึ่งเป็นส่วนประกอบหนึ่งของระบบที่ชนะ Netflix Prize (Koren, 2010)

$$\hat{r}_{ui} = b_{ui} + \frac{\sum_{j \in N_k(i;u)} \mathrm{sim}'(i,j)\,(r_{uj} - b_{uj})}{\sum_{j \in N_k(i;u)} \mathrm{sim}'(i,j)}, \qquad b_{ui} = \mu + b_u + b_i \tag{5.15}$$

โดยที่

- $b_{ui} = \mu + b_u + b_i$ คือค่าฐานของคู่ (u, i) จากแบบจำลองค่าเอนเอียงในสมการ (5.12)
- $r_{uj} - b_{uj}$ คือส่วนที่คะแนนจริงของรายการเพื่อนบ้าน j ต่างจากค่าฐาน
- $N_k(i;u)$ และ $\mathrm{sim}'(i,j)$ มีความหมายเดียวกับสมการ (5.10)

ความคล้ายและเพื่อนบ้านใช้นิยามเดียวกับสมการ (5.10) ถ้าไม่มีเพื่อนบ้าน คำทำนายจะถอยกลับไปที่ b_{ui} แทน $\bar{r}_u$ ซึ่งดีกว่า เพราะยังแยกความนิยมของรายการได้

ตัวอย่างการคำนวณ 5.18 ทำนายคะแนนของ u1 ต่อ i8 ด้วย kNN-Baseline

โจทย์ จงหา $\hat{r}_{u1,i8}$

สมการที่ใช้ สมการ (5.15)

ข้อมูลที่ใช้ ค่าเอนเอียงจากตัวอย่างการคำนวณ 5.17 และตารางที่ 5.16 ความคล้ายจากตัวอย่างการคำนวณ 5.13

ขั้นที่ 1 ค่าฐาน

- $b_{u1,i8} = 3.625 + (-0.3060) + (-0.0625) = 3.2565$
- $b_{u1,i1} = b_{u1,i3} = 3.625 + (-0.3060) + 0.0250 = 3.3440$

ขั้นที่ 2 ส่วนที่เหลือของรายการเพื่อนบ้าน (i1 และ i3 ความคล้ายหลังถ่วง 0.3333)

- $r_{u1,i1} - b_{u1,i1} = 5 - 3.3440 = 1.6560$
- $r_{u1,i3} - b_{u1,i3} = 4 - 3.3440 = 0.6560$

ขั้นที่ 3 แทนค่า

$$\hat{r}_{u1,i8} = 3.2565 + \frac{0.3333(1.6560) + 0.3333(0.6560)}{0.6667} = 4.4125$$

ขั้นที่ 4 ตีความ

คำทำนาย 4.4125 อยู่ระหว่าง Item-kNN (4.5000) กับค่าฐาน (3.2565) เพราะแบบจำลองรู้ว่า u1 ให้ Toy Story 5 สูงกว่าที่ค่าฐานคาดไว้ 1.656 คะแนน จึงคาดว่า u1 จะชอบ Furiosa มากกว่าค่าฐานในสัดส่วนใกล้กัน ส่วนรายการที่ไม่มีเพื่อนบ้าน เช่น i12 kNN-Baseline ให้ 3.7565 (ค่าฐาน) ขณะที่ Item-kNN ให้ 3.2000 (ค่าเฉลี่ยของ u1) คะแนนเสมอกันจึงลดลงมาก

อธิบายแบบเข้าใจง่าย ค่าฐานคือ "ความคาดหวังปกติ" เพื่อนบ้านคือ "ข้อยกเว้น"

ถ้าครูคนหนึ่งให้คะแนนเข้มงวด และวิชานี้ปกติได้คะแนนสูง เราคาดได้ก่อนว่านักเรียนคนหนึ่งจะได้ประมาณเท่าไร (ค่าฐาน) จากนั้นจึงดูว่านักเรียนคนนี้เคยทำได้ดีกว่าที่คาดในวิชาที่คล้ายกันหรือไม่ (เพื่อนบ้าน) kNN-Baseline บวก "ความคาดหวังปกติ" กับ "ข้อยกเว้นที่เคยเกิดในเรื่องคล้ายกัน"

โปรแกรม 5.7 kNN-Baseline แบบอิงรายการ

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}

items = ['i%d' % k for k in range(1, 13)]
lam = 2.0
mu = np.mean([r for d in R.values() for r in d.values()])

# ค่าเอนเอียงของรายการ (คำนวณก่อน)
bi = {}
for i in items:
    dev = [R[u][i] - mu for u in R if i in R[u]]
    bi[i] = sum(dev) / (lam + len(dev))

# ค่าเอนเอียงของผู้ใช้ (ใช้ส่วนที่เหลือหลังหัก bi)
bu = {}
for u, d in R.items():
    dev = [r - mu - bi[i] for i, r in d.items()]
    bu[u] = sum(dev) / (lam + len(dev))

mean = {u: np.mean(list(d.values())) for u, d in R.items()}


def adj_cos(i, j, beta=3):
    """Adjusted Cosine ที่ถ่วงน้ำหนักด้วย beta"""
    us = [u for u in R if i in R[u] and j in R[u]]
    if not us:
```

```
        return 0.0
    a = np.array([R[u][i] - mean[u] for u in us])
    b = np.array([R[u][j] - mean[u] for u in us])
    d = np.linalg.norm(a) * np.linalg.norm(b)
    s = a @ b / d if d else 0.0
    return s * min(len(us), beta) / beta



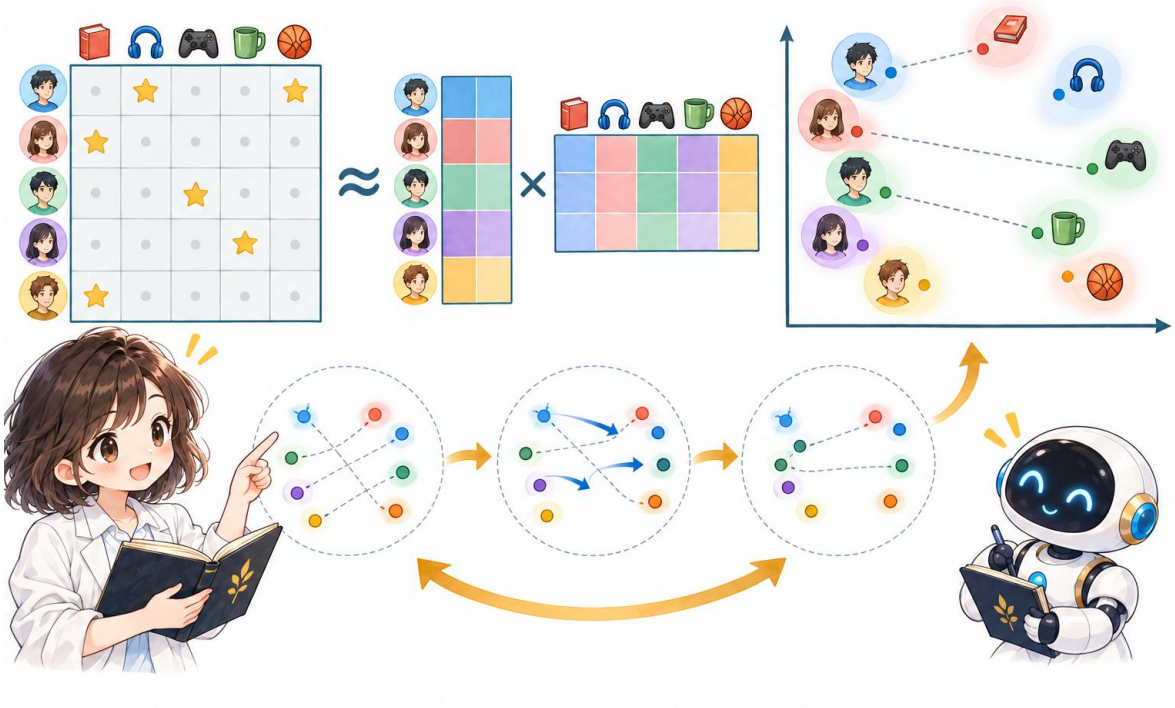
def knn_baseline(u, i, k=2):
    def base(j):
        return mu + bu[u] + bi[j]
    cand = [(adj_cos(i, j), j) for j in R[u]]
    nb = sorted([t for t in cand if t[0] > 0], key=lambda t: -t[0])[:k]
    if not nb:
        return base(i)
    # ส่วนที่เหลือ (คะแนนจริง - ค่าฐาน) ของรายการเพื่อนบ้าน
    num = sum(s * (R[u][j] - base(j)) for s, j in nb)
    den = sum(s for s, j in nb)
    return base(i) + num / den



CANDS = ['i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12']
print({i: round(float(knn_baseline('u1', i)), 4) for i in CANDS})
```

5.7 ปัจจัยแฝงและการแยกตัวประกอบเมทริกซ์ด้วย SGD

5.7.1 ปัจจัยแฝง: โปรไฟล์ผู้ใช้ในปริภูมิที่ซ่อนอยู่ ดังสรุปในภาพที่ 5.33 ภาพนี้สรุปการแยกเมทริกซ์เป็นปัจจัยผู้ใช้และรายการ การปรับซ้ำ และการควบคุมการเรียนรู้เกิน



ภาพที่ 5.33 ปัจจัยแฝงและการเรียนรู้ด้วย SGD

ข้อจำกัดของวิธีเพื่อนบ้านที่ต้องแก้ u1 กับ u3 ไม่มีรายการร่วมเลย (ตารางที่ 5.7) Pearson ของสองคนนี้จึงนิยามไม่ได้ และข้อมูลทั้งหมดของ u3 ไม่มีทางช่วยทำนาย u1 ได้ แต่ทั้งสองมี **ผู้ใช้ตัวกลาง** ร่วมกัน u1 ชอบ Toy Story 5 เช่นเดียวกับ u4 และ u4 ชอบ Ballerina ซึ่ง u3 ไม่ชอบ ห่วงโซ่ u1–u4–u3 นี้มีข้อมูลเรื่องรสนิยมที่วิธีเพื่อนบ้านมองไม่เห็น เพราะวิธีเพื่อนบ้านมองเพียงหนึ่งขั้นของความสัมพันธ์ แบบจำลองปัจจัยแฝงแก้ปัญหานี้ด้วยการวางผู้ใช้และรายการทุกตัวไว้ใน **ปริภูมิร่วม (Shared Space)** เดียวกัน

สมมติฐานข้อที่สองในหัวข้อ 5.1.2 บอกว่าความชอบอธิบายได้ด้วยปัจจัยจำนวนไม่มาก **การแยกตัวประกอบเมทริกซ์ (Matrix Factorization: MF)** นำแนวคิดนี้มาใช้ตรง ๆ คือแทนผู้ใช้แต่ละคนด้วยเวกเตอร์สั้น $\mathbf{p}_u$ และรายการแต่ละเรื่องด้วยเวกเตอร์สั้น $\mathbf{q}_i$ ขนาด k มิติ แล้วทำนายความชอบด้วยผลคูณจุดของทั้งสอง (Koren et al., 2009)

อธิบายแบบเข้าใจง่าย ปัจจัยแฝงคือส่วนผสมของรสชาติ

ลองนึกถึงอาหารที่อธิบายได้ด้วย "ความเผ็ด" กับ "ความหวาน" ลูกค้าแต่ละคนมีระดับความชอบเผ็ดและหวานของตัวเอง ($\mathbf{p}_u$) อาหารแต่ละจานมีระดับเผ็ดและหวานของตัวเอง ($\mathbf{q}_i$) ถ้าทั้งสองตรงกัน ผลคูณจุดจะสูงและลูกค้าน่าจะชอบ

MF หา "ส่วนผสมของรสชาติ" เหล่านี้เองจากคะแนน โดยไม่ต้องมีใครบอกว่าอาหารจานไหนเผ็ดเท่าไร และไม่ต้องมีใครตั้งชื่อว่ามิติไหนคือ "เผ็ด"

ตัวอย่างการคำนวณ 5.19 ผลคูณจุดของปัจจัยแฝงที่ตั้งขึ้นด้วยมือ

โจทย์ สมมติว่ามีสองปัจจัยคือ (แอนิเมชัน, แอ็กชัน) ให้ $\mathbf{p}_{u1} = (1.0, 0.2)$, $\mathbf{p}_{u4} = (0.6, 1.0)$, $\mathbf{q}_{i1} = (1.0, 0.1)$ และ $\mathbf{q}_{i8} = (0.1, 1.0)$ จงหาความชอบทั้งสี่คู่

สมการที่ใช้ ผลคูณจุด $\mathbf{p}_u^\top \mathbf{q}_i = p_{u,1} q_{i,1} + p_{u,2} q_{i,2}$

ข้อมูลที่ใช้ ค่าสมมติเพื่อการอธิบาย (ยังไม่ได้มาจากการฝึก)

- u1 กับ i1 Toy Story 5: $1.0(1.0) + 0.2(0.1) = 1.02$
- u1 กับ i8 Furiosa: $1.0(0.1) + 0.2(1.0) = 0.30$
- u4 กับ i1: $0.6(1.0) + 1.0(0.1) = 0.70$
- u4 กับ i8: $0.6(0.1) + 1.0(1.0) = 1.06$

ตีความ ตัวเลขเพียง 8 ตัว (2 มิติ × 4 เวกเตอร์) สร้างความชอบได้ 4 ค่า และสะท้อนรสนิยมว่า u1 ชอบแอนิเมชัน ส่วน u4 ชอบทั้งสองแบบแต่ชอบแอ็กชันมากกว่า เมื่อมีผู้ใช้ 8 คนและภาพยนตร์ 12 เรื่อง เวกเตอร์ 20 ตัวสร้างค่าได้ครบทั้ง 96 ช่อง MF จึงเรียน **เวกเตอร์จำนวนน้อยที่อธิบายช่องที่มีคะแนน** แล้วใช้เวกเตอร์เดียวกันเติมช่องที่ว่าง

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i \tag{5.16}$$

โดยที่

- μ, b_u และ b_i คือส่วนของค่าเอนเอียงจากสมการ (5.12)
- $\mathbf{p}_u$ คือเวกเตอร์ปัจจัยแฝงของผู้ใช้ u (โปรไฟล์ของผู้ใช้) และ $\mathbf{q}_i$ คือเวกเตอร์ปัจจัยแฝงของรายการ i ทั้งคู่มีขนาด k มิติ
- $\mathbf{p}_u^\top \mathbf{q}_i$ คือผลคูณจุด $\sum_{f=1}^{k} p_{uf}\ q_{if}$ ซึ่งวัดว่ารสนิยมของผู้ใช้ตรงกับลักษณะของรายการเพียงใด

สมการ (5.16) คือแบบจำลองค่าเอนเอียงในหัวข้อ 5.6 ที่เพิ่มพจน์ $\mathbf{p}_u^\top \mathbf{q}_i$ ซึ่งเป็น **ส่วนรสนิยมเฉพาะตัว** เวกเตอร์ $\mathbf{p}_u$ คือ **โปรไฟล์ของผู้ใช้ในปริภูมิแฝง** เทียบได้กับโปรไฟล์ข้อความหรือโปรไฟล์ภาพในบทที่ 4 ต่างกันที่มิติของปริภูมินี้ **ไม่มีใครตั้งชื่อ** แบบจำลองเรียนเองว่ามิติใดควรแยกรสนิยมแบบใด

5.7.2 เชื่อมกับการแยกค่าเอกฐาน (SVD)

ถ้าเมทริกซ์คะแนนมีค่าครบทุกช่อง พีชคณิตเชิงเส้นมีเครื่องมือที่หาเมทริกซ์แรงก์ต่ำที่ดีที่สุดได้โดยตรง คือ **การแยกค่าเอกฐาน (Singular Value Decomposition: SVD)** ซึ่งเขียนเมทริกซ์ใด ๆ เป็นผลคูณของสามเมทริกซ์ ค่าเอกฐาน $\sigma_1 \geq \sigma_2 \geq \cdots$ บอกว่าแต่ละมิติสำคัญเพียงใด

$$\mathbf{R} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top \approx \mathbf{R}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^\top, \qquad \| \mathbf{R} - \mathbf{R}_k \|_F = \sqrt{\sigma_{k+1}^2 + \sigma_{k+2}^2 + \cdots} \tag{5.17}$$

โดยที่

- $\mathbf{R}$ คือเมทริกซ์ที่เต็มทุกช่อง $\mathbf{U}$ และ $\mathbf{V}$ คือเมทริกซ์ของเวกเตอร์เอกฐานด้านซ้ายและขวา
- $\mathbf{\Sigma}$ คือเมทริกซ์ทแยงของค่าเอกฐาน $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$
- $\mathbf{R}_k$ คือเมทริกซ์อันดับ k ที่ได้จากการเก็บเฉพาะ k ค่าเอกฐานแรก ($\mathbf{U}_k$, $\mathbf{\Sigma}_k$, $\mathbf{V}_k$)
- $\|\cdot\|_F$ คือนอร์มโฟรเบเนียส (รากที่สองของผลรวมกำลังสองของทุกช่อง)

ใช้วัดความคลาดเคลื่อนของการประมาณ

ทฤษฎีบทของ Eckart และ Young (1936) บอกว่าการเก็บเพียง k ค่าเอกฐานแรกให้เมทริกซ์แรงก์ k ที่ใกล้ $\mathbf{R}$ ที่สุด ถ้าตั้ง $\mathbf{P} = \mathbf{U}_k\boldsymbol{\Sigma}_k^{1/2}$ และ $\mathbf{Q} = \mathbf{V}_k\boldsymbol{\Sigma}_k^{1/2}$ จะได้ $\mathbf{R}_k = \mathbf{P}\mathbf{Q}^{\top}$ ซึ่งเป็นรูปเดียวกับผลคูณจุดของ MF

ตัวอย่างการคำนวณ 5.20 SVD แบบตัดทอนของเมทริกซ์คะแนนขนาด 3 × 3 ที่ครบทุกช่อง

โจทย์ ผู้ใช้ A, B, C ให้คะแนนภาพยนตร์สามเรื่องครบทุกช่อง (เรื่อง 1 และ 2 เป็นแอนิเมชัน เรื่อง 3 เป็นแอ็กชัน) $\mathbf{R} = \begin{bmatrix} 5 & 4 & 1 \\ 4 & 5 & 2 \\ 1 & 2 & 5 \end{bmatrix}$ จงประมาณด้วยแรงก์ 1 และแรงก์ 2

สมการที่ใช้ สมการ (5.17)

ข้อมูลที่ใช้ เมทริกซ์สมมติเพื่อการอธิบาย คำนวณด้วย numpy.linalg.svd

ขั้นที่ 1 ค่าเอกฐานและพลังงานสะสม

- $\sigma = (9.9243, 4.2151, 0.8606)$
- พลังงานสะสม $\sum_{f \le k} \sigma_f^2 \,/\, \sum_f \sigma_f^2$ = 0.8418, 0.9937, 1.0000

ขั้นที่ 2 ประมาณด้วยแรงก์ 1

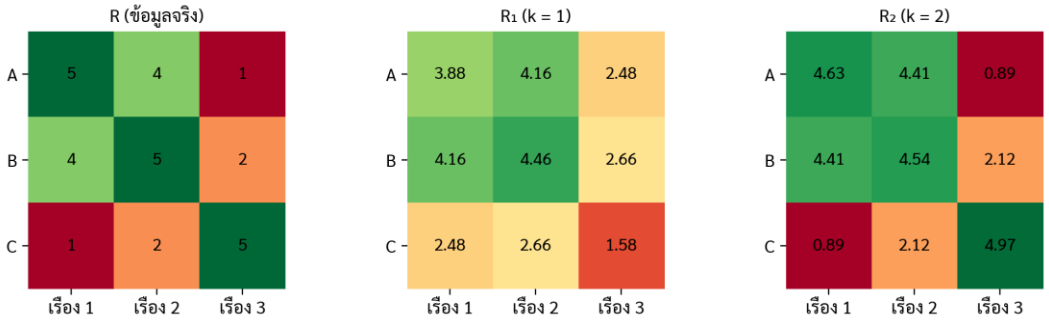
- แถวของผู้ใช้ C กลายเป็น (2.48, 2.66, 1.58) ซึ่งบอกผิดว่า C ชอบแอนิเมชันพอ ๆ กับแอ็กชัน
- ความคลาดเคลื่อน $\sqrt{4.2151^2 + 0.8606^2} = 4.3020$

ขั้นที่ 3 ประมาณด้วยแรงก์ 2

- แถวของ C กลายเป็น (0.89, 2.12, 4.97) ใกล้ค่าจริง (1, 2, 5) มาก
- ความคลาดเคลื่อนเหลือ $\sigma_3 = 0.8606$

ขั้นที่ 4 ตีความ

มิติแรกจับ "ระดับความชอบโดยรวม" ส่วนมิติที่สองแยก "แอนิเมชัน" ออกจาก "แอ็กชัน" เพียงสองมิติก็อธิบายพลังงานได้ร้อยละ 99.37 นี่คือเหตุผลที่แบบจำลองปัจจัยแฝงใช้ k เล็กได้ รสนิยมของคนจำนวนมากถูกอธิบายด้วยมิติไม่กี่มิติ



ภาพที่ 5.11 การประมาณเมทริกซ์ด้วยแรงก์ต่ำ

ภาพที่ 5.11 แสดงผลของตัวอย่างการคำนวณ 5.20 ด้วยสี แรงก์ 1 ทำให้ทุกช่องมีสีกลาง ๆ ใกล้กัน ส่วนแรงก์ 2 คืนรูปแบบ "เขียว–แดง" ของข้อมูลจริงได้เกือบครบ

ข้อควรระวัง เหตุใดจึงใช้ SVD กับเมทริกซ์คะแนนตรง ๆ ไม่ได้

- SVD นิยามบนเมทริกซ์ที่ **ครบทุกช่อง** แต่ MovieMini-12 มีคะแนนเพียง 32 จาก 96 ช่อง และระบบจริงมีน้อยกว่าร้อยละหนึ่ง

- ถ้าเติมช่องว่างด้วยศูนย์ SVD จะพยายามทำนายศูนย์ให้ช่องเหล่านั้น เท่ากับสมมติว่าผู้ใช้เกลียดทุกเรื่องที่ยังไม่ได้ดู
- ถ้าเติมด้วยค่าเฉลี่ยของผู้ใช้ คำทำนายของ u1 ทั้งเจ็ดเรื่องจะอยู่ในช่วงแคบเพียง 2.82–3.45 ใกล้ค่า 3.2 ที่เติมเข้าไปเอง เพราะค่าที่เติม 64 ช่องครอบงำคะแนนจริง 32 ช่อง
- ทางออกของการกรองร่วมคือ **เรียนพารามิเตอร์จากช่องที่รู้ค่าเท่านั้น** ดังหัวข้อ 5.7.3 คำว่า "SVD" ในไลบรารี เช่น surprise.SVD จึงหมายถึง MF ที่ฝึกด้วย SGD ไม่ใช่ SVD ของพีชคณิตเชิงเส้น

5.7.3 ค่าสูญเสียของ MF

$$\min_{\mathbf{b},\mathbf{P},\mathbf{Q}} \sum_{(u,i)\in\mathcal{K}} (r_{ui} - \hat{r}_{ui})^2 + \lambda\left(b_u^2 + b_i^2 + \| \mathbf{p}_u \|^2 + \| \mathbf{q}_i \|^2\right) \tag{5.18}$$

โดยที่

- $\mathcal{K}$ คือเซตของคะแนนที่มีอยู่จริง และ $\hat{r}_{ui}$ คือคำทำนายจากสมการ (5.16)
- $\| \mathbf{p}_u \|^2$ คือผลรวมกำลังสองของสมาชิกในเวกเตอร์ (ขนาดของเวกเตอร์ยกกำลังสอง)
- λ คือค่าเรกูลาไรเซชัน ทำหน้าที่ลงโทษพารามิเตอร์ที่มีขนาดใหญ่

สมการ (5.18) มีสองส่วน

- **ส่วนความคลาดเคลื่อน** ผลรวมของ $(r - \hat{r})^2$ คิดเฉพาะคะแนนที่มีอยู่จริง $\mathcal{K}$ ทั้ง 32 ค่า ไม่คิดช่องว่าง
- **ส่วนเรกูลาไรเซชัน (Regularization)** ลงโทษพารามิเตอร์ที่มีขนาดใหญ่ เพื่อไม่ให้แบบจำลองจำข้อมูลฝึกจนเกินไป เป็นแนวคิดเดียวกับ λ ของ SHRINKAGE ในหัวข้อ 5.6.2

MovieMini-12 ที่ $k = 2$ มีพารามิเตอร์ $(8 + 12) \times (2 + 1) = 60$ ตัว (เวกเตอร์สองมิติและค่าเอนเอียงหนึ่งตัวต่อผู้ใช้และรายการ) แต่มีคะแนนเพียง 32 ค่า ถ้าไม่มีเรกูลาไรเซชัน แบบจำลองจะหาค่าที่ทำให้ความคลาดเคลื่อนเป็นศูนย์ได้ง่าย ๆ ซึ่งไม่ได้แปลว่าทำนายช่องว่างได้ดี

5.7.4 ที่มาของกฎการปรับแบบ SGD

การลดตามเกรเดียนต์แบบสุ่ม (Stochastic Gradient Descent: SGD) ปรับพารามิเตอร์ทีละคะแนน โดยสุ่มลำดับของคะแนนในแต่ละรอบ (Funk, 2006) หลักการเหมือนบทที่ 4 หัวข้อ 4.3.3 คือ "เดินสวนทิศของเกรเดียนต์" ที่มาของกฎหาได้ทีละขั้นดังนี้

- **ขั้นที่ 1 ค่าสูญเสียของคะแนนเดียว** $\ell = e_{ui}^2 + \lambda(b_u^2 + b_i^2 + \| \mathbf{p}_u \|^2 + \| \mathbf{q}_i \|^2)$ โดย $e_{ui} = r_{ui} - \hat{r}_{ui}$
- **ขั้นที่ 2 อนุพันธ์ของ** e^2 ตามกฎลูกโซ่ $\frac{\partial e^2}{\partial \mathbf{p}_u} = 2e\,\frac{\partial e}{\partial \mathbf{p}_u} = 2e\left(-\mathbf{q}_i\right)$ เพราะ $\hat{r}$ มีพจน์ $\mathbf{p}_u^{\top}\mathbf{q}_i$
- **ขั้นที่ 3 รวมบทลงโทษ** $\frac{\partial \ell}{\partial \mathbf{p}_u} = -2e\,\mathbf{q}_i + 2\lambda\mathbf{p}_u$
- **ขั้นที่ 4 เดินสวนเกรเดียนต์** $\mathbf{p}_u \leftarrow \mathbf{p}_u - \eta'\frac{\partial \ell}{\partial \mathbf{p}_u} = \mathbf{p}_u + 2\eta'\left(e\,\mathbf{q}_i - \lambda\mathbf{p}_u\right)$ แล้วรวมเลข 2 เข้ากับอัตราการเรียนรู้ $\eta = 2\eta'$
- **ขั้นที่ 5 ทำแบบเดียวกันกับพารามิเตอร์อื่น** อนุพันธ์ของ $\hat{r}$ เทียบ b_u และ b_i เป็น 1 และเทียบ $\mathbf{q}_i$ เป็น $\mathbf{p}_u$ จึงได้สมการ (5.19)

$$
\begin{aligned}
e_{ui} &= r_{ui} - \hat{r}_{ui} \\
b_u \leftarrow b_u + \eta\left(e_{ui} - \lambda b_u\right), &\quad b_i \leftarrow b_i + \eta\left(e_{ui} - \lambda b_i\right) \\
\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta\left(e_{ui}\,\mathbf{q}_i - \lambda\,\mathbf{p}_u\right), &\quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta\left(e_{ui}\,\mathbf{p}_u - \lambda\,\mathbf{q}_i\right)
\end{aligned}
\tag{5.19}
$$

โดยที่

- e_{ui} คือความคลาดเคลื่อนของคำทำนาย ค่าบวกแปลว่าทายต่ำไป
- η คืออัตราการเรียนรู้ บอกว่าแต่ละก้าวขยับพารามิเตอร์มากเพียงใด
- λ คือค่าเรกูลาไรเซชันจากสมการ (5.18) และเครื่องหมาย ← หมายถึง "แทนค่าเดิมด้วยค่าใหม่"

กฎในสมการ (5.19) อ่านเป็นภาษาธรรมดาได้ว่า "ถ้าทายต่ำไป ($e > 0$) ให้ดัน $\mathbf{p}_u$ ไปทางเดียวกับ $\mathbf{q}_i$ และดัน $\mathbf{q}_i$ ไปทางเดียวกับ $\mathbf{p}_u$ เพื่อให้ผลคูณจุดโตขึ้น แล้วหดทุกค่าลงเล็กน้อยตาม λ" ถ้าทายสูงไป ($e < 0$) ทุกอย่างกลับทิศ

ตัวอย่างการคำนวณ 5.21 SGD หนึ่งก้าวบนคะแนน r(u1, i1) = 5

โจทย์ เริ่มที่ $b_u = b_i = 0$, $\mathbf{p}_{u1} = (0.1, -0.1)$, $\mathbf{q}_{i1} = (0.2, 0.1)$, $\eta = 0.05$ และ $\lambda = 0.05$ จงปรับพารามิเตอร์หนึ่งก้าว แล้วตรวจว่าคำทำนายดีขึ้นจริง

สมการที่ใช้ สมการ (5.16) และ (5.19)

ข้อมูลที่ใช้ $\mu = 3.625$ จากตัวอย่างการคำนวณ 5.2 ค่าเริ่มต้นเป็นค่าสมมติขนาดเล็ก

ขั้นที่ 1 ทายและหาความคลาดเคลื่อน

- $\hat{r} = 3.625 + 0 + 0 + (0.1)(0.2) + (-0.1)(0.1) = 3.625 + 0.01 = 3.6350$
- $e = 5 - 3.6350 = 1.3650$

ขั้นที่ 2 ปรับค่าเอนเอียง

- $b_u = 0 + 0.05(1.3650 - 0.05 \times 0) = 0.0683$ และ b_i ได้ค่าเท่ากัน

ขั้นที่ 3 ปรับเวกเตอร์ (ใช้ $\mathbf{p}_u$ ค่าเดิมในการปรับ $\mathbf{q}_i$)

- $\mathbf{p}_{u1}$ มิติ 1: $0.1 + 0.05(1.3650 \times 0.2 - 0.05 \times 0.1) = 0.1134$
- $\mathbf{p}_{u1}$ มิติ 2: $-0.1 + 0.05(1.3650 \times 0.1 - 0.05 \times (-0.1)) = -0.0929$
- $\mathbf{q}_{i1}$ มิติ 1: $0.2 + 0.05(1.3650 \times 0.1 - 0.05 \times 0.2) = 0.2063$
- $\mathbf{q}_{i1}$ มิติ 2: $0.1 + 0.05(1.3650 \times (-0.1) - 0.05 \times 0.1) = 0.0929$

ขั้นที่ 4 ตรวจผลหลังปรับ

- $\hat{r}_{\text{ใหม่}} = 3.625 + 0.0683 + 0.0683 + (0.1134, -0.0929)^{\mathsf{T}}(0.2063, 0.0929) = 3.7763$
- ค่าสูญเสียของคะแนนนี้ลดจาก 1.8667 เป็น 1.5016

ขั้นที่ 5 ตีความ

ระบบทายต่ำไป 1.365 คะแนน จึงเพิ่มค่าเอนเอียงทั้งสองฝั่ง และดึง $\mathbf{p}_{u1}$ กับ $\mathbf{q}_{i1}$ ให้ชี้ทางเดียวกันมากขึ้น ก้าวเดียวลดความคลาดเคลื่อนได้ราว 0.15 คะแนน การฝึกจริงทำซ้ำแบบนี้กับทุกคะแนนหลายร้อยรอบ ส่วนใหญ่ของการเปลี่ยนแปลงในก้าวแรกมาจากค่าเอนเอียง เพราะเวกเตอร์เริ่มต้นมีค่าเล็ก

ขั้นตอนวิธี การฝึก MF ด้วย SGD

1. สุ่ม $\mathbf{p}_u, \mathbf{q}_i$ จากการแจกแจงปกติค่าเฉลี่ย 0 ส่วนเบี่ยงเบนมาตรฐาน 0.1 และตั้ง $b_u = b_i = 0$
2. ในแต่ละรอบ (epoch) สลับลำดับของคะแนนทั้งหมดแบบสุ่ม

3. สำหรับแต่ละคะแนน คำนวณ e_{ui} แล้วปรับพารามิเตอร์ด้วยสมการ (5.19)
4. บันทึก RMSE บนข้อมูลฝึก (และบนชุดตรวจสอบถ้ามี) ทุกรอบ
5. หยุดเมื่อครบจำนวนรอบ หรือเมื่อ RMSE บนชุดตรวจสอบไม่ลดลงอีก

ข้อควรระวัง เหตุใดต้องสุ่มค่าเริ่มต้นและห้ามเริ่มที่ศูนย์

ถ้าเริ่ม $\mathbf{p}_u$ และ $\mathbf{q}_i$ ที่ศูนย์ทั้งหมด พจน์ $e\,\mathbf{q}_i$ และ $e\,\mathbf{p}_u$ ในสมการ (5.19) จะเป็นศูนย์ตลอดไป เวกเตอร์จะไม่ขยับเลย และแบบจำลองเหลือเพียงค่าเอนเอียง ในทางกลับกัน ถ้าเริ่มด้วยค่าขนาดใหญ่ การฝึกช่วงแรกจะแกว่งและอาจลู่ออก การสุ่มค่าขนาดเล็กจึงเป็นทางสายกลาง ผลที่ตามมาคือ **แบบจำลองที่ได้ขึ้นกับเมล็ดสุ่ม** หัวข้อ 5.13.3 จะแสดงว่าความแปรปรวนนี้ใหญ่เพียงใด

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนปัจจัย k อัตราการเรียนรู้ η เรกูลาไรเซชัน λ และค่าเริ่มต้น

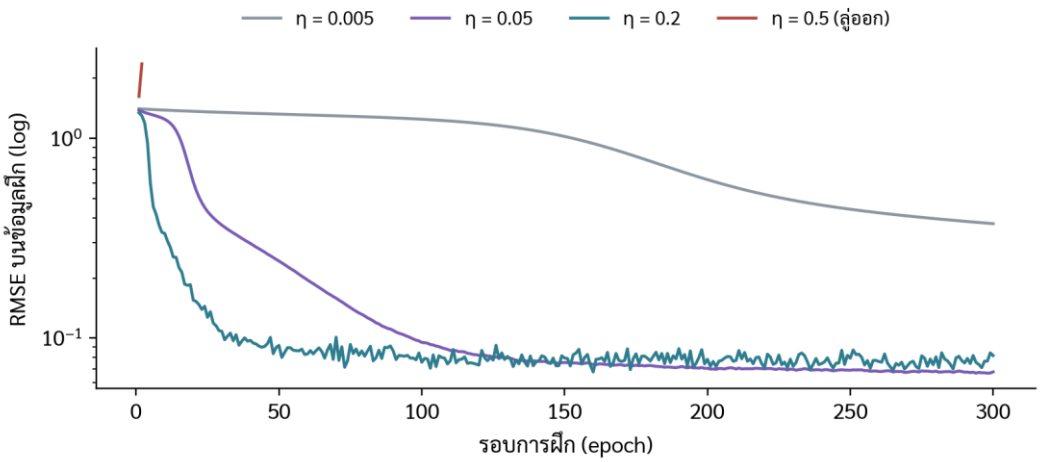
- **k มาจากไหน** งานบนข้อมูล Netflix Prize ใช้ 50–200 มิติ เพราะมีผู้ใช้ราว 480,000 คนและภาพยนตร์ 17,770 เรื่อง (Koren et al., 2009) MovieMini-12 มีคะแนนเพียง 32 ค่า จึงใช้ k = 2 ซึ่งยังให้พารามิเตอร์ 60 ตัว มากกว่าจำนวนคะแนนแล้ว
- **η มาจากไหน** FunkSVD ใช้ค่าราว 0.001 บนข้อมูลใหญ่ เพราะแต่ละรอบมีการปรับหลายล้านครั้ง ข้อมูลเล็กปรับเพียง 32 ครั้งต่อรอบ จึงใช้ค่าใหญ่กว่าได้ บทนี้เลือก η = 0.05 จากการทดลองในหัวข้อ 5.7.5
- **λ มาจากไหน** ทำหน้าที่เดียวกับ SHRINKAGE ในหัวข้อ 5.6 Funk (2006) ใช้ราว 0.02 บทนี้ใช้ 0.05 ในตัวอย่าง และศึกษาค่าอื่นในหัวข้อ 5.7.7
- **จำนวนรอบและค่าเริ่มต้น** ฝึก 300 รอบ เริ่มจากค่าสุ่มรอบศูนย์ที่ส่วนเบี่ยงเบนมาตรฐาน 0.1 ค่าเริ่มต้นเล็กป้องกันไม่ให้ผลคูณจุดใหญ่เกินตั้งแต่ต้น และต้องกำหนดเมล็ดสุ่ม (seed) เพื่อให้ทำซ้ำผลได้

5.7.5 ผลของอัตราการเรียนรู้

ตารางที่ 5.17 RMSE บนข้อมูลฝึกเมื่อใช้ η ต่างกัน

η	รอบที่ 1	รอบที่ 10	รอบที่ 50	รอบที่ 300
0.005	1.4066	1.3858	1.3232	0.3742
0.05	1.3855	1.2474	0.2432	0.0677
0.2	1.3450	0.3345	0.0920	0.0818
0.5	1.6228	ลู่ออก	ลู่ออก	ลู่ออก

ที่มา: k = 2, λ = 0.05 เมล็ดสุ่ม 7 ค่าอื่นเหมือนตารางที่ 5.18



ภาพที่ 5.12 ผลของอัตราการเรียนรู้ต่อ RMSE

ตารางที่ 5.17 และภาพที่ 5.12 แสดงพฤติกรรมสี่แบบของอัตราการเรียนรู้

- **η = 0.005 เล็กเกินไป** ค่าสูญเสียลดลงช้ามาก หลัง 300 รอบยังเหลือ RMSE 0.3742
- **η = 0.05 เหมาะสม** ลดลงอย่างราบเรียบจนเหลือ 0.0677
- **η = 0.2 ใหญ่เล็กน้อย** ลดลงเร็วมากในช่วงแรก แต่แกว่งขึ้นลงจนจบที่ 0.0818 ซึ่งสูงกว่า η = 0.05
- **η = 0.5 ใหญ่เกินไป** พารามิเตอร์ระเบิดจนคำนวณไม่ได้ (ลู่ออก) ภายในรอบที่ 4

5.7.6 ผลการฝึกบน MovieMini-12

ตารางที่ 5.18 โปรไฟล์แฝงของผู้ใช้หลังฝึก MF

ผู้ใช้	$p_{u,1}$	$p_{u,2}$	b_u
u1	1.4168	0.4508	−0.2746
u2	−0.9804	−0.6459	0.0888
u3	−0.2392	−1.4482	0.1077
u4	−0.2818	1.4088	−0.1209
u5	0.1217	0.6860	−0.2620
u6	0.9678	−0.2286	−0.1799
u7	0.6625	−0.8038	0.0421
u8	−1.0700	0.6089	−0.1609

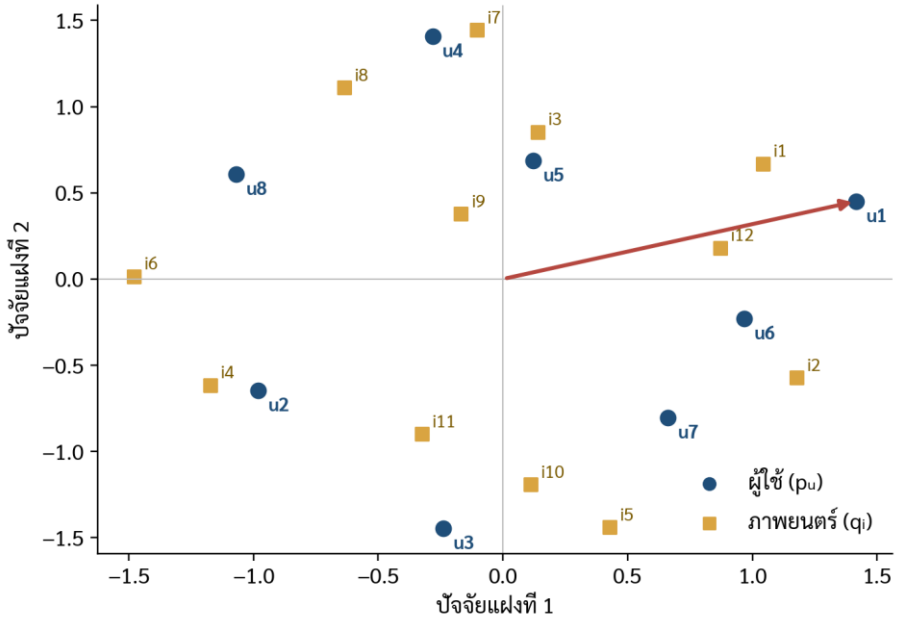
ที่มา: สมการ (5.18) และ (5.19) ที่ k = 2, η = 0.05, λ = 0.05, 300 รอบ เมล็ดสุ่ม 7

ตารางที่ 5.19 เวกเตอร์แฝงของภาพยนตร์หลังฝึก MF

รหัส	ชื่อเรื่อง	$q_{i,1}$	$q_{i,2}$	b_i
i1	Toy Story 5	1.0417	0.6685	−0.1730
i2	Inside Out 2	1.1781	−0.5729	0.2168
i3	The Wild Robot	0.1401	0.8529	0.0076

รหัส	ชื่อเรื่อง	$q_{i,1}$	$q_{i,2}$	b_i
i4	It Ends with Us	−1.1731	−0.6159	−0.3320
i5	Wicked	0.4265	−1.4400	−0.2471
i6	Bridget Jones: Mad About the Boy	−1.4792	0.0159	−0.1686
i7	Ballerina	−0.1042	1.4436	−0.5941
i8	Furiosa: A Mad Max Saga	−0.6354	1.1102	−0.2884
i9	Conclave	−0.1682	0.3776	1.3497
i10	Dune: Part Two	0.1100	−1.1928	−0.4800
i11	Mickey 17	−0.3259	−0.8999	−0.2187
i12	Project Hail Mary	0.8705	0.1806	0.7908

ที่มา: การฝึกเดียวกับตารางที่ 5.18



ภาพที่ 5.13 ปริภูมิแฝงสองมิติของ MF

ภาพที่ 5.13 วางผู้ใช้ (วงกลมสีน้ำเงิน) และภาพยนตร์ (สี่เหลี่ยมสีเหลือง) ไว้ในปริภูมิเดียวกัน ลูกศรสีแดงคือโปรไฟล์ของ u1 อ่านภาพได้ดังนี้

- ภาพยนตร์ที่อยู่ในทิศเดียวกับลูกศร เช่น i1, i12 และ i2 ได้ผลคูณจุดบวก ส่วนภาพยนตร์ฝั่งตรงข้าม เช่น i6 และ i4 ได้ผลคูณจุดลบ
- แกนนอนแยก "แอนิเมชันที่ u1 ชอบ" (ขวา) ออกจาก "ภาพยนตร์ ROMANCE " (ซ้าย)
- แกนตั้งแยก "แอ็กชัน" (บน เช่น i7, i8 และ u4) ออกจาก "แฟนตาซีและ SCI-FI" (ล่าง เช่น i5, i10, i11 และ u3)
- ไม่มีใครบอกแบบจำลองเรื่องแนวภาพยนตร์เลย โครงสร้างนี้เกิดจากคะแนนล้วน ๆ

ตัวอย่างการคำนวณ 5.22 คำทำนายของ MF สำหรับ u1 ต่อ i12 และ i5

โจทย์ จงทำนาย $\hat{r}_{u1,i12}$ และ $\hat{r}_{u1,i5}$

สมการที่ใช้ สมการ (5.16)

ข้อมูลที่ใช้ ตารางที่ 5.18 และ 5.19

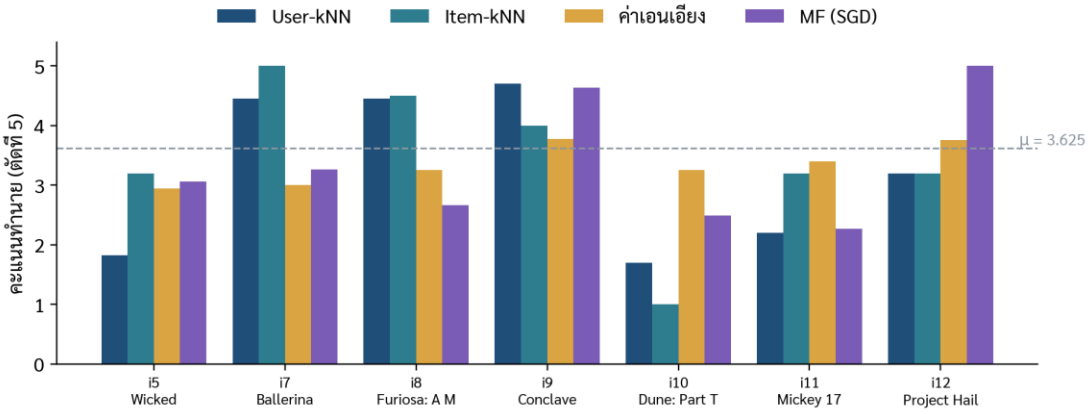
i12 Project Hail Mary

- ผลคูณจุด: $(1.4168)(0.8705) + (0.4508)(0.1806) = 1.3147$
- $\hat{r} = 3.625 + (-0.2746) + (0.7908) + (1.3147) = 5.4558$

i5 Wicked

- ผลคูณจุด: $(1.4168)(0.4265) + (0.4508)(-1.4400) = -0.0448$
- $\hat{r} = 3.625 + (-0.2746) + (-0.2471) + (-0.0448) = 3.0585$

ตีความ MF ทำนาย i12 Project Hail Mary สูงกว่า 5 (ต้องตัดเป็น 5 เมื่อแสดงผล) เพราะทั้งค่าเอนเอียงของ i12 และผลคูณจุดเป็นบวก ส่วน Wicked ได้ผลคูณจุดใกล้ศูนย์ คำทำนายจึงมาจากค่าเฉลี่ยรวมและค่าเอนเอียงเป็นหลัก คำทำนายที่เกินช่วง 1–5 เป็นสัญญาณหนึ่งว่าแบบจำลองเริ่มเรียนรู้เกินบนข้อมูลที่น้อยมาก



ภาพที่ 5.14 คำทำนายสี่วิธีสำหรับ u1

ภาพที่ 5.14 รวมคำทำนายของ u1 จากสี่วิธี (ตารางที่ 5.10, ตารางที่ 5.14, ตัวอย่างการคำนวณ 5.17 และตัวอย่างการคำนวณ 5.22) เส้นประคือค่าเฉลี่ยรวม μ ผู้อ่านควรสังเกตสามประเด็น

- วิธีเพื่อนบ้าน (แท่งสีน้ำเงินและสีเขียวอมฟ้า) ให้คำทำนายกระจายกว้าง ตั้งแต่ใกล้ 1 ถึง 5 เพราะอิงเพื่อนบ้านเพียงหนึ่งหรือสองราย
- แบบจำลองค่าเอนเอียง (สีเหลือง) ให้คำทำนายเกาะกลุ่มใกล้ μ เพราะไม่มีส่วนที่จับรสนิยมเฉพาะคู่
- MF (สีม่วง) อยู่ระหว่างสองกลุ่ม ผลคูณจุดดึง i9 และ i12 ขึ้นและดึง i8 ลง

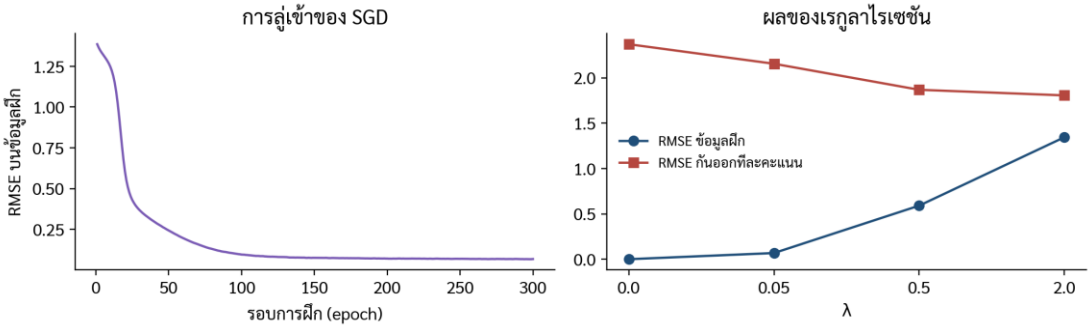
5.7.7 การเรียนรู้เกิน เรกูลาไรเซชัน และจำนวนปัจจัย

ตารางที่ 5.20 ผลของ λ ต่อความคลาดเคลื่อนและการจัดอันดับของ MF

λ	RMSE ข้อมูลฝึก	RMSE กันออกทีละคะแนน	MRR (LOO 20 กรณี)
0.0	0.0000	2.3697	0.2160

λ	RMSE ข้อมูลฝึก	RMSE กันออกทีละคะแนน	MRR (LOO 20 กรณี)
0.05	0.0677	2.1544	0.2223
0.5	0.5906	1.8675	0.3333
2.0	1.3449	1.8055	0.2037

ที่มา: หัวข้อ 5.13 อธิบายโปรโตคอลทั้งสอง



ภาพที่ 5.15 การลู่เข้าและผลของเรกูลาไรเซชัน

ตารางที่ 5.20 และภาพที่ 5.15 แสดงลักษณะของ **การเรียนรู้เกิน (Overfitting)** ชัดเจน

- ที่ λ = 0 แบบจำลองจำข้อมูลฝึกได้เกือบสมบูรณ์ (RMSE ใกล้ 0) แต่ทำนายคะแนนที่ซ่อนผิดมากที่สุด (RMSE 2.3697)
- เมื่อเพิ่ม λ ความคลาดเคลื่อนบนข้อมูลฝึกเพิ่มขึ้น แต่ความคลาดเคลื่อนบนข้อมูลที่ซ่อนลดลง
- MRR สูงสุดที่ λ = 0.5 แล้วลดลงที่ λ = 2 เพราะ SHRINKAGE มากเกินทำให้โปรไฟล์ทุกคนเข้าใกล้ศูนย์จนแยกรสนิยมไม่ได้

ตารางที่ 5.21 ผลของจำนวนปัจจัย k ต่อ MF (λ = 0.05)

k	จำนวนพารามิเตอร์	RMSE ข้อมูลฝึก	RMSE กันออกทีละคะแนน	MRR (LOO)
1	40	0.3275	2.4491	0.2421
2	60	0.0677	2.1544	0.2223
3	80	0.0635	2.2206	0.3316
5	120	0.0651	1.9635	0.3881

ที่มา: โปรโตคอลเดียวกับตารางที่ 5.20

ตารางที่ 5.21 แสดงว่า k = 1 มีพารามิเตอร์น้อยเกินจนอธิบายข้อมูลฝึกได้ไม่ดี (RMSE 0.3275) เมื่อ k ≥ 2 ข้อมูลฝึกถูกอธิบายเกือบสมบูรณ์ ส่วนผลบนข้อมูลที่ซ่อนแกว่งไปมาโดยไม่มีแนวโน้มชัดเจน เพราะ 32 คะแนนน้อยเกินกว่าจะวัดผลต่างเล็ก ๆ ได้แม่น บทเรียนคือ **อย่าเลือก k จากผลที่ต่างกันเพียงเล็กน้อยบนข้อมูลเล็ก** ควรดูช่วงความเชื่อมั่นตามบทที่ 2 หัวข้อ 2.10 ประกอบเสมอ

> **ข้อควรระวัง ค่าที่ดีสำหรับ RMSE อาจไม่ดีสำหรับการจัดอันดับ**

$\lambda = 2$ ให้ RMSE บนข้อมูลที่ซ่อนต่ำที่สุด แต่ MRR ต่ำที่สุดในตารางที่ 5.20 การเลือกไฮเปอร์พารามิเตอร์จึงต้องใช้ตัวชี้วัดที่ตรงกับงานจริงของระบบ ถ้าระบบต้องจัดอันดับ ควรเลือกด้วยตัวชี้วัดการจัดอันดับ ไม่ใช่ RMSE (Cremonesi et al., 2010)

โปรแกรม 5.8 MF ด้วย SGD

```
import numpy as np


R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
users = list(R)
items = ['i%d' % k for k in range(1, 13)]



def train_mf(Rd, k=2, lr=0.05, lam=0.05, epochs=300, seed=7):
    rng = np.random.default_rng(seed)
    P = {u: rng.normal(0, 0.1, k) for u in users}
    Q = {i: rng.normal(0, 0.1, k) for i in items}
    bu = {u: 0.0 for u in users}
    bi = {i: 0.0 for i in items}
    data = [(u, i, r) for u, d in Rd.items() for i, r in d.items()]
    mu = np.mean([r for _, _, r in data])
    for epoch in range(epochs):
        for idx in rng.permutation(len(data)):
            u, i, r = data[idx]
            # 1) ความคลาดเคลื่อน
            e = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
            # 2) ปรับค่าเอนเอียง
            bu[u] += lr * (e - lam * bu[u])
            bi[i] += lr * (e - lam * bi[i])
            # 3) ปรับเวกเตอร์ (ใช้ p_u ค่าเดิมในการปรับ q_i)
            pu = P[u].copy()
            P[u] = P[u] + lr * (e * Q[i] - lam * P[u])
            Q[i] = Q[i] + lr * (e * pu - lam * Q[i])

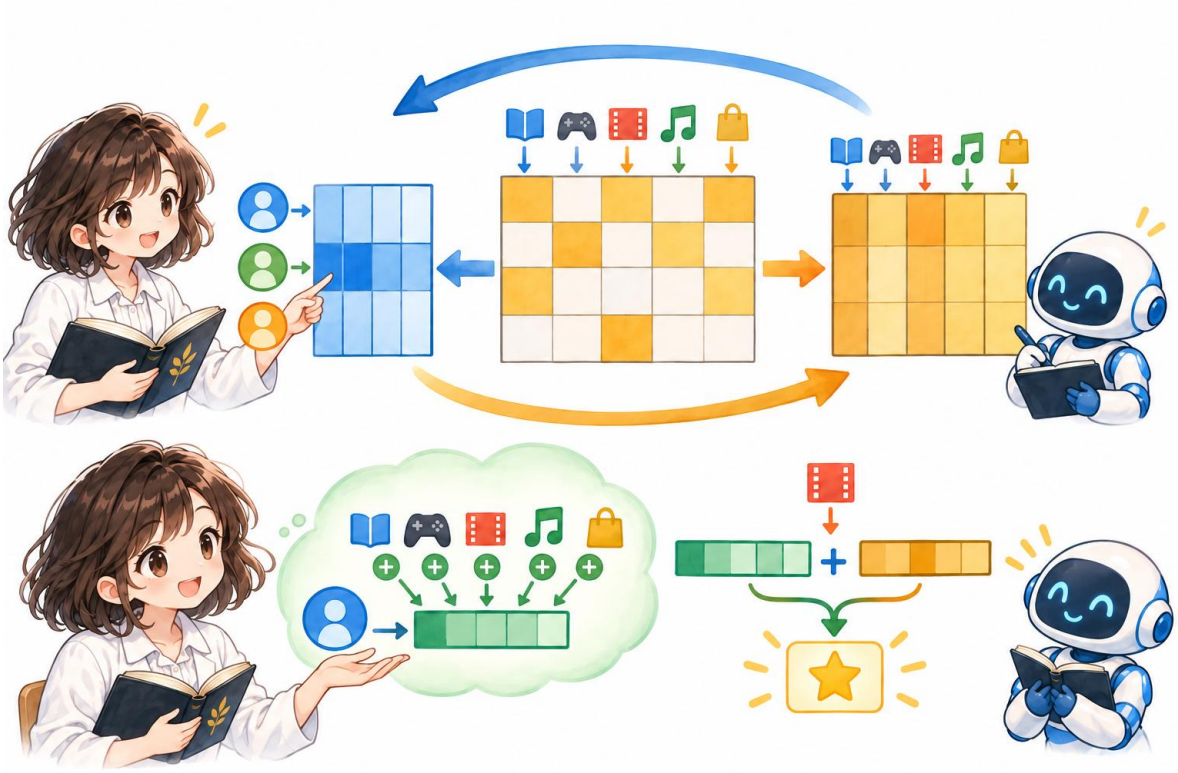

    def predict(u, i):
        return mu + bu[u] + bi[i] + P[u] @ Q[i]
    return predict, P



predict, P = train_mf(R)
print('p_u1 =', P['u1'].round(4))
print({i: round(float(predict('u1', i)), 4) for i in ('i5', 'i9', 'i12')})
```

5.8 กำลังสองน้อยสุดสลับ (ALS) และ SVD++

หัวข้อนี้ตอบคำถามสองข้อที่ตามมาจาก MF ข้อแรก มีวิธีฝึก MF ที่ไม่ต้องเลือกอัตราการเรียนรู้หรือไม่ ข้อที่สอง ใช้ข้อมูลว่า "ผู้ใช้เลือกดูอะไร" ช่วยการทำนายได้อย่างไร ดังสรุปในภาพที่ 5.34 ภาพนี้สรุปการสลับแก้ปัจจัยผู้ใช้กับรายการ และการเสริมโปรไฟล์ด้วยประวัติการมีปฏิสัมพันธ์



ภาพที่ 5.34 ALS และ SVD++

5.8.1 ALS: ที่มาของคำตอบรูปปิด

ค่าสูญเสียในสมการ (5.18) **ไม่นูน** เมื่อปรับ $\mathbf{P}$ และ $\mathbf{Q}$ พร้อมกัน เพราะมีพจน์ผลคูณ $\mathbf{p}_u^{\top}\mathbf{q}_i$ แต่ถ้า **ตรึง** $\mathbf{Q}$ **ไว้** ค่าสูญเสียของ $\mathbf{p}_u$ แต่ละคนจะกลายเป็นการถดถอยแบบสันเขา (Ridge Regression) ธรรมดาซึ่งมีคำตอบรูปปิด **ALS (Alternating Least Squares)** จึงสลับกันแก้ฝั่งผู้ใช้และฝั่งรายการจนลู่เข้า (Hu et al., 2008)

ที่มาของคำตอบหาได้สี่ขั้น โดยให้ $y_{ui} = r_{ui} - \mu - b_u - b_i$ คือส่วนที่ค่าเอนเอียงยังอธิบายไม่ได้

- **ขั้นที่ 1 ค่าสูญเสียของผู้ใช้หนึ่งคน** $L(\mathbf{p}_u) = \sum_{i \in \mathcal{I}_u} (y_{ui} - \mathbf{p}_u^{\top}\mathbf{q}_i)^2 + \lambda \parallel \mathbf{p}_u \parallel^2$
- **ขั้นที่ 2 อนุพันธ์** $\nabla L = -2\sum_i (y_{ui} - \mathbf{p}_u^{\top}\mathbf{q}_i)\,\mathbf{q}_i + 2\lambda\mathbf{p}_u$
- **ขั้นที่ 3 ตั้งเป็นศูนย์** ใช้ $\left(\mathbf{p}_u^{\top}\mathbf{q}_i\right)\mathbf{q}_i = \mathbf{q}_i\mathbf{q}_i^{\top}\mathbf{p}_u$ จะได้ $\left(\sum_i \mathbf{q}_i\,\mathbf{q}_i^{\top} + \lambda\mathbf{I}\right)\mathbf{p}_u = \sum_i y_{ui}\,\mathbf{q}_i$
- **ขั้นที่ 4 เขียนเป็นเมทริกซ์** $\sum_i \mathbf{q}_i\,\mathbf{q}_i^{\top} = \mathbf{Q}_u^{\top}\mathbf{Q}_u$ และ $\sum_i y_{ui}\,\mathbf{q}_i = \mathbf{Q}_u^{\top}\mathbf{y}_u$ จึงได้สมการ (5.20)

$$\mathbf{p}_u = (\mathbf{Q}_u^{\top}\mathbf{Q}_u + \lambda\mathbf{I})^{-1}\mathbf{Q}_u^{\top}\mathbf{y}_u \tag{5.20}$$

โดยที่

- $\mathbf{Q}_u$ คือเมทริกซ์ที่มีแถวเป็นเวกเตอร์ $\mathbf{q}_i$ ของรายการที่ผู้ใช้ u ให้คะแนน (ขนาด $|\mathcal{I}_u| \times k$)
- $\mathbf{y}_u$ คือเวกเตอร์เป้าหมาย $y_{ui} = r_{ui} - \mu - b_u - b_i$ ของรายการเหล่านั้น
- $\mathbf{I}$ คือเมทริกซ์เอกลักษณ์ขนาด $k \times k$ และ $(\cdot)^{-1}$ คือเมทริกซ์ผกผัน

$\mathbf{Q}_u$ คือแถวของ $\mathbf{Q}$ เฉพาะรายการที่ผู้ใช้ u ให้คะแนน สมการนี้คือ **สมการปกติ (Normal Equation)** ของการถดถอยเชิงเส้นที่ผู้อ่านอาจเคยพบในวิชาสถิติ ฝั่งรายการแก้ด้วยสูตรเดียวกันโดยสลับบทบาทของ $\mathbf{P}$ กับ $\mathbf{Q}$

ตัวอย่างการคำนวณ 5.23 แก้โปรไฟล์แฝงของ u1 ด้วยคำตอบรูปปิด

โจทย์ ตรึงเวกเตอร์ของภาพยนตร์และค่าเอนเอียงที่ได้จากการฝึก MF ใช้ $\lambda = 0.05$ จงหา $\mathbf{p}_{u1}$

สมการที่ใช้ สมการ (5.20) และสูตรผกผันของเมทริกซ์ 2×2

ข้อมูลที่ใช้ ตารางที่ 5.18 และ 5.19

ขั้นที่ 1 เป้าหมาย $\mathbf{y}_u$ ของห้าเรื่องที่ u1 ให้คะแนน

- i1: $\mathbf{q} = (1.0417, 0.6685)$, $y = 5 - 3.625 - (-0.2746) - (-0.1730) = 1.8227$
- i2: $\mathbf{q} = (1.1781, -0.5729)$, $y = 5 - 3.625 - (-0.2746) - (0.2168) = 1.4328$
- i3: $\mathbf{q} = (0.1401, 0.8529)$, $y = 4 - 3.625 - (-0.2746) - (0.0076) = 0.6420$
- i4: $\mathbf{q} = (-1.1731, -0.6159)$, $y = 1 - 3.625 - (-0.2746) - (-0.3320) = -2.0184$
- i6: $\mathbf{q} = (-1.4792, 0.0159)$, $y = 1 - 3.625 - (-0.2746) - (-0.1686) = -2.1818$

ขั้นที่ 2 เมทริกซ์ 2×2 ด้านซ้าย

- $\sum q_1^2 + 0.05 = 6.1066$, $\sum q_1 q_2 = 0.8398$, $\sum q_2^2 + 0.05 = 1.9320$
- $\mathbf{A} = \mathbf{Q}^\top \mathbf{Q} + 0.05\mathbf{I} = \begin{bmatrix} 6.1066 & 0.8398 \\ 0.8398 & 1.9320 \end{bmatrix}$

ขั้นที่ 3 เวกเตอร์ด้านขวา

- $\mathbf{b} = \mathbf{Q}^\top \mathbf{y} = (\sum y\, q_1, \sum y\, q_2) = (9.2715, 2.1534)$

ขั้นที่ 4 ผกผันของเมทริกซ์ 2×2

- ดีเทอร์มิแนนต์ = $6.1066 \times 1.9320 - 0.8398^2 = 11.0930$
- $\mathbf{A}^{-1} = \frac{1}{11.0930} \begin{bmatrix} 1.9320 & -0.8398 \\ -0.8398 & 6.1066 \end{bmatrix} = \begin{bmatrix} 0.1742 & -0.0757 \\ -0.0757 & 0.5505 \end{bmatrix}$

ขั้นที่ 5 คูณเพื่อหาคำตอบ

- $p_1 = 0.1742(9.2715) + (-0.0757)(2.1534) = 1.4518$
- $p_2 = (-0.0757)(9.2715) + 0.5505(2.1534) = 0.4835$
- เทียบกับผลของ SGD ในตารางที่ 5.18: $(1.4168, 0.4508)$

ขั้นที่ 6 ตีความ

คำตอบรูปปิดได้ค่าใกล้กับ SGD มาก ต่างกันเล็กน้อยเพราะ SGD ปรับค่าเอนเอียงไปพร้อมกัน ALS ไม่ต้องเลือกอัตราการเรียนรู้ และคำนวณผู้ใช้ทุกคนแยกกันได้ขนาน จึงนิยมในระบบกระจายขนาดใหญ่ เช่น Spark MLlib

ขั้นตอนวิธี การฝึกด้วย ALS

1. สุ่มค่าเริ่มต้นของ $\mathbf{Q}$
2. ตรึง $\mathbf{Q}$ แล้วแก้ $\mathbf{p}_u$ ของผู้ใช้ทุกคนด้วยสมการ (5.20) (ทำขนานกันได้)
3. ตรึง $\mathbf{P}$ แล้วแก้ $\mathbf{q}_i$ ของรายการทุกเรื่องด้วยสูตรเดียวกัน
4. ทำซ้ำขั้นที่ 2–3 จนค่าสูญเสียแทบไม่ลดลง (มักใช้ 10–20 รอบ)

อธิบายแบบเข้าใจง่าย ALS เหมือนจัดที่นั่งทีละฝั่ง

ลองนึกถึงการจัดที่นั่งให้แขกในงานเลี้ยงสองฝั่ง ถ้าขยับทั้งสองฝั่งพร้อมกันจะวุ่นวาย ALS จึงตรึงฝั่งหนึ่งไว้ แล้วจัดอีกฝั่งให้ดีที่สุด (ซึ่งคำนวณได้ทันทีด้วยสูตร) จากนั้นสลับไปตรึงอีกฝั่ง ทุกครั้งที่สลับค่าสูญเสียจะไม่เพิ่มขึ้น จึงลู่เข้าได้เสมอ

ที่มาของไฮเปอร์พารามิเตอร์ จำนวนรอบของ ALS และ $\boldsymbol{\lambda}$

- **จำนวนรอบ** ALS ลู่เข้าเร็วมาก งานส่วนใหญ่ใช้ 10–20 รอบสลับ (หนึ่งรอบคือแก้ผู้ใช้ทุกคนหนึ่งครั้งและรายการทุกเรื่องหนึ่งครั้ง) ภาพที่ 5.17 ในหัวข้อ 5.9 แสดงว่าค่าสูญเสียลดลงเกือบทั้งหมดภายในสองรอบแรก
- $\boldsymbol{\lambda}$ ทำหน้าที่เดียวกับใน SGD และยังทำให้ $\mathbf{Q}_u^\top \mathbf{Q}_u + \lambda \mathbf{I}$ หาผกผันได้เสมอ แม้ผู้ใช้จะให้คะแนนน้อยกว่า k เรื่อง ซึ่งเป็นกรณีที่ $\mathbf{Q}_u^\top \mathbf{Q}_u$ เพียงลำพังหาผกผันไม่ได้

5.8.2 เปรียบเทียบ SGD กับ ALS

ทั้งสองวิธีลดค่าสูญเสียเดียวกัน แต่มีลักษณะเชิงวิศวกรรมต่างกัน ตารางที่ 5.22 สรุปประเด็นที่ใช้ตัดสินใจ โดยทั่วไป SGD เหมาะกับการทดลองแบบจำลองใหม่ที่ต้องเพิ่มพจน์ได้ง่าย ส่วน ALS เหมาะกับข้อมูลขนาดใหญ่ที่ต้องการความขนาน และเหมาะเป็นพิเศษกับข้อมูลโดยปริยายในหัวข้อ 5.9

ตารางที่ 5.22 เปรียบเทียบ SGD กับ ALS

ประเด็น	SGD	ALS
ต้นทุนต่อรอบ	สัดส่วนกับ $\lvert\mathcal{K}\rvert\, k$	สัดส่วนกับ $\lvert\mathcal{K}\rvert\, k^2 + (m+n)\, k^3$
จำนวนรอบ	มาก (หลักร้อย) ต้องเลือก η	น้อย (หลักสิบ) ไม่มี η
การประมวลผลขนาน	ยาก ต้องแบ่งข้อมูลเป็นบล็อก	ง่าย แยกรายผู้ใช้และรายรายการได้อิสระ
ข้อมูลโดยปริยาย (ทุกช่องมีน้ำหนัก)	ช้ามาก ต้องสุ่มช่องว่าง	เหมาะ ใช้เทคนิคลดต้นทุนในหัวข้อ 5.9
การขยายแบบจำลอง	ง่าย เพิ่มพจน์ใดก็ได้ที่หาอนุพันธ์ได้	ยากกว่า ต้องหาคำตอบรูปปิดใหม่
ผู้ใช้ใหม่หลังฝึก (Fold-in)	ต้องวน SGD เพิ่ม	แก้สมการ (5.20) ครั้งเดียว

5.8.3 SVD++: ใช้ข้อมูลว่าผู้ใช้ให้คะแนนอะไรบ้าง

Koren (2008) สังเกตว่า "การที่ผู้ใช้เลือกให้คะแนนเรื่องใด" เป็นข้อมูลในตัวเอง แม้จะไม่ดูค่าคะแนน ผู้ใช้ที่เลือกดู ROMANCE หลายเรื่องบอกรสนิยมบางอย่างแล้ว แม้จะให้คะแนนต่ำก็ตาม **SVD++** จึงเพิ่มโปรไฟล์ส่วนที่สองที่สร้างจากรายการที่ผู้ใช้เคยมีปฏิสัมพันธ์

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{q}_i^\top \left(\mathbf{p}_u + \lvert N(u)\rvert^{-1/2} \sum_{j \in N(u)} \mathbf{y}_j \right) \tag{5.21}$$

โดยที่

- $N(u)$ คือเซตของรายการที่ผู้ใช้ u เคยมีปฏิสัมพันธ์ (ดูหรือให้คะแนน) และ $\lvert N(u)\rvert$ คือจำนวนรายการในเซต
- $\mathbf{y}_j$ คือเวกเตอร์ "ผลของการเลือกดูรายการ j" ที่แบบจำลองเรียนเพิ่ม
- $\lvert N(u)\rvert^{-1/2} = 1/\sqrt{\lvert N(u)\rvert}$ คือตัวปรับขนาด เพื่อไม่ให้ผู้ใช้ที่ดูมากได้ผลรวมใหญ่เกินไป

$N(u)$ คือรายการที่ผู้ใช้มีปฏิสัมพันธ์ และ $\mathbf{y}_j$ คือเวกเตอร์ "ผลของการเลือกดูรายการ j" ที่แบบจำลองเรียนเพิ่ม ผลรวมที่หารด้วย $\sqrt{|N(u)|}$ ทำหน้าที่เหมือนโปรไฟล์ค่าเฉลี่ยของบทที่ 4 แต่เวกเตอร์ของรายการเรียนจากพฤติกรรม ไม่ใช่จากเนื้อหา

ตัวอย่างการคำนวณ 5.24 โปรไฟล์ส่วนที่สองของ SVD++ สำหรับ u1

โจทย์ ให้เวกเตอร์ $\mathbf{y}_j$ ของห้าเรื่องที่ u1 ดูเป็น i1 (0.2, 0.1), i2 (0.3, −0.1), i3 (0.1, 0.2), i4 (−0.2, 0) และ i6 (−0.1, 0.1) จงหาโปรไฟล์รวมของ u1 และผลต่อคำทำนายของ i12

สมการที่ใช้ สมการ (5.21)

ข้อมูลที่ใช้ $\mathbf{y}_j$ เป็นค่าสมมติเพื่อการอธิบาย ส่วน $\mathbf{p}_{u1}$, $\mathbf{q}_{i12}$ และค่าเอนเอียงมาจากตารางที่ 5.18 และ 5.19

- ผลรวม $\sum \mathbf{y}_j = (0.2 + 0.3 + 0.1 - 0.2 - 0.1,\ 0.1 - 0.1 + 0.2 + 0 + 0.1) = (0.3, 0.3)$
- คูณด้วย $|N(u1)|^{-1/2} = 1/\sqrt{5} = 0.4472$ ได้ (0.1342, 0.1342)
- โปรไฟล์รวม = $(1.4168, 0.4508) + (0.1342, 0.1342) = (1.5510, 0.5849)$
- คำทำนาย i12 เปลี่ยนจาก 5.4558 (ตัวอย่างการคำนวณ 5.22) เป็น 5.5969 เพิ่มขึ้น $\mathbf{q}_{i12}^{\top}(0.1342, 0.1342) = 0.1410$

ตีความ ส่วนที่เพิ่มขึ้นมาจาก "สิ่งที่ u1 เลือกดู" ไม่ใช่ "คะแนนที่ u1 ให้" SVD++ ช่วยผู้ใช้ที่มีคะแนนน้อยแต่มีประวัติการดูมาก ซึ่งพบบ่อยในระบบจริง เพราะผู้ใช้ส่วนใหญ่ดูมากแต่ให้คะแนนน้อย

โปรแกรม 5.9 SVD แบบตัดทอนของเมทริกซ์ 3×3 และ ALS 2×2 ของ u1

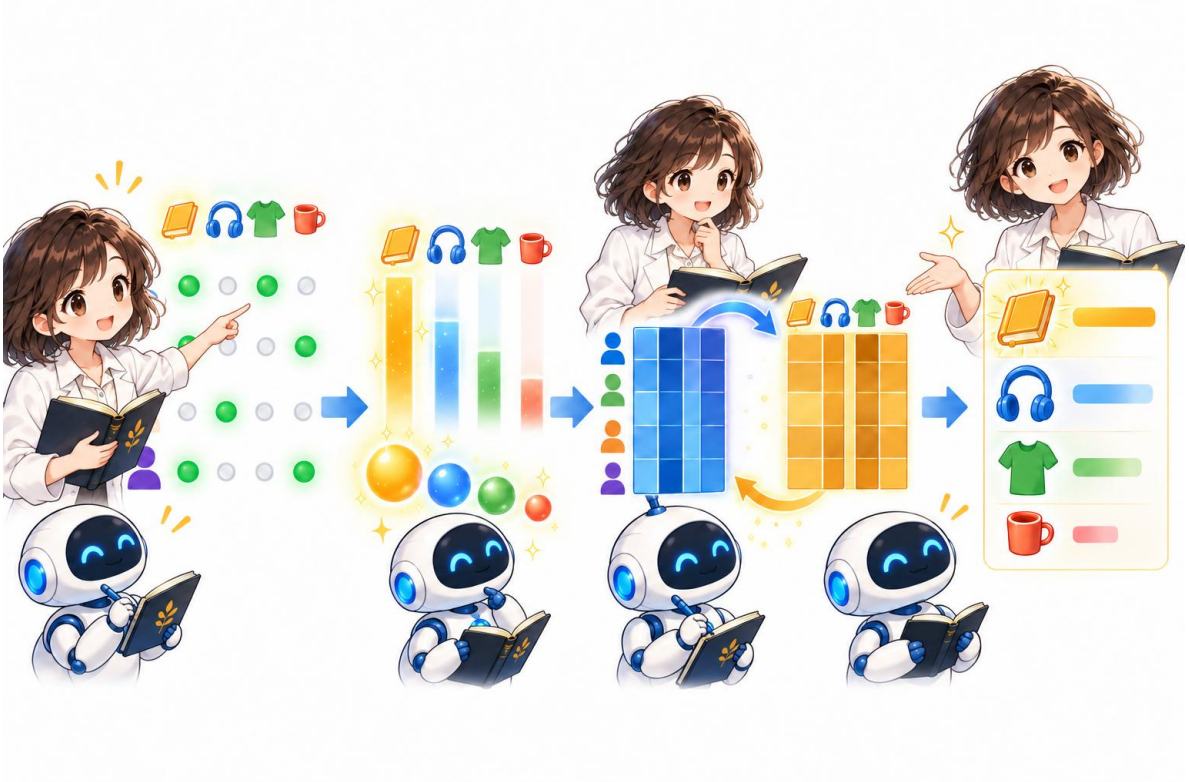
```
import numpy as np


# ส่วนที่ 1: SVD ของเมทริกซ์ 3x3 ในตัวอย่าง
M = np.array([[5.0, 4, 1],
              [4, 5, 2],
              [1, 2, 5]])
U, S, Vt = np.linalg.svd(M)
energy = np.cumsum(S ** 2) / np.sum(S ** 2)
print('sigma', S.round(4), 'energy', energy.round(4))
for k in (1, 2):
    Mk = (U[:, :k] * S[:k]) @ Vt[:k]
    err = np.linalg.norm(M - Mk)
    print('rank', k, Mk.round(2).tolist(), 'error', round(float(err), 4))


# ส่วนที่ 2: แก้ระบบ 2x2 ของ ALS สำหรับ u1
A = np.array([[6.1066, 0.8398],
              [0.8398, 1.9320]])
b = np.array([9.2715, 2.1534])
det = A[0, 0] * A[1, 1] - A[0, 1] * A[1, 0]
A_inv = np.array([[A[1, 1], -A[0, 1]],
                  [-A[1, 0], A[0, 0]]]) / det
print('det', round(float(det), 4), 'p_u1', (A_inv @ b).round(4))
```

5.9 ผลป้อนกลับโดยปริยาย: WRMF

5.9.1 ความชอบกับความเชื่อมั่น ดังสรุปในภาพที่ 5.35 ภาพนี้สรุปความชอบแบบทวิภาค น้ำหนักความเชื่อมั่น การแยกตัวประกอบแบบถ่วง และผลการจัดอันดับ



ภาพที่ 5.35 WRMF สำหรับผลป้อนกลับโดยปริยาย

เมื่อข้อมูลเป็นการคลิกหรือการดู ระบบไม่มี "คะแนนต่ำ" ให้เรียน ทุกช่องที่ไม่มีการกระทำอาจเป็นทั้ง "ไม่ชอบ" และ "ยังไม่เห็น" (หัวข้อ 5.2.5) ถ้าใช้ MF ของหัวข้อ 5.7 ที่คิดเฉพาะช่องที่มีข้อมูล ทุกช่องจะเป็น 1 แบบจำลองจะเรียนได้เพียงว่า "ทุกคนชอบทุกอย่าง"

Hu et al. (2008) จึงเสนอ **WRMF (Weighted Regularized Matrix Factorization)** หรือที่เรียกกันว่า iALS ซึ่งแยกสองแนวคิดออกจากกัน

- **ความชอบ** x_{ui} มีค่า 1 ถ้ามีการกระทำ และ 0 ถ้าไม่มี (สมการ (5.3))
- **ความเชื่อมั่น** c_{ui} บอกว่ามั่นใจในค่าความชอบนั้นเพียงใด ช่องที่มีการกระทำมาก (เช่น ดูซ้ำหลายครั้งหรือให้คะแนนสูง) ได้ความเชื่อมั่นสูง ช่องที่ไม่มีการกระทำได้ความเชื่อมั่นต่ำแต่ไม่เป็นศูนย์

$$\min_{\mathbf{P},\mathbf{Q}} \sum_{u,i} c_{ui}\,(x_{ui} - \mathbf{p}_u^{\top}\mathbf{q}_i)^2 + \lambda\Big(\sum_u \|\,\mathbf{p}_u\,\|^2 + \sum_i \|\,\mathbf{q}_i\,\|^2\Big), \qquad c_{ui} = 1 + \alpha\, r_{ui}\, x_{ui} \tag{5.22}$$

โดยที่

- x_{ui} คือความชอบ (0 หรือ 1) จากสมการ (5.3) และ c_{ui} คือความเชื่อมั่นของช่องนั้น
- α คือค่าที่บอกว่าการกระทำหนึ่งหน่วยเพิ่มความเชื่อมั่นเท่าใด
- $\sum_{u,i}$ คือผลรวมของ **ทุกช่อง** ในเมทริกซ์ ไม่ใช่เฉพาะช่องที่มีข้อมูล และ λ คือค่าเรกูลาไรเซชัน

สมการ (5.22) ต่างจาก MF สองข้อ ข้อแรก ผลรวมคิด **ทุกช่อง** ของเมทริกซ์ รวมช่องที่เป็นศูนย์ เพราะช่องศูนย์ก็เป็นข้อมูล เพียงแต่ความเชื่อมั่นต่ำ ข้อที่สอง แต่ละช่องมีน้ำหนัก c_{ui} ในบทนี้ r_{ui} คือคะแนน 4 หรือ 5 ของเรื่องที่ชอบ ส่วนในระบบจริงมักเป็นจำนวนครั้งที่ดูหรือเวลาที่ใช้

อธิบายแบบเข้าใจง่าย ทำไมต้องนับช่องที่ว่างด้วย

สมมติร้านอาหารมีเมนู 100 อย่าง ลูกค้าคนหนึ่งสั่งผัดกะเพราบ่อยมาก และไม่เคยสั่งเมนูอื่นเลย ถ้าเราสนใจเฉพาะเมนูที่สั่ง เราจะรู้แค่ว่าเขาชอบผัดกะเพรา แต่ไม่รู้ว่าเขาชอบอย่างอื่นน้อยกว่า

WRMF บอกว่า "เมนูที่ไม่เคยสั่งน่าจะชอบน้อยกว่า แต่ไม่แน่ใจนัก" จึงให้น้ำหนักต่ำ (1) กับเมนูเหล่านั้น และให้น้ำหนักสูง (เช่น 11) กับเมนูที่สั่งบ่อย

ตัวอย่างการคำนวณ 5.25 ความเชื่อมั่นของ u1

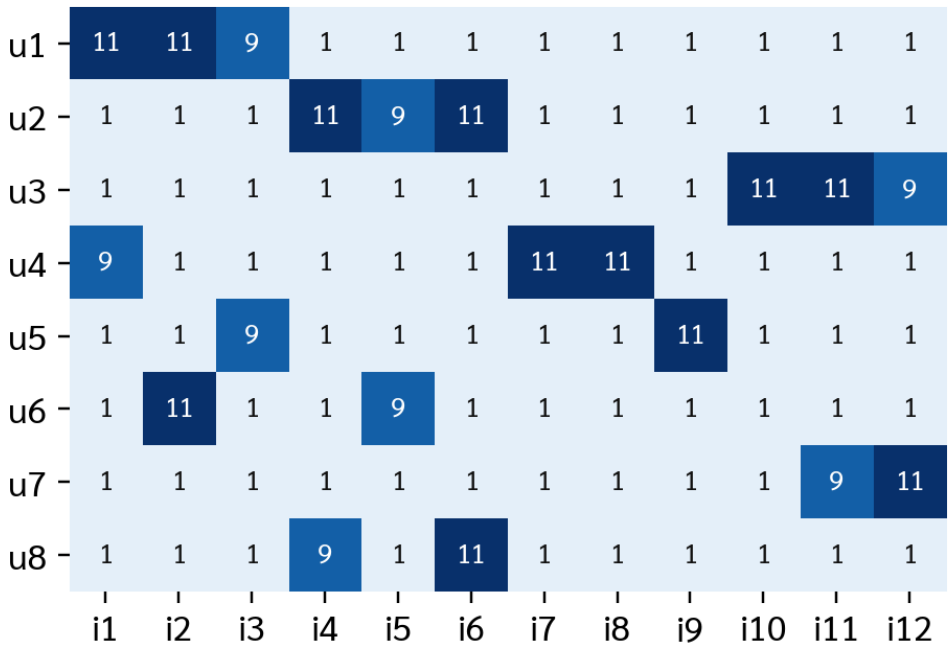
โจทย์ จงหาความเชื่อมั่นของ u1 ทุกเรื่องที่ $\alpha = 2$

สมการที่ใช้ สมการ (5.22) ส่วน c_{ui}

ข้อมูลที่ใช้ ตารางที่ 5.5 และ 5.8

- i1, i2 ($r = 5, x = 1$): $c = 1 + 2 \times 5 \times 1 = 11$
- i3 ($r = 4, x = 1$): $c = 1 + 2 \times 4 = 9$
- i4, i6 (ให้คะแนน 1 จึงไม่ชอบ $x = 0$): $c = 1 + 2 \times 1 \times 0 = 1$
- ทุกเรื่องที่ยังไม่ดู ($x = 0$): $c = 1$

ตีความ WRMF เชื่อว่า u1 ชอบ i1 และ i2 มากกว่าเชื่อว่า u1 ไม่ชอบ i9 ถึง 11 เท่า สังเกตว่า i4 และ i6 ซึ่ง u1 ไม่ชอบจริงได้ความเชื่อมั่นเท่ากับเรื่องที่ไม่เคยดู เพราะกติกาของสมการ (5.3) ทิ้งข้อมูลว่า "เคยดูแต่ไม่ชอบ" ไป



ภาพที่ 5.16 เมทริกซ์ความเชื่อมั่นของ WRMF

5.9.2 ที่มาของสูตร ALS แบบถ่วงน้ำหนัก

การหาคำตอบของ $\mathbf{p}_u$ ทำแบบเดียวกับหัวข้อ 5.8.1 ทุกประการ เพียงคูณแต่ละพจน์ด้วย c_{ui} ผลคือการถดถอยแบบสันเขาที่ถ่วงน้ำหนัก ถ้าเขียนความเชื่อมั่นของผู้ใช้ u เป็นเมทริกซ์ทแยง $\mathbf{C}^u = \text{diag}\left(c_{u1}, \ldots, c_{u|\mathcal{I}|}\right)$ จะได้

$$\mathbf{p}_u = (\mathbf{Q}^\top \mathbf{C}^u \mathbf{Q} + \lambda \mathbf{I})^{-1} \mathbf{Q}^\top \mathbf{C}^u \mathbf{x}_u \tag{5.23}$$

โดยที่

- $\mathbf{Q}$ คือเมทริกซ์ของเวกเตอร์รายการทุกเรื่อง (ขนาด $|\mathcal{I}| \times k$)
- $\mathbf{C}^u = \text{diag}\left(c_{u1}, \ldots, c_{u|\mathcal{I}|}\right)$ คือเมทริกซ์ทแยงของความเชื่อมั่นของผู้ใช้ u
- $\mathbf{x}_u$ คือแถวความชอบของผู้ใช้ u ทั้ง $|\mathcal{I}|$ ช่อง

ปัญหาคือ $\mathbf{Q}^\top \mathbf{C}^u \mathbf{Q}$ ต้องคิดทุกรายการ ถ้ามีหนึ่งล้านรายการก็ต้องรวมหนึ่งล้านพจน์ต่อผู้ใช้หนึ่งคน Hu et al. (2008) แก้ด้วยการแยกเมทริกซ์ออกเป็นสองส่วน

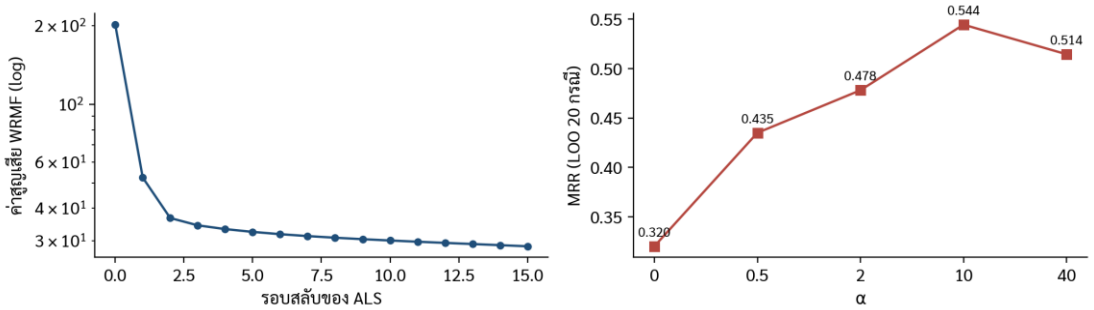
$$\mathbf{Q}^\top \mathbf{C}^u \mathbf{Q} = \mathbf{Q}^\top \mathbf{Q} + \mathbf{Q}^\top (\mathbf{C}^u - \mathbf{I}) \mathbf{Q} = \mathbf{Q}^\top \mathbf{Q} + \sum_{i: x_{ui}=1} \left(c_{ui} - 1\right) \mathbf{q}_i \mathbf{q}_i^\top \tag{5.24}$$

โดยที่

- $\mathbf{Q}^\top \mathbf{Q}$ คือส่วนที่เหมือนกันสำหรับผู้ใช้ทุกคน คำนวณครั้งเดียวต่อรอบ
- $\mathbf{q}_i \mathbf{q}_i^\top$ คือผลคูณภายนอก ได้เมทริกซ์ $k \times k$ ของรายการ i หนึ่งเรื่อง
- ผลรวมด้านขวาคิดเฉพาะรายการที่ $x_{ui} = 1$ เพราะช่องอื่นมี $c_{ui} - 1 = 0$
- **ส่วนแรก** $\mathbf{Q}^\top \mathbf{Q}$ เหมือนกันสำหรับผู้ใช้ทุกคน คำนวณครั้งเดียวต่อรอบ
- **ส่วนที่สอง** คิดเฉพาะรายการที่ผู้ใช้มีการกระทำ ($c_{ui} - 1 \neq 0$) ซึ่งมีไม่กี่เรื่อง
- **ด้านขวา** $\mathbf{Q}^\top \mathbf{C}^u \mathbf{x}_u = \sum_{i: x_{ui}=1} c_{ui}\, \mathbf{q}_i$ เพราะช่องที่ $x_{ui} = 0$ ไม่มีส่วนร่วม

เทคนิคนี้ทำให้การแก้ $\mathbf{p}_{u1}$ ใช้เพียงสามรายการที่ u1 ชอบ (i1, i2, i3) แทนที่จะต้องใช้ครบ 12 รายการ และเป็นกุญแจที่ทำให้ WRMF ใช้ได้กับคลังหลายล้านรายการ โปรแกรม 5.10 ตรวจว่าสูตรแยกส่วนให้คำตอบเดียวกับสูตรเต็ม

เมื่อฝึกครบแล้ว คะแนนของ u1 ต่อเจ็ดเรื่องคือ i9 0.8217, i7 0.7633, i8 0.7633, i5 0.5947, i12 0.0917, i11 0.0908, i10 0.0766 Top-3 คือ i9, i7, i8 เหตุผลคือ i9 อยู่ใกล้ i3 (ผ่าน u5) และ i7, i8 อยู่ใกล้ i1 (ผ่าน u4) ในปริภูมิแฝง ส่วน i10–i12 อยู่อีกฝั่ง เพราะผู้ที่ชอบสามเรื่องนี้ (u3, u7) ไม่ได้ชอบแอนิเมชันเลย



ภาพที่ 5.17 การลู่เข้าของ WRMF และผลของ alpha

ภาพที่ 5.17 ด้านซ้ายแสดงว่าค่าสูญเสียลดจาก 202.0 เหลือ 36.7 ภายในสองรอบแรก แล้วลดช้า ๆ หลังจากนั้น ด้านขวาแสดงผลของ α ต่อการจัดอันดับ

- **α = 0** ทุกช่องมีน้ำหนักเท่ากัน เท่ากับประมาณเมทริกซ์ 0/1 ตรง ๆ ได้ MRR เพียง 0.3202
- **α = 0.5 ถึง 10** MRR เพิ่มขึ้นต่อเนื่องจาก 0.4350 เป็น 0.5444
  เพราะแบบจำลองสนใจช่องที่ชอบมากขึ้น
- **α = 40** MRR ลดลงเล็กน้อยเป็น 0.5144 เพราะช่องศูนย์แทบไม่มีความหมาย
  แบบจำลองจึงให้คะแนนสูงกับเกือบทุกเรื่อง

ที่มาของไฮเปอร์พารามิเตอร์ ค่า α ของ WRMF

- **α มาจากไหน** Hu et al. (2008) ใช้ α = 40 กับข้อมูลการดูโทรทัศน์ที่ r_{ui} คือจำนวนครั้งที่ดู และรายงานว่าผลไม่ไวต่อ α ในช่วงกว้าง
- **ความหมาย** α บอกว่า "การกระทำหนึ่งหน่วยเชื่อถือได้กว่าการไม่มีการกระทำกี่เท่า"
  ค่าที่เหมาะจึงขึ้นกับมาตราของ r_{ui} ถ้า r_{ui} เป็นคะแนน 4–5 ค่า α = 2
  ก็ให้ความเชื่อมั่นต่างจากช่องศูนย์ 9–11 เท่าแล้ว
- **บทนี้ใช้ α = 2** เพื่อให้ตัวเลขในตัวอย่างอ่านง่าย ภาพที่ 5.17 บอกว่าค่าราว 10 ดีกว่าบนข้อมูลชุดนี้
- **รูปแบบลอการิทึม** บางงานใช้ $c_{ui} = 1 + \alpha \log\left(1 + r_{ui}/\epsilon\right)$
  เพื่อไม่ให้รายการที่ถูกดูซ้ำหลายร้อยครั้งครอบงำ (Hu et al., 2008) และงานภายหลังแสดงว่า iALS ที่ปรับค่าดีแล้วยังแข่งขันกับแบบจำลองใหม่ได้ (Rendle et al., 2022)

โปรแกรม 5.10 WRMF ด้วย ALS และสูตรแยกส่วน

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
users = list(R)
items = ['i%d' % k for k in range(1, 13)]
alpha, lam, k = 2.0, 0.1, 2


# ความชอบ x และความเชื่อมั่น c = 1 + alpha * r * x
X = np.zeros((8, 12))
Cm = np.ones((8, 12))
for a, u in enumerate(users):
    for b, i in enumerate(items):
        r = R[u].get(i, 0)
        if r >= 4:
            X[a, b] = 1.0
            Cm[a, b] = 1 + alpha * r


rng = np.random.default_rng(3)
P = rng.normal(0, 0.1, (8, k))
Q = rng.normal(0, 0.1, (12, k))
I_k = np.eye(k)
for it in range(15):
    # ตรึง Q แล้วแก้ p_u ของผู้ใช้ทุกคน
```

```
    for a in range(8):
        Cu = np.diag(Cm[a])
        P[a] = np.linalg.solve(Q.T @ Cu @ Q + lam * I_k,
                               Q.T @ Cu @ X[a])
    # ตรึง P แล้วแก้ q_i ของรายการทุกเรื่อง
    for b in range(12):
        Ci = np.diag(Cm[:, b])
        Q[b] = np.linalg.solve(P.T @ Ci @ P + lam * I_k,
                               P.T @ Ci @ X[:, b])


# สูตรแยกส่วนของ u1: ใช้เฉพาะเรื่องที่ชอบ
liked = np.where(X[0] > 0)[0]
A = Q.T @ Q + lam * I_k
rhs = np.zeros(k)
for b in liked:
    A += (Cm[0, b] - 1) * np.outer(Q[b], Q[b])
    rhs += Cm[0, b] * Q[b]
print('p_u1 หลังฝึก', P[0].round(4))
print('สูตรแยกส่วน', np.linalg.solve(A, rhs).round(4))
s = P[0] @ Q.T
print({items[b]: round(float(s[b]), 4)
       for b in range(12) if items[b] not in R['u1']})
```

5.10 BPR: เรียนการจัดอันดับโดยตรง

ระบบแนะนำส่วนใหญ่ต้อง **เรียงลำดับ** ไม่ใช่ทายคะแนน หัวข้อนี้และหัวข้อ 5.11 นำเสนอแบบจำลองสองแบบที่ออกแบบมาเพื่อการจัดอันดับโดยตรง BPR เรียน "ลำดับ" ระหว่างคู่รายการ ส่วน EASE เป็นแบบจำลองเชิงเส้นที่มีคำตอบรูปปิด ดังสรุปในภาพที่ 5.36 ภาพนี้สรุปการเปรียบเทียบรายการบวกกับรายการลบ การปรับน้ำหนัก และการเรียนรู้อันดับโดยตรง



ภาพที่ 5.36 BPR และการเรียนรู้จากคู่

5.10.1 แนวคิดของการเรียนจากคู่

BPR ตั้งสมมติฐานง่าย ๆ ว่า **ผู้ใช้ชอบรายการที่เคยมีปฏิสัมพันธ์ (i) มากกว่ารายการที่ไม่เคยมี (j)** ข้อมูลฝึกจึงเป็นสามสิ่ง (u, i, j) แทนที่จะเป็นคะแนน (Rendle et al., 2009)

ตัวอย่างการคำนวณ 5.26 นับคู่ฝึกของ BPR

โจทย์ u1 มีคู่ฝึก (i, j) กี่คู่ และทั้งชุดข้อมูลมีกี่คู่

สมการที่ใช้ จำนวนคู่ของผู้ใช้ u = (จำนวนเรื่องที่ชอบ) × (จำนวนเรื่องที่ไม่ได้ชอบ)

ข้อมูลที่ใช้ ตารางที่ 5.8

- u1 ชอบ 3 เรื่อง (i1, i2, i3) และไม่ได้ชอบ 9 เรื่อง จึงมี $3 \times 9 = 27$ คู่ เช่น (i1 > i5), (i2 > i9), (i3 > i4)
- รวมผู้ใช้ทั้งแปดคน ได้ 188 คู่ (ผลรวมของ $n_u(12 - n_u)$ เมื่อ n_u คือจำนวนเรื่องที่ผู้ใช้ u ชอบ)

ตีความ คู่ (i2 > i9) บอกว่า u1 ชอบ Inside Out 2 มากกว่า Conclave ซึ่งอาจไม่จริง เพราะ u1 แค่ยังไม่เคยเห็น Conclave BPR ยอมรับความผิดพลาดนี้ เพราะโดยเฉลี่ยแล้วรายการที่ผู้ใช้เลือกน่าจะดีกว่ารายการที่ไม่เลือก หัวข้อ 5.12 จะกลับมาที่ประเด็นนี้

อธิบายแบบเข้าใจง่าย BPR คือการประกวดทีละคู่

แทนที่จะให้กรรมการให้คะแนนผู้เข้าประกวดทุกคนเป็นตัวเลข BPR ให้กรรมการดูทีละคู่แล้วตอบว่า "คนไหนดีกว่า" เมื่อดูครบหลายคู่ ลำดับรวมก็ออกมาเอง วิธีนี้ไม่ต้องสนใจว่าคะแนนจริงเป็นเท่าใด สนใจเพียงว่าใครควรอยู่สูงกว่าใคร ซึ่งตรงกับสิ่งที่ระบบแนะนำต้องการ

5.10.2 BPR: ค่าสูญเสียและที่มาของกฎการปรับ

$$\max_{\Theta} \sum_{(u,i,j)} \ln \sigma\left(\hat{x}_{uij}\right) - \lambda \parallel \Theta \parallel^2, \qquad \hat{x}_{uij} = \mathbf{p}_u^{\top}\left(\mathbf{q}_i - \mathbf{q}_j\right), \qquad \sigma(x) = \frac{1}{1+e^{-x}} \quad (5.25)$$

โดยที่

- (u, i, j) คือสามสิ่งที่ใช้ฝึก ได้แก่ ผู้ใช้ u รายการบวก i (เคยมีปฏิสัมพันธ์) และรายการลบ j (ไม่เคยมี)
- $\hat{x}_{uij}$ คือผลต่างของคะแนนระหว่างรายการบวกกับรายการลบ
- $\sigma(x)$ คือฟังก์ชันซิกมอยด์ แปลงผลต่างเป็นความน่าจะเป็นระหว่าง 0 ถึง 1 และ ln คือลอการิทึมฐาน e
- Θ คือพารามิเตอร์ทั้งหมด ($\mathbf{p}_u$, $\mathbf{q}_i$, $\mathbf{q}_j$) และ λ คือค่าเรกูลาไรเซชัน

$\sigma(\hat{x}_{uij})$ คือความน่าจะเป็นที่แบบจำลองจัด i ไว้สูงกว่า j BPR ต้องการให้ค่านี้ใกล้ 1 สำหรับทุกคู่ การหาค่ามากสุดของ $\ln\sigma$ จึงเทียบเท่ากับการทำให้ AUC (บทที่ 2 หัวข้อ 2.5) สูงที่สุด เพราะ AUC คือสัดส่วนของคู่ที่จัดลำดับถูก และ σ คือเวอร์ชัน "นุ่มนวล" ของการนับว่าถูกหรือผิด

ที่มาของกฎการปรับหาได้สามขั้น

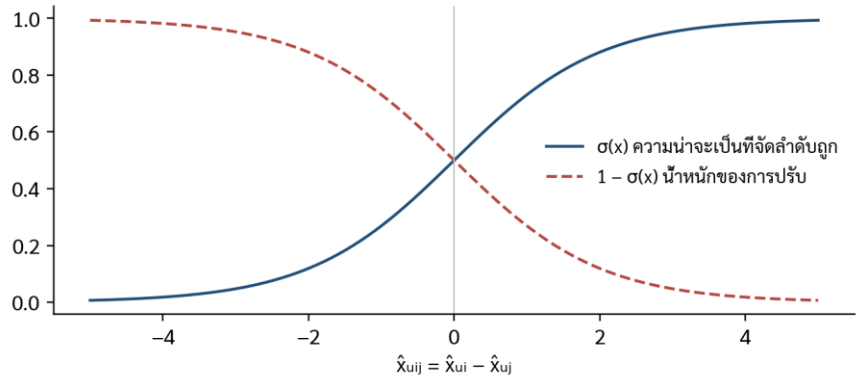
- **ขั้นที่ 1 อนุพันธ์ของ** $\ln\sigma$ จาก $\sigma'(x) = \sigma(x)(1-\sigma(x))$ จะได้ $\frac{d}{dx}\ln\sigma(x) = \frac{\sigma(x)(1-\sigma(x))}{\sigma(x)} = 1 - \sigma(x)$
- **ขั้นที่ 2 อนุพันธ์ของ** $\hat{x}_{uij}$ เทียบ $\mathbf{p}_u$ คือ $\mathbf{q}_i - \mathbf{q}_j$ เทียบ $\mathbf{q}_i$ คือ $\mathbf{p}_u$ และเทียบ $\mathbf{q}_j$ คือ $-\mathbf{p}_u$
- **ขั้นที่ 3 เดินตามเกรเดียนต์** (เพราะต้องการค่ามากสุด) ด้วยอัตรา η ให้ $s = 1 - \sigma(\hat{x}_{uij})$ แล้วคูณตามกฎลูกโซ่ได้สมการ (5.26)

$$\begin{aligned} s &= 1 - \sigma\left(\hat{x}_{uij}\right) \\ \mathbf{p}_u &\leftarrow \mathbf{p}_u + \eta\left(s\left(\mathbf{q}_i - \mathbf{q}_j\right) - \lambda\mathbf{p}_u\right) \\ \mathbf{q}_i &\leftarrow \mathbf{q}_i + \eta\left(s\,\mathbf{p}_u - \lambda\mathbf{q}_i\right) \\ \mathbf{q}_j &\leftarrow \mathbf{q}_j + \eta\left(-s\,\mathbf{p}_u - \lambda\mathbf{q}_j\right) \end{aligned} \quad (5.26)$$

โดยที่

- s คือน้ำหนักของการปรับ ใกล้ 1 เมื่อจัดลำดับผิด และใกล้ 0 เมื่อจัดลำดับถูกอย่างมั่นใจ
- η คืออัตราการเรียนรู้ และ λ คือค่าเรกูลาไรเซชัน
- เวกเตอร์ของรายการบวก $\mathbf{q}_i$ ถูกดึงเข้าหา $\mathbf{p}_u$ ส่วนรายการลบ $\mathbf{q}_j$ ถูกผลักออก

s คือ **น้ำหนักของการปรับ** ถ้าแบบจำลองจัดลำดับถูกอยู่แล้วอย่างมั่นใจ ($\hat{x}_{uij}$ เป็นบวกมาก) s ใกล้ศูนย์และแทบไม่ต้องปรับ ถ้าจัดผิดอย่างมั่นใจ s ใกล้ 1 และปรับมาก (ภาพที่ 5.18)



ภาพที่ 5.18 ฟังก์ชันซิกมอยด์ของ BPR

ตัวอย่างการคำนวณ 5.27 BPR หนึ่งก้าวบนสามสิ่ง (u1, i1, i5)

โจทย์ u1 ชอบ i1 แต่ไม่มีปฏิสัมพันธ์กับ i5 ให้ $\mathbf{p}_{u1} = (0.1,0.2)$, $\mathbf{q}_{i1} = (0.3,0.1)$, $\mathbf{q}_{i5} = (0.2,0.3)$, $\eta = 0.1$, $\lambda = 0.01$ จงปรับหนึ่งก้าวและตรวจค่าสูญเสีย

สมการที่ใช้ สมการ (5.25) และ (5.26)

ข้อมูลที่ใช้ ค่าเริ่มต้นเป็นค่าสมมติขนาดเล็ก

ขั้นที่ 1 ผลต่างของคะแนน

$$\hat{x}_{uij} = (0.1)(0.3 - 0.2) + (0.2)(0.1 - 0.3) = 0.01 - 0.04 = -0.03$$

ขั้นที่ 2 น้ำหนักของการปรับ

$$s = 1 - \sigma(-0.03) = \frac{1}{1 + e^{-0.03}} = 0.5075$$

ขั้นที่ 3 ปรับพารามิเตอร์

- $\mathbf{p}_{u1} = (0.1,0.2) + 0.1\big(0.5075(0.1, -0.2) - 0.01(0.1,0.2)\big) = (0.1050,0.1897)$
- $\mathbf{q}_{i1} = (0.3,0.1) + 0.1\big(0.5075(0.1,0.2) - 0.01(0.3,0.1)\big) = (0.3048,0.1100)$
- $\mathbf{q}_{i5} = (0.2,0.3) + 0.1\big(-0.5075(0.1,0.2) - 0.01(0.2,0.3)\big) = (0.1947,0.2896)$

ขั้นที่ 4 ตรวจผล

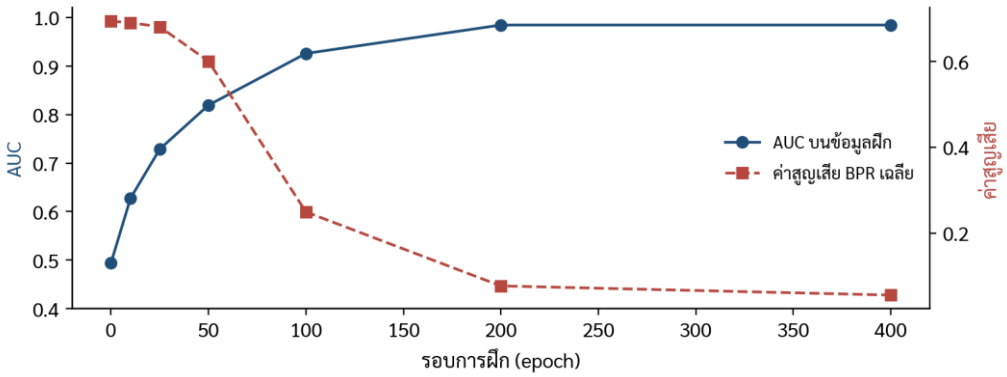
- ผลต่างใหม่ $\hat{x}_{uij} = -0.0225$ เพิ่มขึ้นจาก −0.03
- ค่าสูญเสีย $-\ln\sigma(\hat{x})$ ลดจาก 0.7083 เป็น 0.7045

ตีความ เดิมแบบจำลองจัด i5 ไว้สูงกว่า i1 เล็กน้อย ($\hat{x}_{uij} < 0$) ซึ่งผิด จึงได้น้ำหนักการปรับเกินครึ่ง (0.5075) หลังปรับ i1 ขยับขึ้นและ i5 ขยับลง แม้ผลต่างยังติดลบ แต่การฝึกจริงทำแบบนี้หลายพันครั้งจนลำดับถูกต้อง

5.10.3 การสุ่มตัวอย่าง ความสัมพันธ์กับ AUC และผลบน MovieMini-12

จำนวนคู่ที่เป็นไปได้มีมาก (188 คู่ในชุดเล็กนี้ และหลายพันล้านคู่ในระบบจริง) BPR จึงไม่วนครบทุกคู่ แต่ **สุ่มแบบบูตสแตรป (Bootstrap Sampling)** คือสุ่มคู่ (u, i) ที่เป็นบวก แล้วสุ่ม j จากรายการที่ u ไม่ได้ชอบ บทนี้นับหนึ่งรอบเท่ากับการสุ่ม 20 ครั้ง (จำนวนคู่บวก) และสุ่ม j ใหม่ทุกครั้ง วิธีสุ่ม j มีผลต่อคุณภาพมาก ซึ่งหัวข้อ 5.12 จะอธิบาย

ข้อควรรู้คือ AUC ให้น้ำหนักทุกตำแหน่งในรายการเท่ากัน ขณะที่ผู้ใช้สนใจเฉพาะส่วนบน BPR จึงไม่ได้ปรับ HR@K หรือ nDCG@K โดยตรง งานรุ่นหลังจึงเสนอค่าสูญเสียที่เน้นส่วนบนของรายการ เช่น WARP



ภาพที่ 5.19 AUC และค่าสูญเสียของ BPR

ภาพที่ 5.19 ติดตามการฝึก BPR บน MovieMini-12 (k = 2, η = 0.05, λ = 0.01) ที่รอบ 0 AUC เท่ากับ 0.4947 ซึ่งคือการเดาสุ่ม หลัง 100 รอบ AUC เพิ่มเป็น 0.9255 และหลัง 200 รอบเป็น 0.9840 แปลว่าแบบจำลองจัดลำดับถูกเกือบทุกคู่ของข้อมูลฝึก

เมื่อฝึกครบ 400 รอบ คะแนนของ u1 เรียงเป็น i8 (2.89), i7 (2.46), i5 (0.92), i9 (−0.43), i10 (−3.15), i12 (−4.49), i11 (−4.53) Top-3 ของ BPR จึงเน้นแอ็กชันจาก u4 และ Wicked จาก u6 แต่ Conclave ได้คะแนนติดลบ เพราะในข้อมูลฝึก i9 ถูกใช้เป็นตัวอย่างลบของ u1 หลายครั้ง

ที่มาของไฮเปอร์พารามิเตอร์ η, λ และจำนวนรอบของ BPR

- **ที่มา** Rendle et al. (2009) เลือก η และ λ ด้วยการค้นหาแบบตารางบนชุดตรวจสอบ ค่าทั่วไปคือ η ระหว่าง 0.01–0.1 และ λ ระหว่าง 0.0001–0.01
- **จำนวนรอบ** BPR ปรับครั้งละหนึ่งคู่ จึงต้องใช้รอบมากกว่า MF แบบคะแนนหลายเท่า บทนี้นับหนึ่งรอบเท่ากับจำนวนคู่บวก (20 ครั้ง) และฝึก 400 รอบ ภาพที่ 5.19 แสดงว่า 200 รอบก็เพียงพอ
- **λ ที่เล็กกว่า MF** เพราะ BPR ไม่มีค่าเอนเอียงและสนใจเพียงผลต่าง SHRINKAGE มากเกินจะทำให้ผลต่างทุกคู่เข้าใกล้ศูนย์

โปรแกรม 5.11 BPR หนึ่งก้าวและค่าสูญเสีย

```
import numpy as np

p = np.array([0.1, 0.2])      # p_u1
qi = np.array([0.3, 0.1])     # q_i1 (รายการบวก)
qj = np.array([0.2, 0.3])     # q_i5 (รายการลบ)
eta, lam = 0.1, 0.01

x = p @ (qi - qj)             # ผลต่างของคะแนน
s = 1 / (1 + np.exp(x))       # s = 1 - sigma(x)

p_new = p + eta * (s * (qi - qj) - lam * p)
qi_new = qi + eta * (s * p - lam * qi)
qj_new = qj + eta * (-s * p - lam * qj)
x_new = p_new @ (qi_new - qj_new)
```

```
print('x', round(float(x), 4), 's', round(float(s), 4))
print('p', p_new.round(4), 'qi', qi_new.round(4), 'qj', qj_new.round(4))
loss_old = np.log(1 + np.exp(-x))
loss_new = np.log(1 + np.exp(-x_new))
print('loss', round(float(loss_old), 4), '->', round(float(loss_new), 4))
```

5.11 EASE: แบบจำลองเชิงเส้นรูปปิด

5.11.1 แนวคิดและสูตร ดังสรุปในภาพที่ 5.37

ภาพนี้สรุปการเรียนรู้ความสัมพันธ์ระหว่างรายการแบบรูปปิดและการคำนวณคะแนนแนะนำ



ภาพที่ 5.37 EASE และเมทริกซ์น้ำหนักรายการ

EASE (Embarrassingly Shallow Autoencoder) ทำนายคะแนนของรายการหนึ่งจากผลรวมถ่วงน้ำหนักของรายการอื่นที่ผู้ใช้ชอบ ด้วยเมทริกซ์น้ำหนัก $\mathbf{B}$ ขนาด $|\mathcal{I}| \times |\mathcal{I}|$ (Steck, 2019) ค่า B_{ji} ตอบคำถามว่า "การชอบ j บอกอะไรเกี่ยวกับการชอบ i"

$$\min_{\mathbf{B}} \| \mathbf{X} - \mathbf{XB} \|_F^2 + \lambda \| \mathbf{B} \|_F^2 \quad \text{s.t.} \quad \operatorname{diag}(\mathbf{B}) = \mathbf{0} \tag{5.27}$$

โดยที่

- $\mathbf{X}$ คือเมทริกซ์การชอบขนาด $|\mathcal{U}| \times |\mathcal{I}|$ และ $\mathbf{B}$ คือเมทริกซ์น้ำหนักรายการ–รายการขนาด $|\mathcal{I}| \times |\mathcal{I}|$
- $\|\cdot\|_F^2$ คือผลรวมกำลังสองของทุกช่อง และ λ คือค่าเรกูลาไรเซชัน
- $\operatorname{diag}(\mathbf{B}) = \mathbf{0}$ คือเงื่อนไขว่าแนวทแยงต้องเป็นศูนย์ (s.t. ย่อจาก subject to หรือ "ภายใต้เงื่อนไข")

เงื่อนไข $\mathrm{diag}(\mathbf{B}) = \mathbf{0}$ สำคัญมาก ถ้าไม่มีเงื่อนไขนี้ คำตอบที่ดีที่สุดคือ $\mathbf{B} = \mathbf{I}$ ซึ่งแปลว่า "ทำนายว่าผู้ใช้ชอบสิ่งที่ชอบอยู่แล้ว" และไม่แนะนำอะไรใหม่เลย ถ้าเพิ่ม λ แต่ยังไม่บังคับแนวทแยง คำตอบคือ $\hat{\mathbf{B}} = \mathbf{I} - \lambda \mathbf{G}^{-1}$ ซึ่งแนวทแยงยังไม่เป็นศูนย์ (ดูขั้นที่ 3 ของตัวอย่างการคำนวณ 5.28)

Steck (2019) แสดงว่าปัญหานี้มีคำตอบรูปปิดซึ่งคำนวณได้สี่ขั้น โดยไม่ต้องสุ่มค่าเริ่มต้นและไม่ต้องวนฝึกหลายรอบ

- **ขั้นที่ 1** สร้างเมทริกซ์การเกิดร่วม $\mathbf{G} = \mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ ช่อง G_{ij} คือจำนวนผู้ใช้ที่ชอบทั้ง i และ j ส่วนแนวทแยงคือจำนวนผู้ชอบรายการนั้นบวก λ
- **ขั้นที่ 2** หาผกผัน $\mathbf{P} = \mathbf{G}^{-1}$ ซึ่ง "หักผลซ้อน" ของรายการที่ชอบร่วมกันหลายเรื่อง
- **ขั้นที่ 3** ตั้ง $B_{ij} = -P_{ij}/P_{jj}$ สำหรับ $i \neq j$ และ $B_{jj} = 0$ การหารด้วย P_{jj} คือขั้นที่บังคับให้แนวทแยงเป็นศูนย์
- **ขั้นที่ 4** คะแนนของผู้ใช้คือ $\mathbf{s}_u = \mathbf{x}_u \mathbf{B}$ คือผลรวมแถวของ $\mathbf{B}$ ตามเรื่องที่ผู้ใช้ชอบ

$$\mathbf{B} = \mathbf{I} - \mathbf{P}\,\mathrm{diag}\big(\mathbf{1} \oslash \mathrm{diag}(\mathbf{P})\big), \qquad \mathbf{P} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1}, \qquad \mathbf{s}_u = \mathbf{x}_u \mathbf{B} \tag{5.28}$$

โดยที่

- $\mathbf{P}$ คือผกผันของเมทริกซ์ $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ และ P_{jj} คือค่าบนแนวทแยงของ $\mathbf{P}$
- $\mathbf{1} \oslash \mathrm{diag}(\mathbf{P})$ คือการหาร 1 ด้วยค่าแนวทแยงทีละตัว ผลคือ $B_{ij} = -P_{ij}/P_{jj}$ และ $B_{jj} = 0$
- $\mathbf{x}_u$ คือแถวการชอบของผู้ใช้ u และ $\mathbf{s}_u$ คือคะแนนของทุกรายการสำหรับผู้ใช้คนนั้น

อธิบายแบบเข้าใจง่าย EASE คือ "ผู้ที่ชอบเรื่องนี้มักชอบเรื่องนั้น" แบบหักซ้ำ

ถ้านับตรง ๆ ว่าคนที่ชอบ A ชอบ B ด้วยกี่คน รายการที่ถูกชอบคู่กับหลายเรื่องพร้อมกันจะถูกนับซ้ำ EASE แก้ด้วยการถามว่า "เมื่อรู้แล้วว่าผู้ใช้ชอบเรื่องอื่นทั้งหมด การชอบ A ยังบอกอะไรเพิ่มเกี่ยวกับ B" คำตอบคือค่า B_{AB} ซึ่งอาจเป็นบวก ลบ หรือศูนย์

5.11.2 คำนวณ EASE ด้วยมือ

ตัวอย่างการคำนวณ 5.28 EASE ด้วยมือบนภาพยนตร์สองเรื่อง (i3 และ i9)

โจทย์ ใช้เฉพาะคอลัมน์ i3 The Wild Robot และ i9 Conclave ของเมทริกซ์การชอบ และ $\lambda = 1$ จงหา $\mathbf{B}$ แล้วหาคะแนนของ i9 สำหรับ u1

สมการที่ใช้ สมการ (5.28) และสูตรผกผัน 2×2

ข้อมูลที่ใช้ ตารางที่ 5.8: ผู้ชอบ i3 คือ u1, u5 ผู้ชอบ i9 คือ u5

ขั้นที่ 1 เมทริกซ์การเกิดร่วม G

- $G_{33} = 2 + 1 = 3$ (สองคนชอบ i3 บวก λ), $G_{99} = 1 + 1 = 2$, $G_{39} = G_{93} = 1$ (u5 ชอบทั้งคู่)
- $\mathbf{G} = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$

ขั้นที่ 2 ผกผัน

- ดีเทอร์มิแนนต์ $= 3 \times 2 - 1 \times 1 = 5$
- $\mathbf{P} = \frac{1}{5}\begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} 0.4 & -0.2 \\ -0.2 & 0.6 \end{bmatrix}$

ขั้นที่ 3 คำตอบเมื่อไม่มีเงื่อนไข (เพื่อเปรียบเทียบ)

- $\hat{\mathbf{B}} = \mathbf{I} - \mathbf{P} = \begin{bmatrix} 0.6 & 0.2 \\ 0.2 & 0.4 \end{bmatrix}$ แนวทแยงไม่เป็นศูนย์ i3 จึงทำนายตัวเองด้วยน้ำหนัก 0.6

ขั้นที่ 4 คำตอบของ EASE

- $B_{39} = -P_{39}/P_{99} = -(-0.2)/0.6 = 0.3333$ (การชอบ i3 บอกเรื่อง i9)
- $B_{93} = -P_{93}/P_{33} = -(-0.2)/0.4 = 0.5000$ (การชอบ i9 บอกเรื่อง i3)
- u1 ชอบ i3 แต่ไม่ได้ชอบ i9: $\mathbf{x}_{u1} = (1,0)$ จึงได้คะแนนของ i9 = $1 \times B_{39} = 0.3333$

ขั้นที่ 5 ตีความ

$\mathbf{B}$ ไม่สมมาตร $B_{93} > B_{39}$ เพราะทุกคนที่ชอบ i9 (มีเพียง u5) ชอบ i3 ด้วย แต่คนที่ชอบ i3 มีเพียงครึ่งหนึ่งที่ชอบ i9 นี่คือสิ่งที่การนับร่วมธรรมดาบอกไม่ได้ และ λ ทำหน้าที่ลดน้ำหนักของความสัมพันธ์ที่มีหลักฐานจากผู้ใช้เพียงคนเดียว

5.11.3 EASE บน MovieMini-12 ทั้งชุด

ตัวอย่างการคำนวณ 5.29 EASE บนเมทริกซ์การชอบทั้งหมดของ MovieMini-12

โจทย์ ใช้ $\mathbf{X}$ ทั้ง 12 คอลัมน์และ $\lambda = 1$ จงอธิบายค่าใน $\mathbf{G}$ หาค่า B ที่สำคัญ และคะแนนของ u1

สมการที่ใช้ สมการ (5.28)

ข้อมูลที่ใช้ ตารางที่ 5.8 ผลคำนวณจากโปรแกรม 5.12

ขั้นที่ 1 ตัวอย่างค่าใน G

- $G_{i1,i1} = 2 + 1 = 3$ (u1 และ u4 ชอบ i1 บวก λ)
- $G_{i1,i7} = 1$ (u4 ชอบทั้งสองเรื่อง)
- $G_{i3,i9} = 1$ (u5 ชอบทั้งสองเรื่อง)
- $G_{i1,i9} = 0$ (ไม่มีใครชอบทั้งสองเรื่อง)

ขั้นที่ 2 ผกผันแล้วคำนวณ B (แสดงบางค่า)

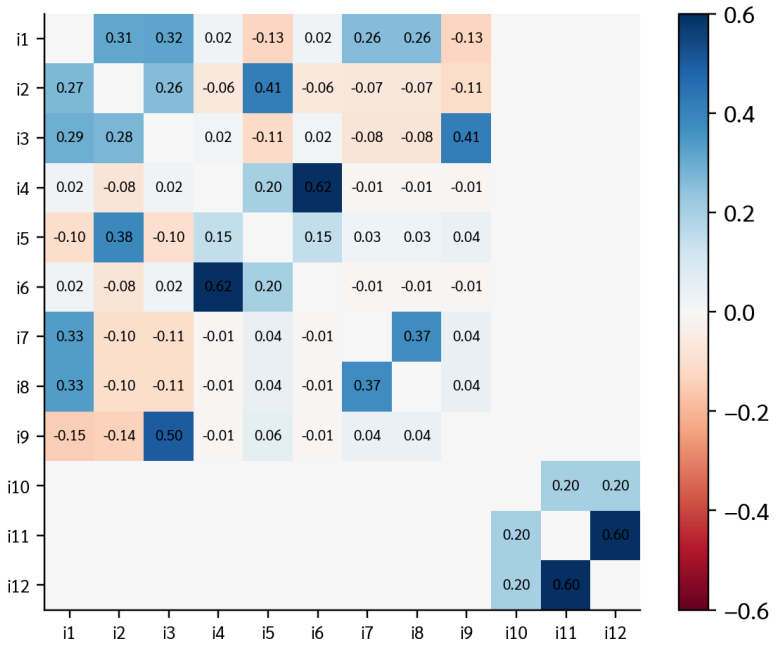
- $B_{i3,i9} = -P_{i3,i9}/P_{i9,i9} = -(-0.2601)/0.6301 = 0.4129$
- $B_{i2,i5} = 0.4143$ และ $B_{i1,i7} = B_{i1,i8} = 0.2581$
- $B_{i3,i5} = -0.1143$ (ค่าลบแม้ไม่มีผู้ใช้ร่วมโดยตรง เพราะผกผันของเมทริกซ์เชื่อมทุกรายการเข้าด้วยกัน)

ขั้นที่ 3 คะแนนของ u1 (ผลรวมแถว i1, i2, i3 ของ B)

- i9: $B_{i1,i9} + B_{i2,i9} + B_{i3,i9} = -0.1307 + -0.1079 + 0.4129 = 0.1743$
- i5: $B_{i1,i5} + B_{i2,i5} + B_{i3,i5} = -0.1286 + 0.4143 + -0.1143 = 0.1714$
- i7: $B_{i1,i7} + B_{i2,i7} + B_{i3,i7} = 0.2581 + -0.0699 + -0.0753 = 0.1129$

ขั้นที่ 4 ตีความ

Top-3 ของ EASE คือ i9, i5, i7 คะแนนของ i9 (0.1743) ต่ำกว่า B_{39} ของตัวอย่างการคำนวณ 5.28 (0.3333) เพราะเมื่อมีรายการครบ ค่าลบจาก i1 และ i2 หักลบบางส่วน EASE จึง "คิดเผื่อ" ว่าการชอบ i1 และ i2 อธิบาย i9 ได้น้อย i10, i11, i12 ได้ศูนย์ เพราะไม่มีความเชื่อมโยงกับเรื่องที่ u1 ชอบเลย



ภาพที่ 5.20 เมทริกซ์น้ำหนักของ EASE

ภาพที่ 5.20 แสดงว่า B แบ่งเป็นกลุ่มชัดเจน กลุ่มแอนิเมชันและแอ็กชัน (i1–i3, i7–i9) กลุ่ม ROMANCE และแฟนตาซี (i4–i6) และกลุ่ม SCI-FI (i10–i12) ซึ่งสะท้อนกลุ่มรสนิยมของผู้ใช้ในข้อมูล

ที่มาของไฮเปอร์พารามิเตอร์ λ ของ EASE

- **หน้าที่** λ ทำให้ $\mathbf{G}$ หาผกผันได้เสมอ
  และเป็นเรกูลาไรเซชันที่ลดน้ำหนักของความสัมพันธ์ที่มีหลักฐานน้อย
- **มาจากไหน** Steck (2019) รายงานค่าที่เหมาะในช่วงหลายร้อยถึงหลายพันบนชุดข้อมูลขนาดใหญ่
  เพราะค่าใน $\mathbf{X}^{\mathsf{T}}\mathbf{X}$ คือจำนวนผู้ใช้ที่ชอบร่วมกัน ซึ่งใหญ่มาก λ ควรมีขนาดเทียบเคียงกับค่าเหล่านั้น
- **ผลบน MovieMini-12** λ = 0.1, 1, 5, 20, 100 ให้ MRR เท่ากันทุกค่า (0.5381)
  เพราะการจัดอันดับในข้อมูลเล็กนี้ไม่เปลี่ยนตาม λ
- **ข้อดี** EASE มีไฮเปอร์พารามิเตอร์เพียงตัวเดียว ไม่ต้องสุ่มค่าเริ่มต้น และไม่ต้องเลือกจำนวนรอบ
  จึงเป็นระบบฐานที่ดีมาก

โปรแกรม 5.12 EASE ด้วยมือบนสองเรื่องและบนข้อมูลทั้งหมด

```
import numpy as np

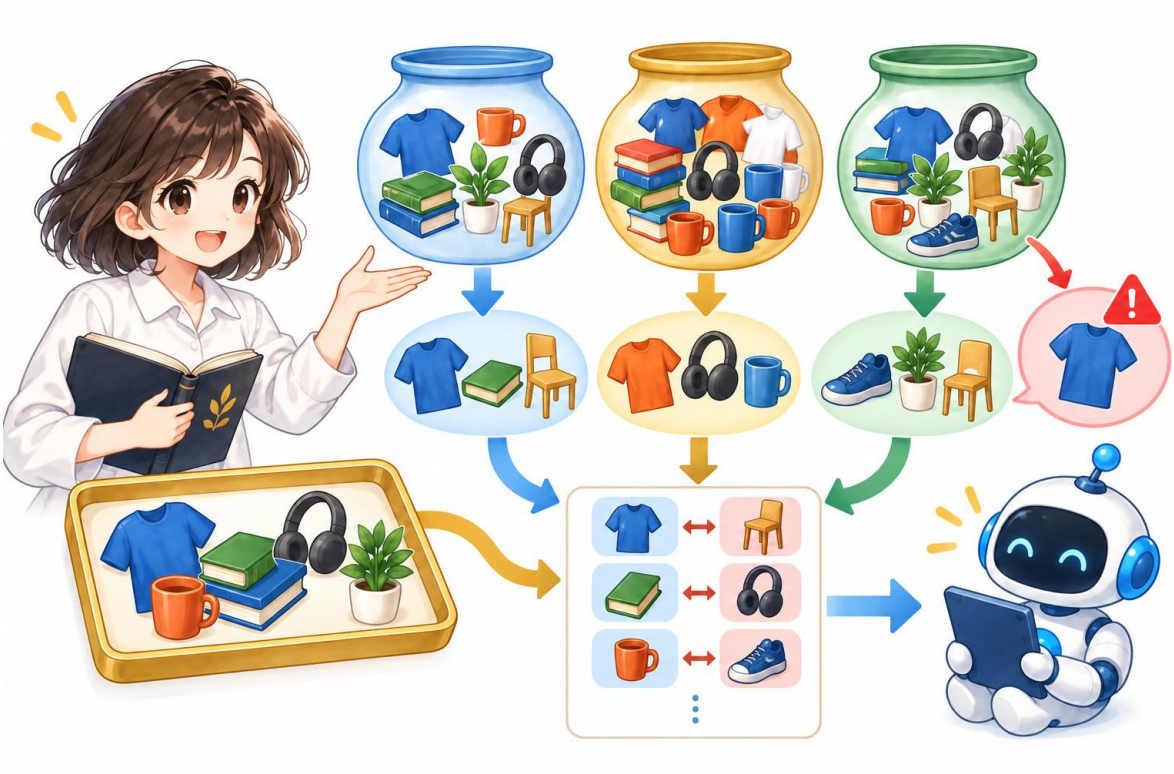

# ส่วนที่ 1: ตัวอย่างสองเรื่อง (i3, i9) ที่ lambda = 1
G = np.array([[3.0, 1.0],
              [1.0, 2.0]])
P = np.linalg.inv(G)
B = -P / np.diag(P)
np.fill_diagonal(B, 0)
print('P', P.round(4).tolist(), 'B', B.round(4).tolist())


# ส่วนที่ 2: EASE บนเมทริกซ์การชอบทั้ง 12 เรื่อง
```

```
items = ['i%d' % k for k in range(1, 13)]
liked = {
    'u1': ['i1', 'i2', 'i3'], 'u2': ['i4', 'i5', 'i6'],
    'u3': ['i10', 'i11', 'i12'], 'u4': ['i1', 'i7', 'i8'],
    'u5': ['i3', 'i9'], 'u6': ['i2', 'i5'],
    'u7': ['i11', 'i12'], 'u8': ['i4', 'i6'],
}
X = np.array([[1.0 if i in L else 0.0 for i in items]
              for L in liked.values()])
P = np.linalg.inv(X.T @ X + 1.0 * np.eye(12))
B = -P / np.diag(P)
np.fill_diagonal(B, 0)
s = X[0] @ B
rated_u1 = {'i1', 'i2', 'i3', 'i4', 'i6'}
print({items[b]: round(float(s[b]), 4)
       for b in range(12) if items[b] not in rated_u1})
```

5.12 การสุ่มตัวอย่างเชิงลบสำหรับผลป้อนกลับโดยปริยาย

BPR (หัวข้อ 5.10) และแบบจำลองเชิงประสาทหลายแบบฝึกด้วยคู่ "บวก–ลบ" รายการบวกคือสิ่งที่ผู้ใช้มีปฏิสัมพันธ์ ส่วนรายการลบต้อง **สุ่ม** จากรายการที่ไม่มีปฏิสัมพันธ์ เพราะระบบไม่รู้ว่าผู้ใช้ไม่ชอบอะไร ในระบบจริงที่มีรายการหลายล้านเรื่อง การใช้รายการลบทุกเรื่องทำไม่ได้ วิธีสุ่มจึงกำหนดว่าแบบจำลองจะเรียนอะไร ดังสรุปในภาพที่ 5.38 ภาพนี้สรุปการสุ่มแบบสม่ำเสมอ แบบอิงความนิยม และแบบยาก พร้อมความเสี่ยงของลบปลอม



ภาพที่ 5.38 การสุ่มตัวอย่างเชิงลบ

5.12.1 สามวิธีหลักของการสุ่ม

$$P_{\text{uni}}(j) = \frac{1}{|\mathcal{I}| - |\mathcal{P}_u|}, \qquad P_{\text{pop}}(j) = \frac{n_j^{\beta}}{\sum_{j'} n_{j'}^{\beta}} \tag{5.29}$$

โดยที่

- $P_{\text{uni}}(j)$ และ $P_{\text{pop}}(j)$ คือความน่าจะเป็นที่รายการ j ถูกสุ่มเป็นตัวอย่างลบแบบสม่ำเสมอและแบบตามความนิยม
- $\mathcal{P}_u$ คือเซตของรายการบวกของผู้ใช้ u ตัวส่วนของ P_{uni} จึงเป็นจำนวนรายการลบที่เป็นไปได้
- n_j คือจำนวนผู้ใช้ที่มีปฏิสัมพันธ์กับรายการ j และ β คือเลขชี้กำลังที่ปรับความเข้มของการเลือกรายการยอดนิยม
- **สุ่มสม่ำเสมอ (Uniform)** ทุกรายการลบมีโอกาสเท่ากัน ($\mathcal{P}_u$ คือเซตของรายการบวกของผู้ใช้ u) ง่ายและไม่ลำเอียง แต่รายการลบส่วนใหญ่ "ง่ายเกินไป" แบบจำลองแยกออกได้ตั้งแต่แรก จึงเรียนได้ช้า
- **สุ่มตามความนิยม (Popularity-based)** รายการยอดนิยมถูกสุ่มบ่อยกว่า (n_j คือจำนวนผู้ใช้ที่มีปฏิสัมพันธ์กับ j) เหตุผลคือ ถ้าผู้ใช้ไม่เคยกดรายการยอดนิยม ทั้งที่น่าจะเคยเห็นแล้ว รายการนั้นน่าจะเป็นลบจริง
- **สุ่มแบบยาก (Hard Negative)** เลือกรายการลบที่แบบจำลองปัจจุบันให้คะแนนสูง เพื่อบังคับให้เรียนเส้นแบ่งที่ละเอียด แต่เสี่ยงเลือก "บวกที่ยังไม่ถูกเห็น" มาเป็นลบ

อธิบายแบบเข้าใจง่าย ตัวอย่างลบคือคู่ซ้อมที่ต้องเลือกให้ดี

นักมวยที่ซ้อมกับคู่ซ้อมอ่อนเกินไปจะไม่เก่งขึ้น ถ้าซ้อมกับคู่ซ้อมเก่งพอดีจะพัฒนาเร็ว แต่ถ้าคู่ซ้อม "จริง ๆ แล้วเป็นเพื่อนร่วมทีม" ที่จับมาสวมนวมผิดฝั่ง การซ้อมจะสอนสิ่งผิด

ตัวอย่างลบก็เช่นกัน ลบที่ง่ายเกินสอนอะไรได้น้อย ลบที่ยากช่วยได้มาก แต่ลบที่แท้จริงเป็นเรื่องที่ผู้ใช้จะชอบ (เพียงยังไม่เห็น) จะสอนแบบจำลองผิดทาง

5.12.2 คำนวณความน่าจะเป็นของการสุ่มสำหรับ u1

ตัวอย่างการคำนวณ 5.30 ความน่าจะเป็นของการสุ่มรายการลบของ u1

โจทย์ u1 มีรายการบวก 3 เรื่อง (i1, i2, i3) และรายการลบที่เป็นไปได้ 9 เรื่อง จงหาความน่าจะเป็นแบบสม่ำเสมอ และแบบตามความนิยมด้วย $\beta = 0.75$

สมการที่ใช้ สมการ (5.29)

ข้อมูลที่ใช้ จำนวนผู้ให้คะแนน n_j จากตารางที่ 5.6

ขั้นที่ 1 แบบสม่ำเสมอ

- $1/9 = 0.1111$ ทุกเรื่อง

ขั้นที่ 2 ยกกำลัง 0.75

- i6, i11 ($n = 4$): $4^{0.75} = 2.8284$
- i4, i5 ($n = 3$): $3^{0.75} = 2.2795$
- i7, i8, i10, i12 ($n = 2$): $2^{0.75} = 1.6818$
- i9 ($n = 1$): $1^{0.75} = 1$
- ผลรวม = $2(2.8284) + 2(2.2795) + 4(1.6818) + 1 = 17.9430$

ขั้นที่ 3 หารด้วยผลรวม

- i6, i11: $2.8284/17.9430 = 0.1576$

- i4, i5: 0.1270
- i7, i8, i10, i12: 0.0937
- i9: $1/17.9430 = 0.0557$

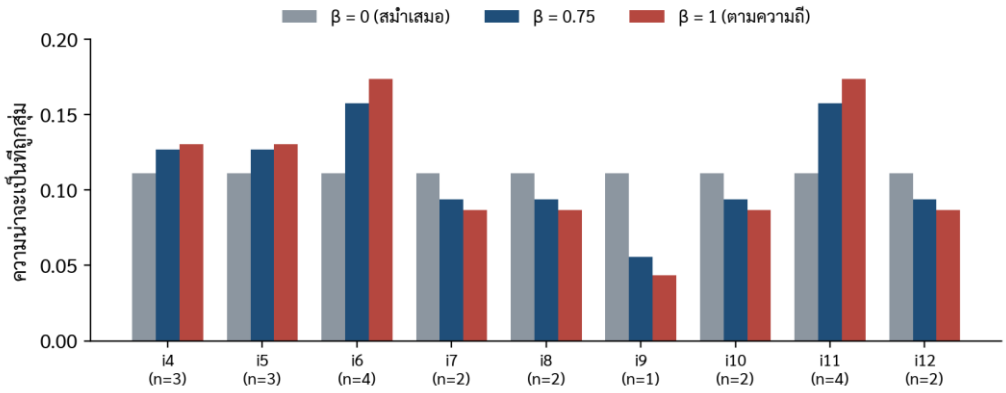
ขั้นที่ 4 ตีความ

การสุ่มตามความนิยมเลือก i6 และ i11 เป็นลบบ่อยกว่า i9 ราว 2.8 เท่า ซึ่งสมเหตุสมผลสำหรับ i6 (u1 ไม่ชอบจริง) และ i9 ซึ่งเป็นเรื่องที่ u1 น่าจะชอบตามหัวข้อ 5.4 จะถูกใช้เป็นตัวอย่างลบน้อยลง ลดความเสี่ยงที่แบบจำลองจะเรียนผิด

ตารางที่ 5.23 ความน่าจะเป็นของการสุ่มรายการลบของ u1 และผลจำลอง

รายการ	n	$\beta = 0$	$\beta = 0.75$	$\beta = 1$	จำลอง ($\beta = 0.75$)
i4	3	0.1111	0.1270	0.1304	0.1217
i5	3	0.1111	0.1270	0.1304	0.1264
i6	4	0.1111	0.1576	0.1739	0.1571
i7	2	0.1111	0.0937	0.0870	0.0937
i8	2	0.1111	0.0937	0.0870	0.0942
i9	1	0.1111	0.0557	0.0435	0.0561
i10	2	0.1111	0.0937	0.0870	0.0965
i11	4	0.1111	0.1576	0.1739	0.1530
i12	2	0.1111	0.0937	0.0870	0.1013

ที่มา: สมการ (5.29) การจำลองสุ่ม 10,000 ครั้งด้วยเมล็ดสุ่ม 2026



ภาพที่ 5.21 ความน่าจะเป็นของการสุ่มรายการลบ

ตารางที่ 5.23 และภาพที่ 5.21 แสดงว่า β ทำหน้าที่เป็น "ปุ่มปรับ" ระหว่างสุ่มสม่ำเสมอ ($\beta = 0$) กับสุ่มตามความถี่ ($\beta = 1$) คอลัมน์สุดท้ายยืนยันว่าการสุ่มจริง 10,000 ครั้งให้สัดส่วนใกล้กับค่าทางทฤษฎีต่างกันไม่เกิน 0.006

5.12.3 ลบแบบยากและปัญหาลบปลอม

ถ้าใช้คะแนนของ WRMF (หัวข้อ 5.9) เลือกลบแบบยากของ u1 รายการที่ได้คะแนนสูงสุดสามอันดับคือ i9 (0.8217), i8 (0.7633), i7 (0.7633) ซึ่ง **ตรงกับ Top-3 ที่ระบบควรแนะนำพอดี** ถ้านำสามเรื่องนี้ไปใช้เป็นตัวอย่างลบ แบบจำลองจะถูกสอนให้ลดคะแนนของเรื่องที่ควรแนะนำ ปรากฏการณ์นี้เรียกว่า **ลบปลอม (False Negative)**

วิธีลดปัญหาที่ใช้กันมีสามแบบ ได้แก่ สุ่มจากรายการอันดับกลาง ๆ แทนอันดับสูงสุด ผสมลบแบบยากกับลบแบบสุ่มในสัดส่วนที่เหมาะสม และใช้ข้อมูลการแสดงผล (Impression) เพื่อเลือกเฉพาะรายการที่ผู้ใช้ **เห็นแล้วแต่ไม่กด**

ที่มาของไฮเปอร์พารามิเตอร์ เลขชี้กำลัง $\beta = 0.75$ และจำนวนตัวอย่างลบต่อตัวอย่างบวก

- **$\beta = 0.75$ มาจากไหน** มาจากงาน word2vec (Mikolov et al., 2013) ที่ทดลองแล้วพบว่าการยกกำลังสามในสี่ให้ผลดีกว่าการสุ่มตามความถี่ตรง ๆ ($\beta = 1$) ซึ่งเลือกรายการยอดนิยมบ่อยเกิน และดีกว่าสุ่มสม่ำเสมอ ($\beta = 0$) ระบบแนะนำนำค่านี้มาใช้ต่อ แต่ควรทดลองค่าในช่วง 0–1 กับข้อมูลของตน
- **อัตราส่วนลบต่อบวก** BPR ใช้ 1 ต่อ 1 ต่อหนึ่งก้าว ส่วนแบบจำลองเชิงประสาท NCF ใช้ 4 ต่อ 1 ซึ่ง He et al. (2017) ทดลองแล้วพบว่าดีกว่า 1 ต่อ 1 อัตราส่วนสูงขึ้นให้สัญญาณมากขึ้นต่อรอบ แต่ใช้เวลาฝึกมากขึ้น

ข้อควรระวัง การสุ่มเชิงลบตอนประเมินผลต่างจากตอนฝึก

หัวข้อนี้พูดถึงการสุ่มเพื่อ **ฝึก** แบบจำลอง การสุ่มรายการลบเพื่อ **ประเมินผล** (เช่น เทียบรายการถูก 1 เรื่องกับรายการลบ 100 เรื่อง) ทำให้ตัวชี้วัดเพี้ยนและอาจสลับผู้ชนะได้ ตามที่บทที่ 2 หัวข้อ 2.9 อธิบายไว้ (Krichene & Rendle, 2020) บทนี้จึงประเมินกับรายการทั้งหมดที่ผู้ใช้ยังไม่ได้ให้คะแนน

โปรแกรม 5.13 ความน่าจะเป็นของการสุ่มเชิงลบและการจำลอง

```
import numpy as np

# จำนวนผู้ให้คะแนนของรายการลบทั้ง 9 เรื่องของ u1
n = {'i4': 3, 'i5': 3, 'i6': 4, 'i7': 2, 'i8': 2,
     'i9': 1, 'i10': 2, 'i11': 4, 'i12': 2}
negs = list(n)
for beta in (0, 0.75, 1):
    w = np.array([n[i] ** beta for i in negs])
    p = w / w.sum()
    print('beta', beta, {i: round(float(x), 4) for i, x in zip(negs, p)})

# จำลองการสุ่ม 10,000 ครั้งที่ beta = 0.75
w = np.array([n[i] ** 0.75 for i in negs])
p = w / w.sum()
draws = np.random.default_rng(2026).choice(negs, size=10000, p=p)
print('จำลอง', {i: round(float(np.mean(draws == i)), 4) for i in negs})
```

5.13 การประเมินผล: ทำนายคะแนนหรือจัดอันดับ

การกรองร่วมประเมินได้สองแบบ ซึ่งตอบคำถามต่างกัน แบบแรกวัดว่า **ทายคะแนนแม่นเพียงใด** (MAE, RMSE) แบบที่สองวัดว่า **เรียงลำดับได้ดีเพียงใด** (HR@K, MRR, nDCG@K) บทที่ 2 อธิบายสูตรของทั้งสองแบบไว้แล้ว หัวข้อนี้ใช้ทั้งสองแบบกับทุกเทคนิคของบท ดังสรุปในภาพที่ 5.39 ภาพนี้แยกการวัดความคลาดเคลื่อนของคะแนนออกจากคุณภาพอันดับ และแสดงปัจจัยที่ทำให้ผลเปลี่ยนแปลง



ภาพที่ 5.39 การประเมินการทำนายและการจัดอันดับ

อธิบายแบบเข้าใจง่าย ทายคะแนนถูก กับ จัดอันดับถูก ไม่เหมือนกัน

ลองนึกถึงการพยากรณ์อากาศ คนหนึ่งทายอุณหภูมิได้ใกล้เคียงทุกวันแต่บอกไม่ได้ว่าวันไหนร้อนที่สุดของสัปดาห์ อีกคนทายตัวเลขคลาดเคลื่อนมากแต่บอกลำดับวันร้อนได้ถูกต้อง ถ้าเราต้องการเลือกวันไปเที่ยวทะเล คนที่สองมีประโยชน์กว่า

ผู้ใช้ไม่เคยเห็นตัวเลข 4.45 หรือ 3.20 ผู้ใช้เห็นเพียงรายการที่ถูกจัดอันดับ การจัดอันดับถูกจึงสำคัญกว่าการทายคะแนนเป๊ะ

5.13.1 ความแม่นยำของการทำนายคะแนน

โปรโตคอลคือ **กันออกทีละคะแนน (Leave-One-Rating-Out)** ทำซ้ำ 32 ครั้ง ดังนี้

1. ซ่อนคะแนนหนึ่งค่าจากทั้งหมด 32 ค่า
2. ฝึกหรือคำนวณทุกอย่างใหม่จากคะแนนที่เหลือ 31 ค่า
3. ทายคะแนนที่ซ่อน แล้วตัดคำทำนายให้อยู่ในช่วง 1–5

4. เฉลี่ยความคลาดเคลื่อนของทั้ง 32 ครั้งเป็น MAE และ RMSE

$$\text{RMSE} = \sqrt{\frac{1}{|T|}\sum_{(u,i)\in T}(r_{ui} - \hat{r}_{ui})^2}, \qquad \text{MAE} = \frac{1}{|T|}\sum_{(u,i)\in T}|r_{ui} - \hat{r}_{ui}| \tag{5.30}$$

โดยที่

- T คือเซตของคะแนนที่ถูกซ่อนไว้ทดสอบ และ $|T|$ คือจำนวนคะแนนในเซตนั้น
- r_{ui} คือคะแนนจริง และ $\hat{r}_{ui}$ คือคะแนนที่ทำนาย
- RMSE ยกกำลังสองความคลาดเคลื่อนก่อนเฉลี่ย จึงลงโทษการทายผิดมากหนักกว่า MAE ซึ่งใช้ค่าสัมบูรณ์

ตัวอย่างการคำนวณ 5.31 RMSE และ MAE จากคะแนนทั้งห้าของ u1

โจทย์ ซ่อนคะแนนของ u1 ทีละค่า แล้วเทียบความคลาดเคลื่อนของค่าเฉลี่ยรวม μ กับ Item-kNN

สมการที่ใช้ สมการ (5.30)

ข้อมูลที่ใช้ ค่าทำนายจากโปรแกรม Lab 5 เมื่อซ่อนคะแนนนั้น (μ คิดจาก 31 คะแนนที่เหลือ)

รายการ	r	μ (ไม่รวมค่าที่ซ่อน)	คลาดเคลื่อน	Item-kNN	คลาดเคลื่อน
i1	5	3.5806	+1.4194	2.7500	+2.2500
i2	5	3.5806	+1.4194	2.7500	+2.2500
i3	4	3.6129	+0.3871	3.0000	+1.0000
i4	1	3.7097	−2.7097	1.0000	+0.0000
i6	1	3.7097	−2.7097	1.0000	+0.0000

ขั้นที่ 1 ค่าเฉลี่ยรวม μ

- ยกกำลังสองของความคลาดเคลื่อน: 2.0146, 2.0146, 0.1498, 7.3424, 7.3424
- RMSE = $\sqrt{(2.0146 + 2.0146 + 0.1498 + 7.3424 + 7.3424)/5} = 1.9424$ และ MAE = 1.7290

ขั้นที่ 2 Item-kNN

- ยกกำลังสอง: 5.0625, 5.0625, 1.0000, 0.0000, 0.0000
- RMSE = $\sqrt{(5.0625 + 5.0625 + 1.0000 + 0.0000 + 0.0000)/5} = 1.4916$ และ MAE = 1.1000

ขั้นที่ 3 ตีความ

Item-kNN ทายคะแนน 1 ของ ROMANCE ทั้งสองเรื่องได้ตรงพอดี เพราะสองเรื่องนี้คล้ายกันมาก (ตัวอย่างการคำนวณ 5.12) แต่เมื่อซ่อนเรื่องที่ชอบ กลับทายต่ำ เพราะไม่มีรายการคล้ายเชิงบวกเหลือค่าเฉลี่ยรวมพลาดในทิศตรงข้าม คือพลาดเรื่องที่ไม่ชอบอย่างมาก RMSE ลงโทษความคลาดเคลื่อนขนาดใหญ่หนักกว่า MAE จึงไวต่อการพลาดคะแนนสุดขั้วแบบนี้มาก

ตารางที่ 5.24 ความคลาดเคลื่อนแบบกันออกทีละคะแนน (32 ครั้ง)

วิธี	MAE	RMSE
ค่าเฉลี่ยรวม μ	1.2581	**1.4541**

วิธี	MAE	RMSE
ค่าเฉลี่ยของผู้ใช้	1.5625	1.8226
ค่าเฉลี่ยของรายการ	1.9194	2.2006
ค่าเอนเอียง (λ = 2)	1.7139	1.9531
User-kNN	1.5234	1.8534
Item-kNN	1.2969	1.6331
kNN-Baseline	1.4759	1.7447
MF (λ = 0.05)	1.9777	2.1544

ที่มา: สมการ (5.30) ยิ่งต่ำยิ่งดี

ผลในตารางที่ 5.24 อาจดูขัดกับสัญชาตญาณ **ค่าเฉลี่ยรวมซึ่งไม่ปรับตามบุคคลเลยได้ RMSE ต่ำที่สุด** (1.4541) สาเหตุมีสามข้อ

- **คะแนนแบ่งขั้ว** (ภาพที่ 5.4) การทายผิดฝั่งครั้งเดียวทำให้ความคลาดเคลื่อนกำลังสองสูงมาก การทายกลาง ๆ ไว้ก่อนจึงปลอดภัยที่สุดสำหรับ RMSE
- **หลักฐานส่วนบุคคลน้อยเกินไป** เมื่อซ่อนคะแนนหนึ่งค่า ผู้ใช้หลายคนเหลือคะแนนเพียง 2–3 ค่า และรายการหลายเรื่องเหลือ 0–1 ค่า
- **วิธีที่ซับซ้อนกว่าต้องการข้อมูลมากกว่า** ข้อสรุปนี้ไม่ใช้กับข้อมูลขนาดใหญ่ ใน Netflix Prize ค่าเอนเอียงชนะค่าเฉลี่ยรวม และวิธีเพื่อนบ้านกับปัจจัยแฝงชนะค่าเอนเอียงอย่างชัดเจน (Koren, 2010)

ในกลุ่มที่ปรับตามบุคคล Item-kNN ทำได้ดีที่สุด (RMSE 1.6331) และ kNN-Baseline (1.7447) ดีกว่า User-kNN สิ่งที่ควรจำคือ **ความซับซ้อนต้องมีข้อมูลรองรับ**

5.13.2 การจัดอันดับบนโปรโตคอลเดียวกับบทที่ 3–4

การจัดอันดับใช้ **กันออกทีละรายการ (Leave-One-Out: LOO)** แบบเดียวกับบทที่ 3 หัวข้อ 3.12 และบทที่ 4 หัวข้อ 4.11 ทำได้สี่ขั้น

1. ซ่อนเรื่องที่ผู้ใช้ชอบ ($r \geq 4$) ทีละเรื่อง ได้ทั้งหมด 20 กรณี
2. คำนวณทุกอย่างใหม่จากข้อมูลที่เหลือ
3. จัดอันดับรายการที่ผู้ใช้ยังไม่ได้ให้คะแนนรวมกับเรื่องที่ซ่อน คะแนนเสมอกันเรียงตามเลขรหัส
4. บันทึกอันดับ r ของเรื่องที่ซ่อน แล้วคำนวณ HR@3, MRR และ nDCG@3

ตัวอย่างการคำนวณ 5.32 คำนวณ HR@3, MRR และ nDCG@3 ของ User-kNN จากอันดับ 20 กรณี

โจทย์ จงคำนวณตัวชี้วัดทั้งสามจากอันดับของเรื่องที่ซ่อน

สมการที่ใช้ HR@K = $\frac{1}{N}\sum \mathbb{1}[r \leq K]$, MRR = $\frac{1}{N}\sum \frac{1}{r}$, nDCG@K = $\frac{1}{N}\sum_{r \leq K} \frac{1}{\log_2(r+1)}$ (บทที่ 2 หัวข้อ 2.4–2.5)

ข้อมูลที่ใช้ อันดับ 20 กรณีของ User-kNN จาก Lab 5 ส่วนที่ 7 (กรณีที่ 3 คือตัวอย่างการคำนวณ 5.10)

ขั้นที่ 1 อันดับของเรื่องที่ซ่อน (เรียงตาม u1 ถึง u8)

- 2, 4, 4, 1, 2, 1, 7, 1, 1, 1, 3, 3, 3, 6, 10, 3, 2, 2, 1, 1

ขั้นที่ 2 HR@3

- อันดับ 1 มี 7 กรณี อันดับ 2 มี 4 กรณี อันดับ 3 มี 4 กรณี รวม 15 กรณี
- HR@3 = $15/20 = 0.7500$

ขั้นที่ 3 MRR

- ผลรวมของ $1/r$ ทั้ง 20 กรณี = 11.2429
- MRR = $11.2429/20 = 0.5621$

ขั้นที่ 4 nDCG@3 (มีเรื่องที่ถูกต้องเรื่องเดียว ค่าในอุดมคติจึงเป็น 1)

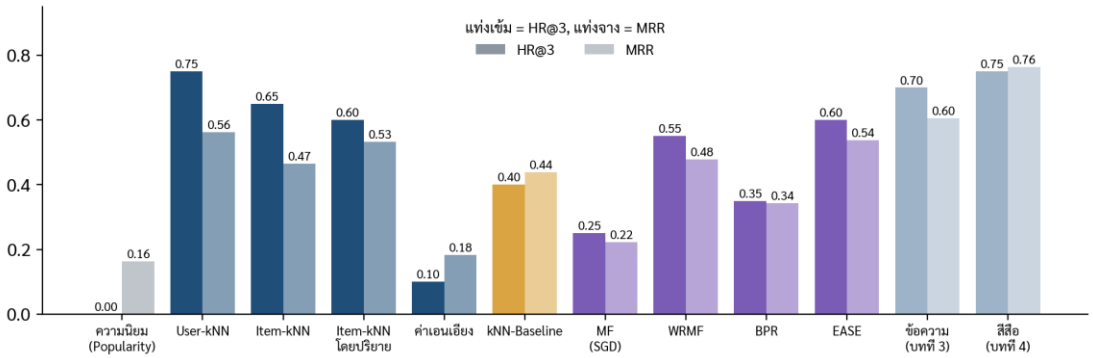
- $\frac{1}{20}\left(7 \times 1 + 4 \times \frac{1}{\log_2 3} + 4 \times \frac{1}{\log_2 4}\right) = \frac{1}{20}\left(7 + 2.5237 + 2.0000\right) = 0.5762$

ตีความ HR@3 บอกว่าร้อยละ 75 ของกรณีที่ซ่อนอยู่ใน Top-3 MRR บอกว่าโดยเฉลี่ยเรื่องที่ซ่อนอยู่ใกล้อันดับ 2 (1/0.5621 ≈ 1.8) ส่วน nDCG@3 ให้คะแนนบางส่วนกับอันดับ 2–3

ตารางที่ 5.25 ผลการจัดอันดับแบบกันออกทีละรายการ 20 กรณี

วิธี	HR@3	MRR	nDCG@3
ความนิยม (Popularity)	0.0000	0.1629	0.0000
User-kNN	0.7500	**0.5621**	0.5762
Item-kNN	0.6500	0.4655	0.4643
Item-kNN โดยปริยาย	0.6000	0.5327	0.4881
ค่าเอนเอียง	0.1000	0.1831	0.0565
kNN-Baseline	0.4000	0.4385	0.3565
MF (SGD)	0.2500	0.2223	0.1315
WRMF (ALS)	0.5500	0.4781	0.4327
BPR-MF	0.3500	0.3430	0.2512
EASE	0.6000	0.5381	0.4881
อ้างอิง: ข้อความ (บทที่ 3)	0.7000	0.6044	0.5946
อ้างอิง: ครบสี่สื่อ (บทที่ 4)	0.7500	0.7635	0.7250

ที่มา: โปรโตคอลเดียวกับบทที่ 3 หัวข้อ 3.12 และบทที่ 4 ตารางที่ 4.17 (Lab 5 ภาคผนวก 5ก)



ภาพที่ 5.22 ผลการจัดอันดับเทียบระบบอิงเนื้อหา

ตารางที่ 5.25 และภาพที่ 5.22 ให้ข้อสังเกตห้าข้อ

- **User-kNN ดีที่สุดในกลุ่มการกรองร่วม** (HR@3 0.7500, MRR 0.5621) ตามด้วย EASE และ Item-kNN แบบโดยปริยาย ซึ่งทั้งสองใช้ "การชอบร่วมกัน" ในทางเดียวกัน
- **ความนิยมล้มเหลวทั้งหมด** (HR@3 = 0) เพราะในชุดนี้ทุกเรื่องมีผู้ชอบเพียง 1–2 คน ผลนี้ต่างจากข้อมูลจริงขนาดใหญ่ ซึ่งความนิยมมักเป็นระบบฐานที่แข็งแรงมาก (บทที่ 1 หัวข้อ 1.12.5)
- **แบบจำลองที่ทายคะแนนไม่ได้จัดอันดับเก่ง** ค่าเอนเอียงและ MF ได้ MRR ต่ำ เพราะค่าเอนเอียงทำให้ทุกคนได้อันดับคล้ายกัน และ MF เรียนรู้เกินบนข้อมูล 32 ค่า (หัวข้อ 5.7.7) kNN-Baseline ดีขึ้นมากเมื่อเทียบกับค่าเอนเอียงอย่างเดียว (MRR 0.4385 เทียบกับ 0.1831)
- **แบบจำลองที่ออกแบบเพื่อข้อมูลโดยปริยายดีกว่าแบบคะแนน** WRMF และ EASE ได้ MRR สูงกว่า MF ชัดเจน
- **การกรองร่วมทุกแบบแพ้ระบบอิงเนื้อหา** บนชุดข้อมูลนี้ ระบบครบสี่สื่อของบทที่ 4 ได้ MRR สูงกว่า User-kNN ถึง 0.2014

ตารางที่ 5.26 อันดับของเรื่องที่ซ่อนในแต่ละกรณี (ห้าวิธีหลัก)

ผู้ใช้	เรื่องที่ซ่อน	Pop	User-kNN	Item-kNN	WRMF	EASE
u1	i1 Toy Story 5	4	2	3	4	3
u1	i2 Inside Out 2	4	4	4	4	4
u1	i3 The Wild Robot	4	4	3	4	4
u2	i4 It Ends with Us	5	1	1	1	1
u2	i5 Wicked	5	2	2	1	3
u2	i6 Bridget Jones: Mad About the Boy	5	1	1	1	1
u3	i10 Dune: Part Two	9	7	7	2	9
u3	i11 Mickey 17	9	1	2	1	1
u3	i12 Project Hail Mary	9	1	2	1	1
u4	i1 Toy Story 5	7	1	2	3	1
u4	i7 Ballerina	9	3	5	6	5
u4	i8 Furiosa: A Mad Max Saga	9	3	5	9	5
u5	i3 The Wild Robot	7	3	3	5	3

ผู้ใช้	เรื่องที่ซ่อน	Pop	User-kNN	Item-kNN	WRMF	EASE
u5	i9 Conclave	9	6	7	6	4
u6	i2 Inside Out 2	6	10	8	6	4
u6	i5 Wicked	6	3	4	6	4
u7	i11 Mickey 17	9	2	2	2	2
u7	i12 Project Hail Mary	9	2	2	2	2
u8	i4 It Ends with Us	5	1	1	2	1
u8	i6 Bridget Jones: Mad About the Boy	5	1	1	2	1

ที่มา: ตารางที่ 5.25 แยกรายกรณี



ภาพที่ 5.23 MRR รายผู้ใช้ของแต่ละวิธี

ตารางที่ 5.26 และภาพที่ 5.23 แยกผลตามผู้ใช้ ข้อสังเกตหลักมีดังนี้

- การกรองร่วมทำได้ดีกับ u2, u3, u7 และ u8 ซึ่งมีผู้ใช้อื่นที่รสนิยมเหมือนกันชัดเจน เช่น u2 กับ u8 ชอบ ROMANCE และ u3 กับ u7 ชอบ SCI-FI
- การกรองร่วมทำได้แย่กับ u1, u5 และ u6 ซึ่งรสนิยมผสมหลายแนวหรือไม่มีผู้ใช้อื่นที่ชอบชุดเดียวกัน
- ระบบอิงเนื้อหาของบทที่ 4 (แถวล่างสุด) ทำได้ดีกับ u1 มาก เพราะแอนิเมชันสามเรื่องที่ u1 ชอบมีเนื้อหาคล้ายกันชัดเจน แต่ทำได้ไม่ดีกับ u5 และ u6 เช่นกัน

เมื่อเทียบ MRR ระดับผู้ใช้ของระบบครบสี่สื่อ (บทที่ 4) กับ User-kNN ด้วย Paired Bootstrap 4,000 รอบ (บทที่ 2 หัวข้อ 2.10.2) ผลต่าง d_u ของ u1–u8 คือ 0.6667, 0.1667, 0.2857, −0.0913, −0.0786, 0.0500, 0.5000, 0.0000 ค่าเฉลี่ย 0.1874 และช่วงความเชื่อมั่น 95% คือ [0.0149, 0.3761] ช่วงไม่คร่อมศูนย์แต่ขอบล่างใกล้ศูนย์มาก และผู้ใช้สองคน (u4, u5) ได้ผลดีกว่าเมื่อใช้การกรองร่วม ข้อสรุปที่เหมาะสมคือ "บนชุดข้อมูลนี้ ระบบอิงเนื้อหาดีกว่าโดยรวม แต่ทั้งสองแนวทางเก่งกับผู้ใช้คนละกลุ่ม" ซึ่งเป็นเหตุผลของระบบผสมในบทที่ 7

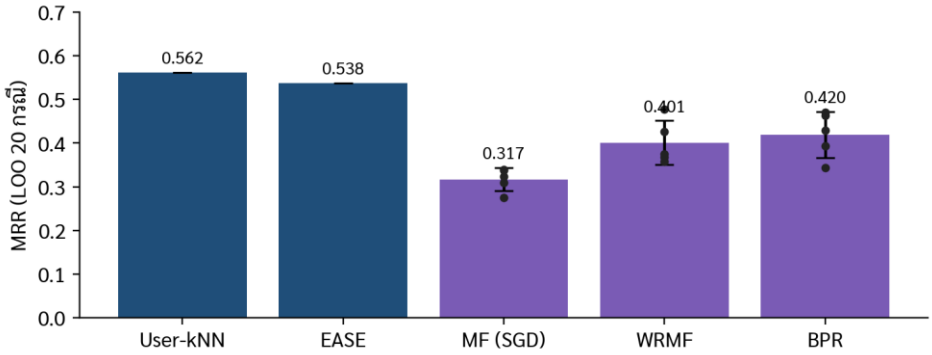
5.13.3 ผลที่ขึ้นกับเมล็ดสุ่ม

วิธีเพื่อนบ้านและ EASE ไม่มีการสุ่ม ผลจึงเหมือนเดิมทุกครั้ง แต่ MF, WRMF และ BPR เริ่มจากค่าสุ่ม (หัวข้อ 5.7.4) ผลจึงเปลี่ยนตามเมล็ดสุ่ม ตารางที่ 5.27 ฝึกแต่ละแบบจำลองด้วยเมล็ดสุ่ม 1–5 บนโปรโตคอลเดียวกัน

ตารางที่ 5.27 MRR ของแบบจำลองที่มีการสุ่มเมื่อเปลี่ยนเมล็ดสุ่ม 1–5

แบบจำลอง	เมล็ด 1–5	ค่าเฉลี่ย	SD	ค่าในตารางที่ 5.25
MF (SGD)	0.340, 0.324, 0.275, 0.338, 0.309	0.3170	0.0268	0.2223
WRMF	0.368, 0.426, 0.478, 0.376, 0.357	0.4010	0.0504	0.4781
BPR	0.470, 0.463, 0.429, 0.393, 0.343	0.4196	0.0526	0.3430

ที่มา: ตารางที่ 5.25 ใช้เมล็ดสุ่ม 7 (MF), 3 (WRMF) และ 5 (BPR)



ภาพที่ 5.24 ผลของเมล็ดสุ่มต่อ MRR

ตารางที่ 5.27 และภาพที่ 5.24 ให้บทเรียนสำคัญสองข้อ

- **ความแปรปรวนจากเมล็ดสุ่มมีขนาดใกล้เคียงกับผลต่างระหว่างแบบจำลอง** เช่น MRR ของ BPR อยู่ระหว่าง 0.343 ถึง 0.470 ขึ้นกับเมล็ดสุ่มเพียงอย่างเดียว การรายงานผลจากเมล็ดเดียวจึงอาจสรุปผิดได้
- **ต้องรายงานค่าเฉลี่ยและ SD จากหลายเมล็ดสุ่ม** อย่างน้อย 3–5 เมล็ด และใช้การทดสอบทางสถิติตามบทที่ 2 หัวข้อ 2.10 ก่อนสรุปว่าแบบจำลองหนึ่งดีกว่าอีกแบบ

5.13.4 ความครอบคลุมและความแปลกใหม่

ตารางที่ 5.28 ความครอบคลุมและความแปลกใหม่ของ Top-3 ผู้ใช้ 8 คน

วิธี	Top-3 ของ u1	Coverage@3	Novelty@3 (บิต)
ความนิยม	i5, i11, i12	0.5833	1.4048
User-kNN	i9, i7, i8	0.7500	1.7177
Item-kNN	i7, i8, i9	0.9167	1.6831
WRMF	i9, i7, i8	0.6667	1.9575
EASE	i9, i5, i7	0.6667	1.6202

ที่มา: Top-3 จากข้อมูลทั้งหมด Novelty ใช้ $p(i)$ = สัดส่วนผู้ใช้ที่ให้คะแนน (บทที่ 2 สมการ (2.10))

ตารางที่ 5.28 แสดงว่าความนิยมแนะนำเรื่องซ้ำ ๆ ให้ทุกคน (Coverage 0.5833) ขณะที่ Item-kNN กระจายคำแนะนำได้กว้างที่สุด (0.9167) WRMF ได้ Novelty สูงที่สุด แปลว่ามักแนะนำเรื่องที่มีผู้ให้คะแนนน้อย ระบบจริงต้องดูตัวชี้วัดเหล่านี้คู่กับความแม่นยำเสมอ

โปรแกรม 5.14 คำนวณ HR@3, MRR และ nDCG@3 จากอันดับ

```
import numpy as np


# อันดับของเรื่องที่ซ่อนของ User-kNN ทั้ง 20 กรณี
ranks = np.array([2, 4, 4, 1, 2, 1, 7, 1, 1, 1,
                  3, 3, 3, 6, 10, 3, 2, 2, 1, 1])
K = 3
hit = ranks <= K
print('HR@3', np.mean(hit))
print('MRR', round(float(np.mean(1 / ranks)), 4))
gain = np.where(hit, 1 / np.log2(ranks + 1), 0)
print('nDCG@3', round(float(np.mean(gain)), 4))


# ความคลาดเคลื่อน e = r - r_hat ของห้าคะแนนของ u1
e_mu = np.array([1.4194, 1.4194, 0.3871, -2.7097, -2.7097])
e_ik = np.array([2.25, 2.25, 1.0, 0.0, 0.0])
for name, e in (('mu', e_mu), ('Item-kNN', e_ik)):
    rmse = np.sqrt(np.mean(e ** 2))
    mae = np.mean(np.abs(e))
    print(name, 'RMSE', round(float(rmse), 4), 'MAE', round(float(mae), 4))
```

5.14 ความท้าทายของการกรองร่วม

ผลในหัวข้อ 5.13 ชี้ให้เห็นจุดอ่อนหลายข้อของการกรองร่วม หัวข้อนี้รวบรวมความท้าทายหลักห้าข้อ พร้อมทดลองให้เห็นผลบน MovieMini-12 ดังสรุปในภาพที่ 5.40 ภาพนี้สรุปการเริ่มต้นเย็น ความเบาบาง อคติความนิยม การขยายขนาด และผู้ใช้แกะเทา



ภาพที่ 5.40 ความท้าทายของการกรองร่วม

5.14.1 การเริ่มต้นเย็น

การเริ่มต้นเย็น (Cold Start) เกิดเมื่อผู้ใช้หรือรายการยังไม่มีพฤติกรรมให้เรียน (Schein et al., 2002) การกรองร่วมพึ่งพฤติกรรมเพียงอย่างเดียว จึงได้รับผลกระทบโดยตรง

ตัวอย่างการคำนวณ 5.33 เมื่อ Conclave ยังไม่มีใครให้คะแนน

โจทย์ ซ่อนคะแนนเดียวของ i9 (u5 ให้ 5) เพื่อจำลองภาพยนตร์ใหม่ แล้วดูว่าแต่ละวิธีทำนาย i9 ให้ u1 อย่างไร

สมการที่ใช้ สมการ (5.8), (5.10), (5.12) และ (5.28)

ข้อมูลที่ใช้ ตารางที่ 5.5 ที่ตัดคะแนน r(u5, i9) ออก

- User-kNN และ Item-kNN: ไม่มีเพื่อนบ้านเลย คำทำนายเท่ากับค่าเฉลี่ยของ u1 คือ 3.2000
- ค่าเอนเอียง: $b_{i9} = 0/(2+0) = 0$ คำทำนายเท่ากับ $\mu + b_{u1}$ = 3.2869
- EASE และ WRMF: คอลัมน์ของ i9 เป็นศูนย์ทั้งหมด คะแนนเป็น 0 เท่ากับเรื่องที่ไม่มีความเชื่อมโยงใดเลย

ตีความ ก่อนซ่อน User-kNN ทำนาย i9 สูงสุด (4.7) หลังซ่อน ทุกวิธีให้ค่ากลาง ๆ หรือศูนย์ ภาพยนตร์ใหม่จะไม่มีวันถูกแนะนำจนกว่าจะมีคนดู ทางแก้คือใช้เนื้อหา (บทที่ 3–4) หรือแบบจำลองที่รวมเนื้อหากับพฤติกรรม เช่น VBPR (บทที่ 4 หัวข้อ 4.10)

ตัวอย่างการคำนวณ 5.34 ผู้ใช้ใหม่ที่ชอบ Toy Story 5 เพียงเรื่องเดียว

โจทย์ ผู้ใช้ใหม่ชอบ i1 เพียงเรื่องเดียว จงแนะนำด้วย EASE โดยไม่ต้องฝึกใหม่

สมการที่ใช้ สมการ (5.28) $\mathbf{s} = \mathbf{xB}$

ข้อมูลที่ใช้ $\mathbf{B}$ จากตัวอย่างการคำนวณ 5.29

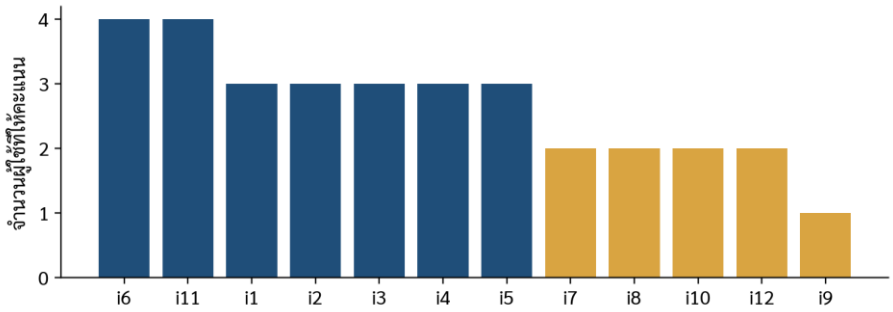
- $\mathbf{x}$ มีค่า 1 เฉพาะ i1 คะแนนจึงเท่ากับแถว i1 ของ $\mathbf{B}$ พอดี
- เรียงได้ i3 (0.3166), i2 (0.3103), i7 (0.2581), i8 (0.2581)
- User-kNN ทำไม่ได้ เพราะผู้ใช้ที่มีคะแนนเรื่องเดียวคำนวณ Pearson ไม่ได้ (หัวข้อ 5.3.2)

ตีความ EASE แนะนำ The Wild Robot และ Inside Out 2 (ผ่าน u1) ตามด้วย Ballerina และ Furiosa (ผ่าน u4) ทันทีที่ผู้ใช้มีการกระทำแรก นี่คือข้อดีของแบบจำลองรายการ–รายการ ซึ่งไม่ต้องมีพารามิเตอร์ของผู้ใช้ ส่วน MF ต้องคำนวณ $\mathbf{p}_u$ ใหม่ด้วยสมการ (5.20) ก่อน

5.14.2 ความเบาบาง

ตารางที่ 5.7 แสดงแล้วว่าผู้ใช้ 21 คู่จาก 28 คู่มีรายการร่วมไม่เกินหนึ่งเรื่อง แม้ความหนาแน่นจะสูงถึง 33% ข้อมูลจริงเบาบางกว่านี้หลายร้อยเท่า การหาเพื่อนบ้านโดยตรงจึงทำได้ยาก แบบจำลองปัจจัยแฝงและกราฟช่วยได้ เพราะเชื่อมผู้ใช้ผ่าน "เพื่อนของเพื่อน" ได้ เช่น ห่วงโซ่ u1–u4–u3 ในหัวข้อ 5.7.1

5.14.3 อคติความนิยม



ภาพที่ 5.25 จำนวนผู้ให้คะแนนของแต่ละรายการ

ภาพที่ 5.25 แสดงว่าภาพยนตร์ 7 เรื่องมีผู้ให้คะแนนอย่างน้อย 3 คน (แท่งสีน้ำเงิน) ส่วนอีก 5 เรื่องมีเพียง 1 – 2 คน การกรองร่วมมีแนวโน้มแนะนำเรื่องที่มีข้อมูลมาก เพราะความคล้ายและปัจจัยแฝงของเรื่องเหล่านั้นน่าเชื่อถือกว่า ผลคือเรื่องยอดนิยมยิ่งถูกแนะนำ และเรื่องในหางยาวยิ่งถูกมองข้าม (Abdollahpouri et al., 2017)

ตารางที่ 5.29 สัดส่วนของช่อง Top-3 ที่เป็นรายการกลุ่มหัว

วิธี	สัดส่วนกลุ่มหัว
ความนิยม	0.9583
User-kNN	0.6250
Item-kNN	0.6250
WRMF	0.5000
EASE	0.7917

ที่มา: Top-3 ของผู้ใช้ 8 คน (24 ช่อง) จากตารางที่ 5.28 กลุ่มหัวคือรายการที่มีผู้ให้คะแนนอย่างน้อย 3 คน ได้แก่ i1, i2, i3, i4, i5, i6, i11

ตารางที่ 5.29 ยืนยันว่ารายการกลุ่มหัวมีเพียง 7 จาก 12 เรื่อง (58%) แต่ความนิยมใช้ช่อง Top-3 ให้กลุ่มนี้ถึง 0.9583 และ EASE ใช้ 0.7917 ขณะที่ WRMF กระจายให้หางยาวได้มากที่สุด การวัดและแก้อคตินี้จะอธิบายในบทที่ 9

5.14.4 การขยายขนาด

ตารางที่ 5.30 ต้นทุนการคำนวณโดยประมาณของแต่ละวิธี

วิธี	ต้นทุนหลัก	ทำเมื่อใด
User-kNN	เทียบผู้ใช้ทุกคู่ $O(\|\mathcal{U}\|^2)$	ออนไลน์หรือล่วงหน้า
Item-kNN	เทียบรายการทุกคู่ $O(\|\mathcal{I}\|^2)$	ล่วงหน้า (ออฟไลน์)
MF (SGD)	$O(\|\mathcal{K}\|\, k)$ ต่อรอบ	ออฟไลน์
ALS / WRMF	แก้ระบบ $k \times k$ ต่อผู้ใช้และรายการ	ออฟไลน์ ขนานได้

วิธี	ต้นทุนหลัก	ทำเมื่อใด
EASE	ผกผัน $\lvert\mathcal{I}\rvert \times \lvert\mathcal{I}\rvert$ คือ $O(\lvert\mathcal{I}\rvert^3)$	ออฟไลน์ เหมาะเมื่อรายการไม่เกินราวแสนเรื่อง

ตัวอย่างการคำนวณ 5.35 ต้นทุนของการเทียบผู้ใช้ทุกคู่

โจทย์ จงประมาณจำนวนคู่ผู้ใช้และเวลาที่ต้องใช้

สมการที่ใช้ จำนวนคู่ = $\lvert\mathcal{U}\rvert(\lvert\mathcal{U}\rvert - 1)/2$

ข้อมูลที่ใช้ สมมติว่าเทียบได้ 10 ล้านคู่ต่อวินาที

- MovieMini-12: $8 \times 7/2 = 28$ คู่
- ผู้ใช้ 100,000 คน: $100{,}000 \times 99{,}999/2 \approx 5.0 \times 10^9$ คู่ ใช้ราว 500 วินาที
- ผู้ใช้ 1 ล้านคน: ราว 5.0×10^{11} คู่ ใช้ราว 14 ชั่วโมง

ตีความ ผู้ใช้เพิ่ม 10 เท่า ต้นทุนเพิ่ม 100 เท่า ระบบจริงจึงคำนวณล่วงหน้าแบบออฟไลน์ และใช้ดัชนีค้นหาเพื่อนบ้านใกล้สุดโดยประมาณในขั้นออนไลน์ (บทที่ 7)

5.14.5 ผู้ใช้แกะเทา

ผู้ใช้แกะเทา (Gray Sheep) คือผู้ใช้ที่รสนิยมไม่ตรงกับกลุ่มใดชัดเจน u5 และ u6 ใน MovieMini-12 มีลักษณะนี้ เช่น u5 ชอบ The Wild Robot กับ Conclave ซึ่งต่างแนวกันมาก ภาพที่ 5.23 แสดงว่าทุกวิธีได้ MRR ต่ำกับทั้งสองคน รวมทั้งระบบอิงเนื้อหาด้วย

ผู้ใช้กลุ่มนี้ได้ประโยชน์จากโปรไฟล์หลายจุดสนใจ (บทที่ 4 หัวข้อ 4.8.5) หรือการเพิ่มความหลากหลายในรายการแนะนำ ส่วนประเด็นการโจมตีด้วยโปรไฟล์ปลอม (Shilling Attack) และความเป็นส่วนตัวของข้อมูลพฤติกรรม จะอธิบายในบทที่ 9

อธิบายแบบเข้าใจง่าย การกรองร่วมคือ "ภูมิปัญญาของฝูงชน" ที่ต้องมีฝูงชน

ความท้าทายทั้งห้าข้อมีรากเดียวกัน การกรองร่วมเก่งเมื่อมีผู้คนจำนวนมากแสดงพฤติกรรมที่คล้ายกันซ้ำ ๆ เมื่อฝูงชนยังไม่มา (เริ่มต้นเย็น) มาน้อย (เบาบาง) มารวมกันที่เดียว (อคติความนิยม) มามากเกิน (ขยายขนาด) หรือเราเป็นคนเดียวที่เดินแปลกทาง (แกะเทา) ความฉลาดของฝูงชนก็ลดลง

โปรแกรม 5.15 จำลองการเริ่มต้นเย็นและแนะนำผู้ใช้ใหม่ด้วย EASE

```
import numpy as np


R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
items = ['i%d' % k for k in range(1, 13)]
```

```
def ease_B(Rd, lam=1.0):
    X = np.array([[1.0 if Rd[u].get(i, 0) >= 4 else 0.0 for i in items]
                  for u in Rd])
    P = np.linalg.inv(X.T @ X + lam * np.eye(12))
    B = -P / np.diag(P)
    np.fill_diagonal(B, 0)
    return X, B



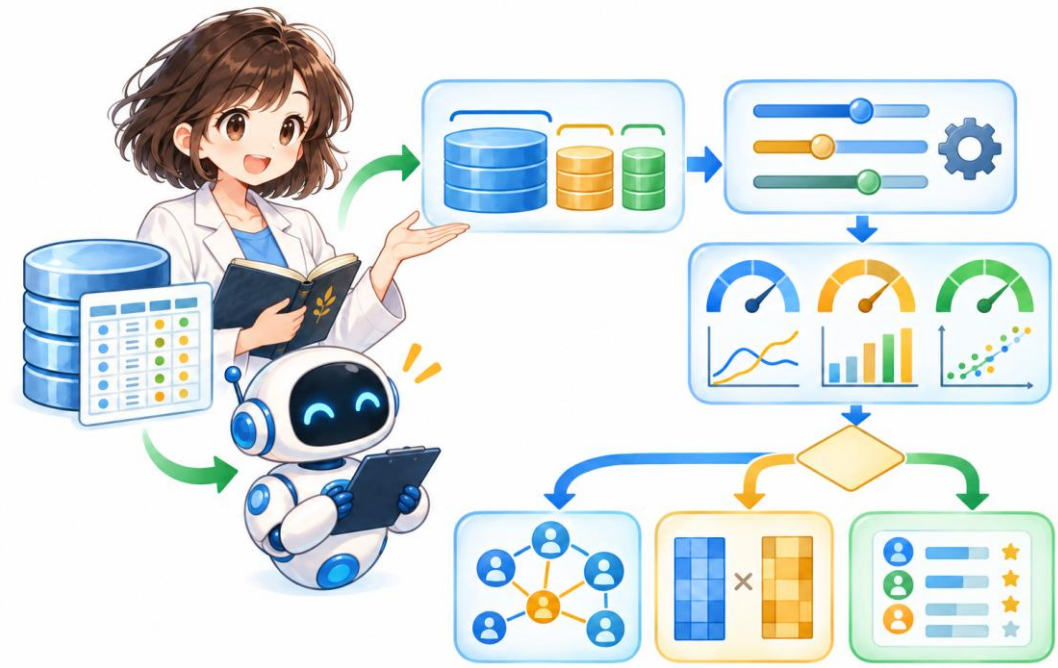
X, B = ease_B(R)


# ผู้ใช้ใหม่ชอบ i1 เพียงเรื่องเดียว
x_new = np.zeros(12)
x_new[0] = 1
s = x_new @ B
top = sorted(((round(float(s[b]), 4), items[b]) for b in range(1, 12)),
             reverse=True)
print('ผู้ใช้ใหม่', top[:4])


# ซ่อนคะแนนเดียวของ i9 (u5 ให้ 5) เพื่อจำลองภาพยนตร์ใหม่
Rc = {u: {i: r for i, r in d.items() if not (u == 'u5' and i == 'i9')}
      for u, d in R.items()}
Xc, Bc = ease_B(Rc)
before = (X[0] @ B)[8]
after = (Xc[0] @ Bc)[8]
print('คะแนน i9 ของ u1 ก่อน/หลังซ่อน',
      round(float(before), 4), round(float(after), 4))
```

5.15 แนวปฏิบัติ: เตรียมข้อมูล ตั้งค่า และเลือกแบบจำลอง

5.15.1 การเตรียมข้อมูลก่อนสร้างระบบ



ภาพที่ 5.41 แนวปฏิบัติและการเลือกแบบจำลอง

คุณภาพของการกรองร่วมขึ้นกับการเตรียมข้อมูลไม่น้อยกว่าการเลือกสูตร ขั้นตอนเต็มอยู่ในบทที่ 1 หัวข้อ 1.13 ในที่นี้สรุปเฉพาะขั้นที่มีผลต่อบทนี้โดยตรง

1. **ขจัดข้อมูลซ้ำ** ถ้าผู้ใช้ให้คะแนนรายการเดิมหลายครั้ง ให้เก็บค่าล่าสุดหรือค่าเฉลี่ย และบันทึกกฎไว้
2. **ทำดัชนีใหม่และใช้เมทริกซ์เบาบาง** แปลงรหัสผู้ใช้และรายการเป็นเลข 0 ถึง $m-1$ และ 0 ถึง $n-1$ แล้วเก็บเป็นเมทริกซ์ CSR ห้ามสร้างเมทริกซ์เต็ม
3. **กำหนดนิยามของ "ชอบ"** สำหรับข้อมูลโดยปริยาย เช่น ดูเกินร้อยละ 70 ของความยาว หรือซื้อซ้ำ และแปลงความเข้มของสัญญาณ เช่น ใช้ $\log(1 + \text{จำนวนครั้ง})$
4. **กรองแบบ k-core อย่างระมัดระวัง**
   การตัดผู้ใช้และรายการที่มีปฏิสัมพันธ์น้อยช่วยให้ความคล้ายเสถียรขึ้น แต่ทำให้ตัวชี้วัดดูดีเกินจริง ต้องรายงานค่า k และสัดส่วนที่ถูกตัด
5. **แบ่งข้อมูลตามเวลา** ใช้ข้อมูลอดีตฝึกและข้อมูลอนาคตทดสอบ (บทที่ 2 หัวข้อ 2.8)
6. **คำนวณสถิติทุกตัวจากชุดฝึกเท่านั้น** ได้แก่ ค่าเฉลี่ย ค่าเอนเอียง และตารางความคล้าย (ข้อควรระวังในหัวข้อ 5.4.4)

5.15.2 เลือกไฮเปอร์พารามิเตอร์อย่างถูกวิธี

ทุกเทคนิคในบทนี้มีไฮเปอร์พารามิเตอร์ และหัวข้อ 5.7.7 กับ 5.9 แสดงแล้วว่าค่าที่ต่างกันเปลี่ยนผลได้มาก เช่น α ของ WRMF เปลี่ยน MRR จาก 0.3202 เป็น 0.5444 ความผิดพลาดที่พบบ่อยที่สุดคือ **เลือกค่าบนชุดทดสอบ** แล้วรายงานผลบนชุดเดียวกัน ซึ่งทำให้ผลดีเกินจริง แนวปฏิบัติที่ถูกต้องมีดังนี้

1. แบ่งข้อมูลเป็นสามส่วน คือ ฝึก ตรวจสอบ และทดสอบ

2. เลือกค่าด้วยตัวชี้วัดที่ตรงกับงานจริง ถ้าระบบจัดอันดับ ให้เลือกด้วย nDCG หรือ MRR ไม่ใช่ RMSE (ตารางที่ 5.20)
3. ค้นหาบนมาตราลอการิทึมสำหรับ λ, η และ α เช่น 0.001, 0.01, 0.1, 1 และใช้การหยุดก่อนกำหนด (Early Stopping) กำหนดจำนวนรอบ
4. เมื่อเลือกได้แล้ว ให้ฝึกใหม่บนข้อมูลฝึกรวมชุดตรวจสอบ แล้วรายงานผลบนชุดทดสอบเพียงครั้งเดียว พร้อมค่าเฉลี่ยจากหลายเมล็ดสุ่ม (หัวข้อ 5.13.3)
5. ปรับระบบฐานทุกตัวด้วยความพยายามเท่ากัน
   เพราะงานวิจัยจำนวนมากชนะระบบฐานที่ไม่ได้ปรับเท่านั้น (Ferrari Dacrema et al., 2019; Rendle et al., 2022)

ตัวอย่างการคำนวณ 5.36 เลือก λ ของ MF ด้วยชุดตรวจสอบ

โจทย์ แบ่ง 20 กรณีของ LOO เป็นชุดตรวจสอบ (กรณีของ u1–u4 จำนวน 12 กรณี) และชุดทดสอบ (u5–u8 จำนวน 8 กรณี) จงเลือก λ จากชุดตรวจสอบ แล้วรายงานผลบนชุดทดสอบ

สมการที่ใช้ MRR (บทที่ 2 หัวข้อ 2.5)

ข้อมูลที่ใช้ อันดับรายกรณีของ MF ที่ λ = 0, 0.05, 0.5 และ 2 (ข้อมูลเดียวกับตารางที่ 5.20)

λ	MRR ชุดตรวจสอบ (12 กรณี)	MRR ชุดทดสอบ (8 กรณี)
0.0	0.1899	0.2552
0.05	0.2295	0.2115
0.5	**0.3674**	0.2823
2.0	0.2088	0.1961

- ชุดตรวจสอบเลือก λ = 0.5 (MRR 0.3674 สูงสุด)
- รายงานผลบนชุดทดสอบ: MRR = 0.2823
- ผลบนชุดทดสอบต่ำกว่าบนชุดตรวจสอบ ซึ่งเป็นเรื่องปกติ เพราะค่าที่เลือกถูก "ปรับให้เข้ากับ" ชุดตรวจสอบไปแล้วบางส่วน

ตีความ ถ้าเลือกค่าจากชุดทดสอบโดยตรง เราจะไม่มีทางรู้ว่าผลที่รายงานสูงเกินจริงเพียงใด บนข้อมูลเล็กเช่นนี้ ควรใช้การตรวจสอบไขว้ซ้อน (Nested Cross-validation) ซึ่งเลือกค่าภายในแต่ละรอบของการกันออก โดยไม่ให้คะแนนที่ซ่อนมีส่วนในการเลือก

5.15.3 สรุปที่มาของไฮเปอร์พารามิเตอร์ทั้งบท

ตารางที่ 5.31 ไฮเปอร์พารามิเตอร์ของการกรองร่วม: ค่าที่ใช้และที่มา

ไฮเปอร์พารามิเตอร์	เทคนิค	ค่าในบท	ค่าในงานต้นแบบ	วิธีเลือกในงานจริง
β ถ่วงนัยสำคัญ	User/Item-kNN	3	50 (Herlocker et al., 1999)	ราวจำนวนรายการร่วมปกติของข้อมูล
k เพื่อนบ้าน	User/Item-kNN	2	20–50 (Herlocker et al., 1999)	ลอง 10–100 บนชุดตรวจสอบ
M เพื่อนบ้านที่เก็บต่อรายการ	Item-kNN (ให้บริการ)	ทั้งหมด	หลักสิบ–ร้อย (Sarwar et al., 2001)	ดูหน่วยความจำและผลบนชุดตรวจสอบ

ไฮเปอร์พารามิเตอร์	เทคนิค	ค่าในบท	ค่าในงานต้นแบบ	วิธีเลือกในงานจริง
λ SHRINKAGE	ค่าเอนเอียง, kNN-Baseline	2	25 และ 10 (Koren, 2010)	ตามจำนวนคะแนนต่อรายการ
k ปัจจัย	MF, ALS, WRMF, BPR	2	50–200 (Koren et al., 2009)	เพิ่มจนผลบนชุดตรวจสอบหยุดดีขึ้น
η อัตราการเรียนรู้	MF (SGD), BPR	0.05	0.001 (Funk, 2006)	มาตราลอการิทึม ดูว่าไม่ลู่ออก
λ เรกูลาไรเซชัน	MF, BPR, WRMF	0.05, 0.01, 0.1	0.02 (Funk, 2006)	มาตราลอการิทึม เลือกด้วย MRR/nDCG
α ความเชื่อมั่น	WRMF	2	40 (Hu et al., 2008)	ลอง 1–100 ตามมาตราของ r
λ ของ EASE	EASE	1	หลายร้อยถึงหลายพัน (Steck, 2019)	ขนาดเทียบกับค่าใน $\mathbf{X}^\top\mathbf{X}$
β การสุ่มลบ	BPR	0.75	0.75 (Mikolov et al., 2013)	ลอง 0–1
เกณฑ์ "ชอบ"	ข้อมูลโดยปริยาย	$r \geq 4$	ขึ้นกับโดเมน	เช่น ดูเกินร้อยละ 70

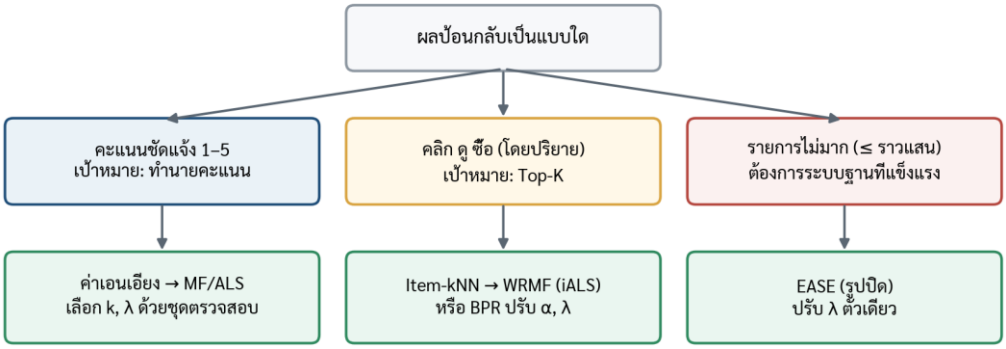
ตารางที่ 5.31 สรุปไฮเปอร์พารามิเตอร์ทุกตัวในบท ข้อสังเกตสำคัญคือ **ค่าในงานต้นแบบเกือบทั้งหมดใหญ่กว่าค่าในบทนี้มาก** เพราะงานเหล่านั้นใช้ข้อมูลที่มีผู้ใช้หลายแสนคน ค่าเหล่านี้จึงเป็นเพียงจุดเริ่มต้นของการค้นหา ไม่ใช่ค่าที่ใช้ได้กับทุกข้อมูล

5.15.4 ตารางวินิจฉัยปัญหาที่พบบ่อย

ตารางที่ 5.32 อาการที่พบบ่อย สาเหตุ และวิธีแก้

อาการ	สาเหตุที่เป็นไปได้	วิธีแก้
คำทำนายหลายรายการเท่ากับค่าเฉลี่ยของผู้ใช้	ไม่มีเพื่อนบ้านเชิงบวก (ความเบาบาง)	ใช้ kNN-Baseline, Item-kNN โดยปริยาย หรือ MF
ผลแย่ลงมากเมื่อเปลี่ยน Pearson เป็น Cosine	Cosine ดิบให้ความคล้ายบวกแก่ผู้ใช้ที่รสนิยมตรงข้าม	ลบค่าเฉลี่ยก่อน (ตารางที่ 5.11)
เพื่อนบ้านอันดับหนึ่งมีรายการร่วม 1–2 เรื่อง	ไม่ได้ถ่วงนัยสำคัญ	ใช้สมการ (5.6) หรือ SHRINKAGE
ผลออฟไลน์ดีเกินจริง	ใช้สถิติหรือความคล้ายที่รวมข้อมูลทดสอบ	คำนวณจากชุดฝึกเท่านั้น (หัวข้อ 5.4.4)
RMSE บนข้อมูลฝึกเกือบศูนย์ แต่ข้อมูลที่ซ่อนแย่	MF เรียนรู้เกิน	เพิ่ม λ ลด k หยุดก่อนกำหนด
ค่าสูญเสียกลายเป็น nan	η ใหญ่เกิน	ลด η (ตารางที่ 5.17)
ผลเปลี่ยนทุกครั้งที่รัน	ไม่ได้กำหนดเมล็ดสุ่มหรือฝึกหลายเธรด	กำหนดเมล็ดสุ่ม รายงานค่าเฉลี่ยหลายเมล็ด
แนะนำรายการชุดเดิมให้ทุกคน	อคติความนิยม	วัด Coverage และ Novelty (ตารางที่ 5.28)
รายการใหม่ไม่เคยถูกแนะนำ	การเริ่มต้นเย็นของรายการ	ผสานกับระบบอิงเนื้อหา (บทที่ 3–4 และบทที่ 7)
ระบบช้าเมื่อจำนวนผู้ใช้เพิ่ม	ใช้ User-kNN คำนวณสดทุกคำขอ	เปลี่ยนเป็น Item-kNN พร้อมตารางเพื่อนบ้านล่วงหน้า

5.15.5 เลือกแบบจำลองให้เหมาะกับข้อมูล



ภาพที่ 5.26 แนวทางเลือกแบบจำลองการกรองร่วม

ภาพที่ 5.26 สรุปแนวทางเลือกแบบจำลองที่ใช้เป็นจุดเริ่มต้นได้ คำถามแรกคือ **ชนิดของผลป้อนกลับ** ถ้าเป็นคะแนนชัดแจ้งและต้องการทำนายคะแนน ให้เริ่มจากค่าเอนเอียงแล้วจึง MF หรือ ALS ถ้าเป็นคลิก การดู หรือการซื้อ และเป้าหมายคือ Top-K ให้เริ่มจาก Item-kNN แบบโดยปริยาย แล้วจึง WRMF หรือ BPR คำถามที่สองคือ **ขนาดของคลัง** EASE ต้องใช้หน่วยความจำเป็นสัดส่วนกับ $|\mathcal{I}|^2$ จึงเหมาะเมื่อมีรายการไม่เกินราวหนึ่งแสนเรื่อง ส่วน WRMF ขยายได้ถึงหลายล้านรายการ

ตารางที่ 5.33 เปรียบเทียบวิธีการกรองร่วมทั้งหมดในบทนี้

วิธี	ข้อมูลที่ใช้	สิ่งที่ต้องเรียน	ไฮเปอร์พารามิเตอร์หลัก	จุดเด่น	ข้อจำกัด
User-kNN	คะแนน	ไม่มี (อิงความจำ)	k, β	อธิบายง่าย ปรับตามบุคคลชัด	ขยายขนาดยาก ไวต่อความเบาบาง
Item-kNN	คะแนน	ตารางความคล้ายรายการ	k, β, M	คำนวณล่วงหน้าได้ อธิบายด้วยรายการ	รายการใหม่ไม่มีเพื่อนบ้าน
Item-kNN โดยปริยาย	การกระทำ	ตารางความคล้ายรายการ	M, เกณฑ์ชอบ	ง่าย เร็ว ใช้กับคลิกได้	คะแนนเสมอกันบ่อย
ค่าเอนเอียง	คะแนน	μ, b_u, b_i	λ	เรียบง่าย เป็นฐานของ MF	ไม่ปรับอันดับตามบุคคล
kNN-Baseline	คะแนน	ค่าเอนเอียง + ความคล้าย	k, β, λ	ไม่ถอยไปค่าเฉลี่ยหยาบ ๆ	ยังไวต่อความเบาบาง
MF (SGD/ALS)	คะแนน	$\mathbf{p}_u, \mathbf{q}_i, b$	k, η, λ	จับโครงสร้างแฝง	เรียนรู้เกินเมื่อข้อมูลน้อย
SVD++	คะแนน + การเลือกดู	$\mathbf{p}_u, \mathbf{q}_i, \mathbf{y}_j, b$	เหมือน MF	ใช้ข้อมูลการเลือกดู	ฝึกช้ากว่า MF
WRMF (iALS)	การกระทำ	$\mathbf{p}_u, \mathbf{q}_i$	k, λ, α	ใช้ช่องศูนย์อย่างมีน้ำหนัก ขยายขนาดได้	ไวต่อ α
BPR	การกระทำ	$\mathbf{p}_u, \mathbf{q}_i$	k, η, λ, วิธีสุ่มลบ	เรียนลำดับโดยตรง	ไวต่อการสุ่มและเมล็ดสุ่ม
EASE	การกระทำ	B (รูปปิด)	λ	แข็งแรง ไม่สุ่ม ไฮเปอร์พารามิเตอร์ตัวเดียว	หน่วยความจำ $\|\|\times\|\|$

ข้อควรระวัง เวกเตอร์ปัจจัยแฝงเปรียบเทียบข้ามการฝึกไม่ได้

ถ้าหมุนหรือสลับเครื่องหมายของมิติทั้งใน P และ Q พร้อมกัน ผลคูณ PQ^Tไม่เปลี่ยน ปริภูมิแฝงจึงไม่มีทิศทางที่ตายตัว เวกเตอร์ของผู้ใช้จากการฝึกเมื่อวานกับเวกเตอร์ของรายการจากการฝึกวันนี้ห้ามนำมาคูณกัน ระบบที่ให้บริการจริงต้องเก็บ P และ Q จากการฝึกครั้งเดียวกันไว้ด้วยกันเสมอ ส่วนผู้ใช้ใหม่หลังฝึก ให้ตรึง Q แล้วแก้สมการ (5.20) สำหรับผู้ใช้คนนั้นคนเดียว (Fold-in) ซึ่งใช้เวลาน้อยมาก

โปรแกรม 5.16 เลือก λ ของ MF ด้วยชุดตรวจสอบ

```
import numpy as np

R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
users = list(R)
items = ['i%d' % k for k in range(1, 13)]



def train_mf(Rd, k=2, lr=0.05, lam=0.05, epochs=200, seed=7):
    rng = np.random.default_rng(seed)
    P = {u: rng.normal(0, 0.1, k) for u in users}
    Q = {i: rng.normal(0, 0.1, k) for i in items}
    bu = {u: 0.0 for u in users}
    bi = {i: 0.0 for i in items}
    data = [(u, i, r) for u, d in Rd.items() for i, r in d.items()]
    mu = np.mean([r for _, _, r in data])
    for epoch in range(epochs):
        for idx in rng.permutation(len(data)):
            u, i, r = data[idx]
            # 1) ความคลาดเคลื่อน
            e = r - (mu + bu[u] + bi[i] + P[u] @ Q[i])
            # 2) ปรับค่าเอนเอียง
            bu[u] += lr * (e - lam * bu[u])
            bi[i] += lr * (e - lam * bi[i])
            # 3) ปรับเวกเตอร์ (ใช้ p_u ค่าเดิมในการปรับ q_i)
            pu = P[u].copy()
            P[u] = P[u] + lr * (e * Q[i] - lam * P[u])
            Q[i] = Q[i] + lr * (e * pu - lam * Q[i])

    def predict(u, i):
        return mu + bu[u] + bi[i] + P[u] @ Q[i]
    return predict, P



def reciprocal_ranks(lam):
    out = []
    for u in users:
        for h in [i for i, r in R[u].items() if r >= 4]:
            # ซ่อนคะแนน (u, h) แล้วฝึกจากข้อมูลที่เหลือ
            Rd = {v: {j: r for j, r in R[v].items()
                      if not (v == u and j == h)} for v in users}
            if not any(r >= 4 for r in Rd[u].values()):
                continue
            predict, _ = train_mf(Rd, lam=lam)
```

```
            cands = [i for i in items if i not in R[u]] + [h]
            sc = {i: predict(u, i) for i in cands}
            order = sorted(cands,
                           key=lambda i: (-round(sc[i], 12), int(i[1:])))
            out.append((u, 1 / (order.index(h) + 1)))
    return out



VAL = ('u1', 'u2', 'u3', 'u4')
for lam in (0.0, 0.05, 0.5, 2.0):
    rr = reciprocal_ranks(lam)
    val = [x for u, x in rr if u in VAL]
    test = [x for u, x in rr if u not in VAL]
    print(lam, 'validation', round(float(np.mean(val)), 4),
          'test', round(float(np.mean(test)), 4))
```

5.16 แนวทางโปรแกรมประกอบหัวข้อและ Lab ท้ายบท

5.16.1 แผนที่ของโปรแกรมประกอบหัวข้อ ดังสรุปในภาพที่ 5.42

ภาพนี้สรุปการทดลองที่ทำซ้ำได้ ตั้งแต่ข้อมูลและโปรแกรมไปจนถึงการเปรียบเทียบผล



ภาพที่ 5.42 โปรแกรมประกอบหัวข้อและ Lab

ทุกหัวข้อหลักมีโปรแกรมสั้นที่คำนวณตัวอย่างซ้ำได้ด้วย NumPy อย่างเดียว ผู้อ่านควรรันโปรแกรมของแต่ละหัวข้อทันทีหลังอ่านตัวอย่างการคำนวณ แล้วเทียบตัวเลขที่ได้กับในตัวอย่าง ตารางที่ 5.34 สรุปแผนที่ของโปรแกรม

ตารางที่ 5.34 แผนที่ของโปรแกรมประกอบหัวข้อในบทที่ 5

โปรแกรม	สิ่งที่คำนวณ	หัวข้อ
5.1	สถิติพื้นฐาน จำนวนรายการร่วม และโปรไฟล์หลังลบค่าเฉลี่ย	5.2
5.2	Jaccard, Cosine, Pearson และการถ่วงนัยสำคัญ	5.3

โปรแกรม	สิ่งที่คำนวณ	หัวข้อ
5.3	User-kNN สามแบบ และเพื่อนบ้านลบ	5.4
5.4	Adjusted Cosine และ Item-kNN	5.5
5.5	Item-kNN แบบโดยปริยาย	5.5.4
5.6	ค่าเอนเอียงและการตรวจค่าสูญเสีย	5.6
5.7	kNN-Baseline	5.6.4
5.8	MF ด้วย SGD	5.7
5.9	SVD 3×3 และ ALS 2×2	5.7–5.8
5.10	WRMF ด้วย ALS และสูตรแยกส่วน	5.9
5.11	BPR หนึ่งก้าวและค่าสูญเสีย	5.10
5.12	EASE ด้วยมือและบนข้อมูลทั้งหมด	5.11
5.13	ความน่าจะเป็นของการสุ่มลบและการจำลอง	5.12
5.14	HR@3, MRR, nDCG@3, RMSE และ MAE	5.13
5.15	การเริ่มต้นเย็นและผู้ใช้ใหม่	5.14
5.16	เลือก λ ด้วยชุดตรวจสอบ	5.15
5.17	วิธีเพื่อนบ้านและ SVD ด้วย Surprise	5.16.2
5.18	ALS และ BPR ด้วย implicit	5.16.2

5.16.2 ใช้ไลบรารีมาตรฐานกับ MovieMini-12

หลังจากเขียนทุกวิธีเองด้วย NumPy แล้ว ผู้อ่านควรรู้วิธีใช้ไลบรารีมาตรฐานซึ่งเร็วกว่ามากบนข้อมูลจริง ไลบรารี **Surprise** (Hug, 2020) รวบรวมวิธีเพื่อนบ้านและ MF สำหรับคะแนนชัดแจ้ง ส่วนไลบรารี **implicit** (Frederickson, 2016) รวบรวม ALS และ BPR สำหรับข้อมูลโดยปริยาย

โปรแกรม 5.17 วิธีเพื่อนบ้านและ SVD ด้วยไลบรารี Surprise

```
import pandas as pd
from surprise import Dataset, Reader, KNNWithMeans, SVD


R = {
    'u1': {'i1': 5, 'i2': 5, 'i3': 4, 'i4': 1, 'i6': 1},
    'u2': {'i4': 5, 'i5': 4, 'i6': 5, 'i1': 2},
    'u3': {'i10': 5, 'i11': 5, 'i12': 4, 'i7': 1},
    'u4': {'i1': 4, 'i7': 5, 'i8': 5, 'i5': 1},
    'u5': {'i3': 4, 'i9': 5, 'i10': 2, 'i6': 3},
    'u6': {'i2': 5, 'i5': 4, 'i11': 3},
    'u7': {'i11': 4, 'i12': 5, 'i8': 2, 'i3': 3},
    'u8': {'i4': 4, 'i6': 5, 'i2': 2, 'i11': 3},
}
rows = [(u, i, r) for u in R for i, r in R[u].items()]
df = pd.DataFrame(rows, columns=['user', 'item', 'rating'])
data = Dataset.load_from_df(df, Reader(rating_scale=(1, 5)))
ts = data.build_full_trainset()


# User-kNN แบบลบค่าเฉลี่ย (KNNWithMeans) ด้วยสองมาตรวัด
for name in ('pearson', 'cosine'):
```

```
    opts = {'name': name, 'user_based': True}
    algo = KNNWithMeans(k=2, min_k=1, verbose=False, sim_options=opts)
    algo.fit(ts)
    est = {i: round(float(algo.predict('u1', i).est), 4)
           for i in ('i7', 'i8', 'i9', 'i12')}
    print(name, est)


# SVD ของ Surprise คือ MF แบบมีค่าเอนเอียง (ไม่ใช่ SVD ทางคณิตศาสตร์)
svd = SVD(n_factors=2, n_epochs=300, lr_all=0.05, reg_all=0.05,
          init_std_dev=0.1, biased=True, random_state=7)
svd.fit(ts)
cands = ('i5', 'i7', 'i8', 'i9', 'i10', 'i11', 'i12')
est = {i: round(float(svd.predict('u1', i).est), 4) for i in cands}
print('SVD', sorted(est.items(), key=lambda t: -t[1])[:3])
```

ผลของ Surprise ต่างจากตารางที่ 5.10 ในจุดที่น่าสนใจ ซึ่งผู้อ่านควรอธิบายได้

- **Pearson** ให้ i9 = 4.7000 ตรงกับตารางที่ 5.10 แต่ให้ i7 และ i8 = 3.2000 แทน 4.45 เพราะ Surprise ลบค่าเฉลี่ยบน **รายการร่วมเท่านั้น** เมื่อ u1 กับ u4 มีรายการร่วมเรื่องเดียว ความคล้ายจึงเป็น 0 และ u4 ไม่ถูกนับเป็นเพื่อนบ้าน
- **Cosine** ให้ i12 = 4.7000 เพราะ Cosine ดิบให้ u7 เป็นเพื่อนบ้านเชิงบวก ทั้งที่ Pearson บอกว่ารสนิยมตรงข้ามกับ u1 (หัวข้อ 5.3.1)
- **SVD** ของ Surprise คือ MF แบบมีค่าเอนเอียงในสมการ (5.16) ไม่ใช่ SVD ของพีชคณิตเชิงเส้น ให้ Top-3 เป็น i9, i5, i12 ต่างจาก Top-3 ของ MF ในบทนี้ เพราะลำดับการสุ่มภายในต่างกัน (หัวข้อ 5.13.3)

โปรแกรม 5.18 ALS และ BPR ด้วยไลบรารี implicit

```
import numpy as np
import scipy.sparse as sp
from implicit.als import AlternatingLeastSquares
from implicit.bpr import BayesianPersonalizedRanking


items = ['i%d' % k for k in range(1, 13)]
liked = {
    'u1': ['i1', 'i2', 'i3'], 'u2': ['i4', 'i5', 'i6'],
    'u3': ['i10', 'i11', 'i12'], 'u4': ['i1', 'i7', 'i8'],
    'u5': ['i3', 'i9'], 'u6': ['i2', 'i5'],
    'u7': ['i11', 'i12'], 'u8': ['i4', 'i6'],
}
dense = [[1.0 if i in v else 0.0 for i in items] for v in liked.values()]
L = sp.csr_matrix(np.array(dense, dtype=np.float32))


als = AlternatingLeastSquares(factors=2, regularization=0.1, alpha=2.0,
                              iterations=15, random_state=3)
als.fit(L, show_progress=False)
ids, sc = als.recommend(0, L[0], N=3, filter_already_liked_items=True)
print('implicit ALS u1:',
      [(items[i], round(float(s), 4)) for i, s in zip(ids, sc)])


# num_threads=1 ทำให้ผลของ BPR ทำซ้ำได้
bpr = BayesianPersonalizedRanking(factors=2, learning_rate=0.05,
                                  regularization=0.01, iterations=400,
                                  random_state=5, num_threads=1)
bpr.fit(L, show_progress=False)
ids, sc = bpr.recommend(0, L[0], N=3, filter_already_liked_items=True)
print('implicit BPR u1:',
      [(items[i], round(float(s), 4)) for i, s in zip(ids, sc)])
```

ไลบรารี implicit ให้ Top-3 ของ ALS เป็น i5, i8, i7 และของ BPR เป็น i9, i8, i7 ต่างจากโปรแกรม 5.10 ทั้งที่ใช้ค่าตั้งใกล้กัน สาเหตุคือไลบรารีคำนวณความเชื่อมั่นเป็น $1 + \alpha \times$ ค่าในเมทริกซ์ (ซึ่งเป็น 1 ทุกช่องที่ชอบ) แทน $1 + \alpha r$ ใช้วิธี Conjugate Gradient แทนการแก้สมการตรง และเริ่มต้นเวกเตอร์ต่างกัน สำหรับ BPR ต้องกำหนด `num_threads=1` เพราะการฝึกหลายเธรดทำให้ผลเปลี่ยนทุกครั้งที่รัน บทเรียนคือ **การใช้ไลบรารีไม่ได้ทำให้ผู้พัฒนาพ้นจากการเข้าใจสูตร** เมื่อผลต่างจากที่คาด ผู้พัฒนาต้องย้อนกลับไปตรวจนิยามได้เอง

5.16.3 แม่แบบการทดลองบนข้อมูลจริง

แม่แบบต่อไปนี้ใช้ทดลองวิธีหลักของบทบน MovieLens-1M (Harper & Konstan, 2015) ซึ่งมีคะแนนราวหนึ่งล้านค่าจากผู้ใช้ราว 6,000 คนต่อภาพยนตร์ราว 3,700 เรื่อง แม่แบบแยกการประเมินเป็นสองงาน คือ ทำนายคะแนนด้วย RMSE และจัดอันดับด้วย HR@10 บนการกันออกรายการล่าสุดที่ชอบของผู้ใช้แต่ละคน (Leave-Last-Out) ผู้อ่านต้องดาวน์โหลดข้อมูลจาก GroupLens เอง ผลจึงไม่แสดงในตำรา

โปรแกรม 5.19 แม่แบบการทดลองบน MovieLens-1M (ต้องดาวน์โหลดข้อมูลก่อน)

```
import numpy as np
import pandas as pd
import scipy.sparse as sp
from surprise import Dataset, Reader, SVD, accuracy
from implicit.als import AlternatingLeastSquares


cols = ['user', 'item', 'rating', 'ts']
ratings = pd.read_csv('ml-1m/ratings.dat', sep='::', engine='python',
                      names=cols)


# ---- ส่วน A: ทำนายคะแนน (แบ่งสุ่ม 80/20) ด้วย SVD ของ Surprise
rng = np.random.default_rng(2026)
test = rng.random(len(ratings)) < 0.2
uir = ['user', 'item', 'rating']
reader = Reader(rating_scale=(1, 5))
train = Dataset.load_from_df(ratings.loc[~test, uir], reader)
train = train.build_full_trainset()
svd = SVD(n_factors=64, reg_all=0.05, lr_all=0.005, n_epochs=30,
          random_state=2026)
svd.fit(train)
pairs = list(ratings.loc[test, uir].itertuples(index=False, name=None))
print('SVD RMSE', accuracy.rmse(svd.test(pairs), verbose=False))


# ---- ส่วน B: จัดอันดับ (ซ่อนเรื่องล่าสุดที่ชอบ) ด้วย EASE และ iALS
pos = ratings[ratings.rating >= 4].sort_values(['user', 'ts'])
last = pos.groupby('user').tail(1)
tr = pos.drop(last.index)
uid = {u: k for k, u in enumerate(sorted(pos.user.unique()))}
iid = {i: k for k, i in enumerate(sorted(pos.item.unique()))}
rows = tr.user.map(uid)
cols_ = tr.item.map(iid)
X = sp.csr_matrix((np.ones(len(tr), np.float32), (rows, cols_)),
                  shape=(len(uid), len(iid)))
tu = last.user.map(uid).to_numpy()
ti = last.item.map(iid).to_numpy()



def hr10(S):
    S = np.asarray(S, float)
    S[X[tu].nonzero()] = -np.inf          # ตัดเรื่องที่เคยชอบออก
    target = S[np.arange(len(tu)), ti][:, None]
    rank = (S > target).sum(axis=1)
    return float(np.mean(rank < 10))
```

```
G = (X.T @ X).toarray()
P = np.linalg.inv(G + 500 * np.eye(G.shape[0]))
B = -P / np.diag(P)
np.fill_diagonal(B, 0)
print('EASE HR@10', hr10(X[tu] @ B))

als = AlternatingLeastSquares(factors=64, regularization=0.05,
                              alpha=10.0, iterations=15, random_state=1)
als.fit(X, show_progress=False)
print('iALS HR@10', hr10(als.user_factors[tu] @ als.item_factors.T))
```

Lab 5 ในภาคผนวก 5ก รวมทุกวิธีเป็นสายงานเดียวแปดส่วน ตั้งแต่สถิติพื้นฐานจนถึงผลของเมล็ดสุ่ม ตารางที่ 5.35 สรุปงานและผลที่ผู้อ่านต้องได้ในแต่ละส่วน

ตารางที่ 5.35 โครงสร้างของ Lab 5

ส่วน	งาน	ผลที่ต้องได้	เวลา
1 สถิติ	μ, ความหนาแน่น, จำนวนรายการร่วม	ตัวอย่างการคำนวณ 5.2 และ 5.3	10 นาที
2 อิงความจำ	ความคล้ายของ u1, User-kNN, Item-kNN, Item-kNN โดยปริยาย	ตารางที่ 5.9, 5.10, 5.14	25 นาที
3 ค่าเอนเอียงและ MF	$\mu + b_u + b_i$, kNN-Baseline และ MF ด้วย SGD	ตัวอย่างการคำนวณ 5.17, 5.18, 5.22	20 นาที
4 SVD และ ALS	SVD 3×3 และแก้ $\mathbf{p}_{u1}$ ด้วยรูปปิด	ตัวอย่างการคำนวณ 5.20 และ 5.23	15 นาที
5 ข้อมูลโดยปริยาย	WRMF, BPR, EASE	ตัวอย่างการคำนวณ 5.29 และหัวข้อ 5.9–5.10	20 นาที
6 สุ่มเชิงลบ	P_pop ที่ β = 0.75	ตารางที่ 5.23	5 นาที
7 ประเมินผล	LOO 20 กรณี ทุกวิธี	ตารางที่ 5.25	20 นาที
8 ไฮเปอร์พารามิเตอร์	α ของ WRMF และเมล็ดสุ่มของ MF	ภาพที่ 5.17 และตารางที่ 5.27	15 นาที
เสริม ข้อมูลจริง	Surprise และ implicit กับ MovieLens-1M	โปรแกรม 5.19	60 นาที

ประเด็นสำคัญ ข้อแนะนำในการทำ Lab

- รันบน Google Colab หรือเครื่องที่มี Python 3 และ NumPy โปรแกรม Lab 5 ใช้เวลาไม่ถึง 10 วินาที ส่วนโปรแกรมไลบรารีต้องติดตั้ง `pip install scikit-surprise implicit` ก่อน
- ทุกส่วนใช้เมล็ดสุ่มคงที่ ผลจึงต้องตรงกับตัวเลขในบทถึงทศนิยมตำแหน่งที่สี่ ถ้าไม่ตรงให้ตรวจลำดับของรายการและการจัดการค่าเสมอ
- ลองเปลี่ยนไฮเปอร์พารามิเตอร์ทีละตัวตามแบบฝึกหัดระดับที่ 2 แล้วบันทึกผลเป็นตาราง

เอกสารอ้างอิง

Abdollahpouri, H., Burke, R., & Mobasher, B. (2017). Controlling popularity bias in learning-to-rank recommendation. In Proceedings of the 11th ACM Conference on Recommender Systems (pp. 42–46).

Aggarwal, C. C. (2016). Recommender systems: The textbook. Springer.

Bennett, J., & Lanning, S. (2007). The Netflix Prize. In Proceedings of KDD Cup and Workshop 2007.

Cremonesi, P., Koren, Y., & Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. In Proceedings of the 4th ACM Conference on Recommender Systems (pp. 39–46).

Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. Psychometrika, 1(3), 211–218.

Ferrari Dacrema, M., Cremonesi, P., & Jannach, D. (2019). Are we really making much progress? A worrying analysis of recent neural recommendation approaches. In Proceedings of the 13th ACM Conference on Recommender Systems (pp. 101–109).

Frederickson, B. (2016). Implicit: Fast Python collaborative filtering for implicit datasets [Computer software]. https://github.com/benfred/implicit

Funk, S. (2006, December 11). Netflix update: Try this at home. https://sifter.org/~simon/journal/20061211.html

Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using collaborative filtering to weave an information tapestry. Communications of the ACM, 35(12), 61–70.

Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. ACM Transactions on Interactive Intelligent Systems, 5(4), Article 19.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T.-S. (2017). Neural collaborative filtering. In Proceedings of the 26th International Conference on World Wide Web (pp. 173–182).

Herlocker, J. L., Konstan, J. A., Borchers, A., & Riedl, J. (1999). An algorithmic framework for performing collaborative filtering. In Proceedings of the 22nd International ACM SIGIR Conference (pp. 230–237).

Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In Proceedings of the 8th IEEE International Conference on Data Mining (pp. 263–272).

Hug, N. (2020). Surprise: A Python library for recommender systems. Journal of Open Source Software, 5(52), Article 2174.

Jinopong, V., Praserttitipong, D., & Bootkrajang, J. (2022). Multiple latent space ensemble for matrix factorization based collaborative filtering. ECTI Transactions on Computer and Information Technology, 16(4), 469–477.

Koren, Y. (2008). Factorization meets the neighborhood: A multifaceted collaborative filtering model. In Proceedings of the 14th ACM SIGKDD International Conference (pp. 426–434).

Koren, Y. (2010). Factor in the neighbors: Scalable and accurate collaborative filtering. ACM Transactions on Knowledge Discovery from Data, 4(1), Article 1.

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30–37.

Krichene, W., & Rendle, S. (2020). On sampled metrics for item recommendation. In Proceedings of the 26th ACM SIGKDD International Conference (pp. 1748–1757).

Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet Computing, 7(1), 76–80.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. In Advances in Neural Information Processing Systems 26 (pp. 3111–3119).

Praserttitipong, D., & Sophatsathit, P. (2012). A distributed recommender agent model based on user's perspective SVD technique. International Journal of Digital Content Technology and its Applications, 6(10), 108–117.

Praserttitipong, D., & Sophatsathit, P. (2013). A loosely collaborative dependency framework for a fast adaptive agent model using extended RSVD. International Journal of Artificial Intelligence, 11(A13), 1–19.

Praserttitipong, D., & Sophatsathit, P. (2014). An agent model for information filtering using revolutionary RSVD technique. Chiang Mai Journal of Science, 41(5.2), 1429–1438.

Praserttitipong, D., & Srisujjalertwaja, W. (2020). A collaborative filtering model based on matrix factorization and trust information. In Proceedings of the 17th International Joint Conference on Computer Science and Software Engineering (JCSSE 2020) (pp. 177–182).

Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. In Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (pp. 452–461).

Rendle, S., Krichene, W., Zhang, L., & Anderson, J. (2020). Neural collaborative filtering vs. matrix factorization revisited. In Proceedings of the 14th ACM Conference on Recommender Systems (pp. 240–248).

Rendle, S., Krichene, W., Zhang, L., & Koren, Y. (2022). Revisiting the performance of iALS on item recommendation benchmarks. In Proceedings of the 16th ACM Conference on Recommender Systems (pp. 427–435).

Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. In Proceedings of the 1994 ACM Conference on Computer Supported Cooperative Work (pp. 175–186).

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th International Conference on World Wide Web (pp. 285–295).

Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. In Proceedings of the 25th International ACM SIGIR Conference (pp. 253–260).

Srisujjalertwaja, W., & Praserttitipong, D. (2015). A hybrid of singular value decomposition and regression analysis collaborative filtering with linear incremental update method. Journal of Theoretical and Applied Information Technology, 80(2), 247–256.

Steck, H. (2019). Embarrassingly shallow autoencoders for sparse data. In Proceedings of The World Wide Web Conference (pp. 3251–3257).